dsh-superagent multi-channel

English documentation set

Version 2.4.0 · 2026-11-14 · MIT

Generated from the English documents under docs/; screenshots are redacted.

Contents

Project overview

Documentation overview

000 Philosophy

001 Domain model

002 Authorization

003 Execution

004 Prompts

005 Panel

006 Deployment

007 Architecture

008 Cookbook

Extensions: the multi-channel layer

Capability matrix

ADR-005 WeChat backend

ADR-006 Health key compatibility

ADR-007 WeChat capability alignment

ADR-008 Outbound attachments

ADR-009 Projex feedback and parity

ADR-010 Yunxiao sign-in and status

ADR-011 Aligning WeChat with Feishu

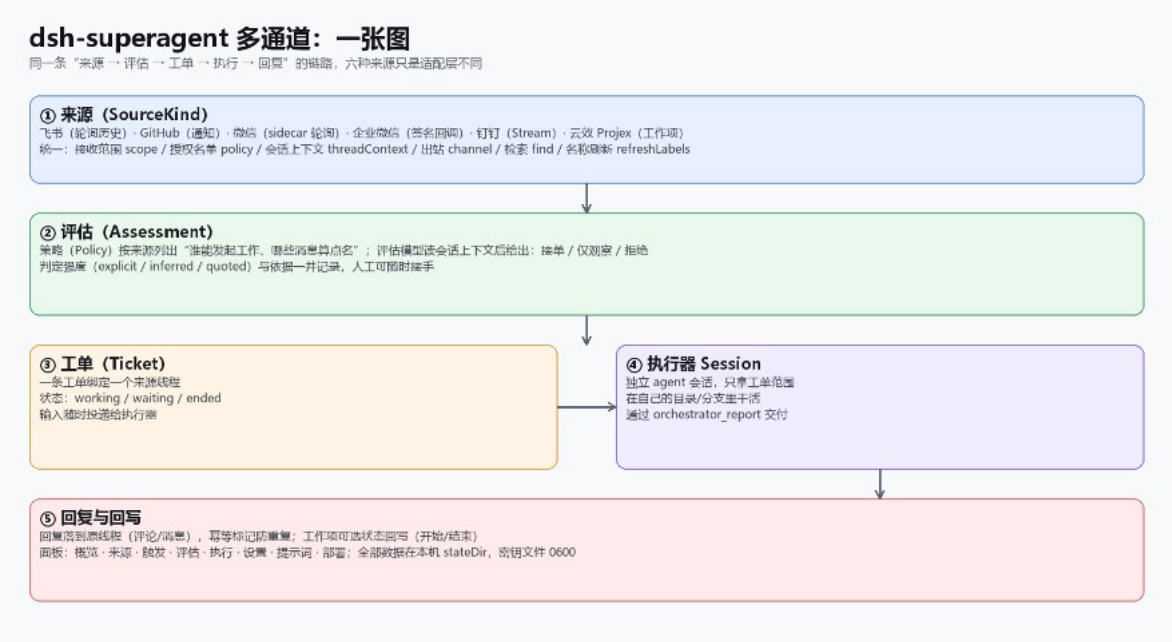
ADR-012 Aligning WeCom with Feishu

dsh-superagent multi-channel

A DSH plugin that wires Feishu, GitHub, WeChat, WeCom, DingTalk and Alibaba Cloud Yunxiao Projex into one assess → ticket → execute → reply loop.

Version `2.4.0` · 102 test files / 1,624 passing · MIT · installs beside the official `dsh-superagent`

> *An extension of the official open-source superagent: the two official sources (Feishu, GitHub) stay as they are, and*
> *WeChat, WeCom, DingTalk and Yunxiao Projex are **added** and aligned to the same capability surface (reception scope,*
> *authorization list, thread context, worker reads, search, label refresh, reply and status write-back). See the*
> *capability matrix.*



Architecture: one loop, six sources

In one sentence

Someone mentions the bot or messages it on any of six channels → the plugin reads the context under the policy and decides whether to take it → an **isolated executor session** works in its own directory → the result goes back to the **same thread**, and a work item's board status can follow along.

The life of one message



Lifecycle

- **Trigger** — all six sources produce one `Trigger` shape (thread, sender, structured `@`, native id).
- **Assess** — the policy names "who may originate work" and "what counts as addressed"; the model reads the thread and

answers accept / observe / refuse, recording the evidence strength.

- **Ticket** — one ticket binds one source thread; later messages arrive as inputs for the executor.
- **Execute** — the executor session sees only its ticket's scope, works in its own directory/branch, delivers with

`orchestrator_report`.

- **Reply & write back** — the reply lands in the original thread (comment/message, idempotency-marked); a work item's status

can move to "in progress" and "done". Everything is in the local `stateDir`; a person can take over at any step.

Three real scenarios

1. **A Yunxiao defect** — someone @-mentions the bot on a work item ("login 502 intermittently") → the assessment reads the

item and its comments → a ticket opens → the executor fixes it in an isolated directory → the result is a comment, and the board moves from "in progress" to "to test".

2. **A WeChat ops group** — "@bot can you check the expiring certificate?" → the reply goes only into that thread.
3. **A WeCom direct message** — "change this copy" → the reply returns to the same conversation, split into ordered parts,

with a file delivered as a comment/link when needed.

What decides it becomes work?



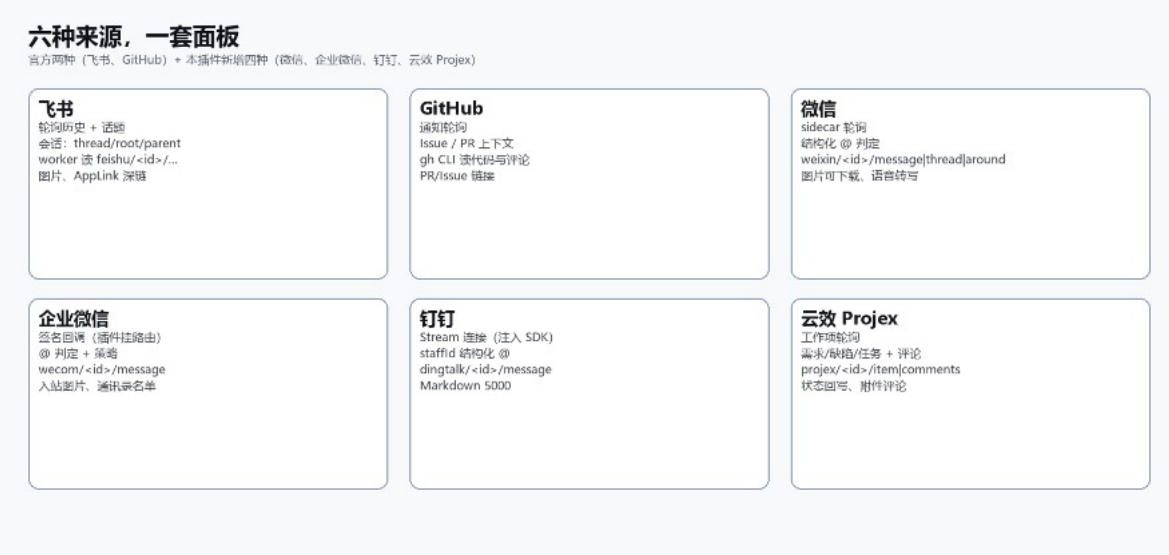
Gates and assessment

Four gates (enabled source → allowed list → structured mention → assessment). Failing any one only *observes* the message: no ticket, no reply, no remote change.

Feature checklist

Stage	What exists
Receive	six sources normalised to one Trigger; groups/DMs; structured @; optional images; dedupe by platform id
Authorize	per-source "who may originate work" + admission table (listed/unlisted × @/un-@/DM)
Assess	rule switches, mode (off/observe/auto), deadline, model and effort; every decision records strength and evidence
Ticket	one ticket per thread; inputs delivered any time; interruptible, supplementable, human-takeover
Execute	isolated executor session and directory/branch; structured report; files returned
Reply	back to the original thread; idempotent marker; optional work-item status write-back; visible retries
Read originals	per-source worker resources, read-only
Panel	overview/sources/triggers/assessment/execution/ settings/prompts/deployment; prompts and six skills overridable
Security	local state, 0600 secrets, reply-only, platform limits stated

Six sources, one panel



Six sources

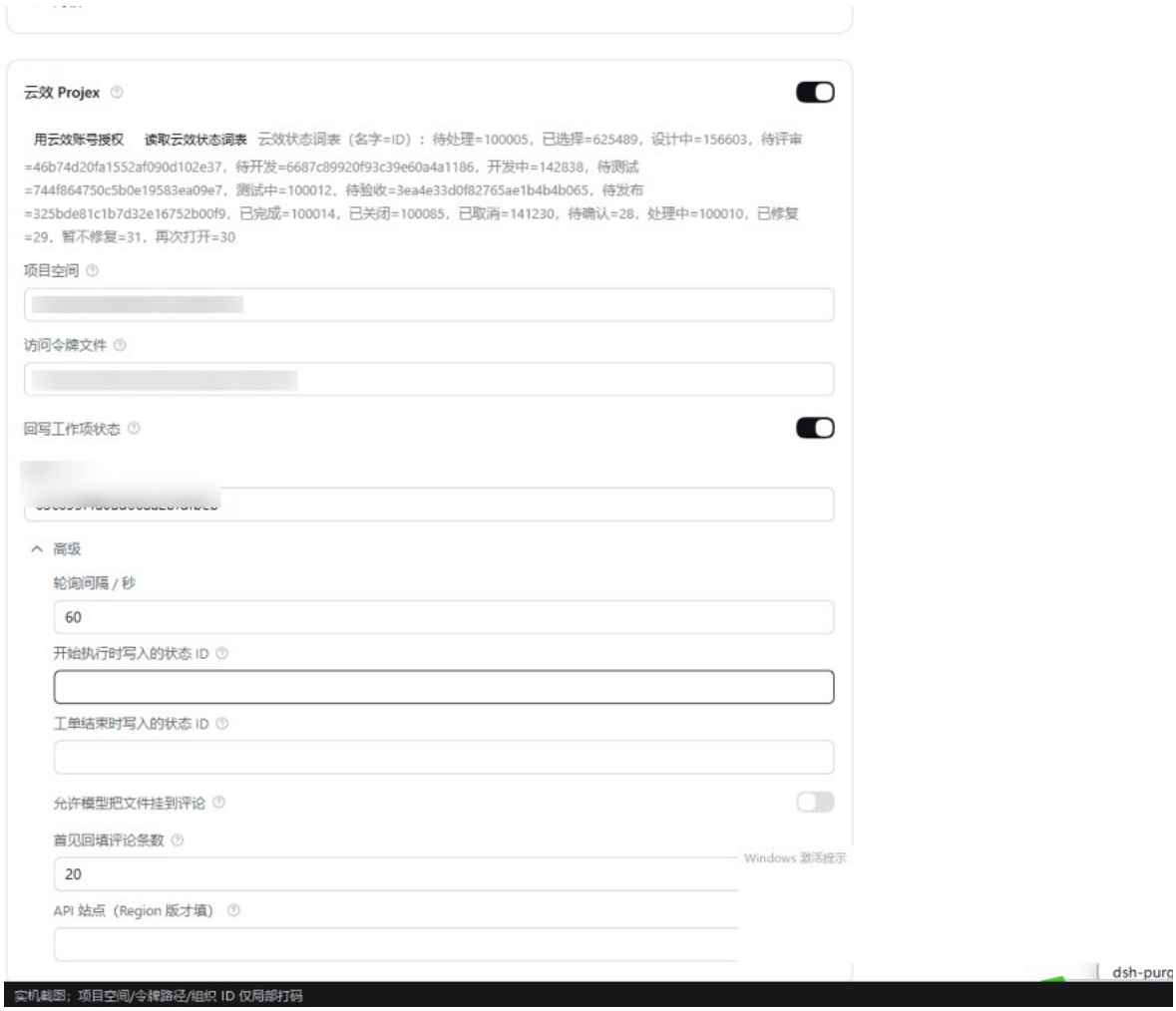
Source	Reception	Highlights
Feishu (official)	history polling	topic/reply chains, AppLink deep links, images, worker reads `feishu//...`
GitHub (official)	notification polling	issue/PR context, `gh` for code, PR/issue links
WeChat (added)	sidecar polling	structured `@`, voice transcripts, downloadable images, worker reads `weixin//message·thread·around`, post-reattach catch-up
WeCom (added)	signed callback (plugin mounts `/wecom/`)	`@` judgement, inbound images, contact-list names, worker reads `wecom//message`
DingTalk (added)	Stream (deployment injects the SDK connection)	staffId `@`, Markdown 5000, worker reads `dingtalk//message`
Yunxiao Projex (added)	work-item polling	requirement/bug/task + comments, status write-back, attachment comments, worker reads `projex//item·comments`

Panel tour

The plugin has its own sidebar entry, "Superagent multi-channel", beside the official one. **The screenshots below are real captures; project IDs, token paths, organisation IDs, names and chat names are masked.**



Policy and assessment



Yunxiao Projex source card



Prompts

Install and first run



Install flow

4. **Install** the bundle from the plugin manager (directory, npm package or tarball), or from the market once listed.
5. **Configure**: leave `stateDir` empty to adopt an existing `~/.dsh/state/superagent` (no migration); two superagent

installs each need their own directory.

6. **Refresh** the web page and open "Superagent multi-channel".
7. **Connect a source**: fill its card and use the card's Check / Read action before enabling it.

More in deployment; per-channel steps in EXTENSIONS.md.

Security and boundaries



Security boundaries

- **Reply only**: no broadcasts, no group invites, no friend requests.
- **Secrets stay local**: tokens/keys live in 0600 files rather than settings; screenshots and logs never print them.
- **Idempotency and dedupe**: comments carry a marker, callbacks dedupe by platform message id; WeCom and DingTalk have no

server-side idempotency key and say so in `facts`.

- **No faked platform behaviour**: WeChat/WeCom/DingTalk have no message deep link, the two push backends have no history to

page, and DingTalk's stream needs an injected SDK connection. See the capability matrix.

FAQ

Does it auto-reply to everyone? No. Only threads the assessment accepted get a reply; a message from an unlisted person is observed at most. **Where does data go?** Everything stays in the local `stateDir`; data reaches a platform only when the plugin calls that platform's API to read or reply. **Can it run beside the official superagent?** Install both, but enable one at a time (a Host accepts one Source kind per id); each keeps its own `stateDir`. **Does upgrading lose data?** No. `stateDir` is unchanged and, since 2.3.0, the agent presets keep the shared ids so old sessions resume. **A source receives nothing?** See the troubleshooting table in docs/008-cookbook.md: WeCom callback URL and token, DingTalk's injected stream connection, the WeChat bridge `/health`.

Documentation map

Document	What it covers	Length / Audience
docs/README.en-US.md	Reading order and the full map (中文: docs/README.md)	46 lines · start here
docs/000-philosophy.md	Why assess → ticket → execute	35 lines · everyone, ~10 min
docs/001-domain.md	Trigger / Thread / Ticket / Assignment	114 lines · developers, integrators
docs/002-authorization.md	Policy, allowed lists, addressed detection	157 lines · admins, security review
docs/003-execution.md	Executor sessions, directories, delivery	205 lines · developers, operators
docs/004-prompts.md	Prompts and skills	98 lines · prompt maintainers
docs/005-panel.en-US.md	The panel page by page (with screenshots)	60 lines · everyday users
docs/006-deployment.md	Deployment, settings, upgrades, coexistence	298 lines · deployers, operators
docs/007-architecture.md	Architecture, module and test tables	713 lines · maintainers, contributors (as an index)
docs/008-cookbook.en-US.md	Recipes and troubleshooting	51 lines · on-call, troubleshooting
docs/extensions/capability-matrix.md	Six-source comparison and platform limits	42 lines · selection, review
docs/extensions/	ADR-005 … ADR-012 (each in English and Chinese)	40–356 lines each · decision history
docs/gallery.en.html	Illustrated preview (中文: docs/gallery.html)	image pages · demo, review

Develop and test

```
pnpm typecheck   # tsc --noEmit
pnpm test        # vitest run --maxWorkers=2
pnpm build       # tsdown, eight entries
```

Latest full regression (2026-11-14, 2.4.0): `tsc` 0; **102 files passed / 5 skipped, 1,624 passed / 41 skipped, 0 failed**; `tsdown` build complete.

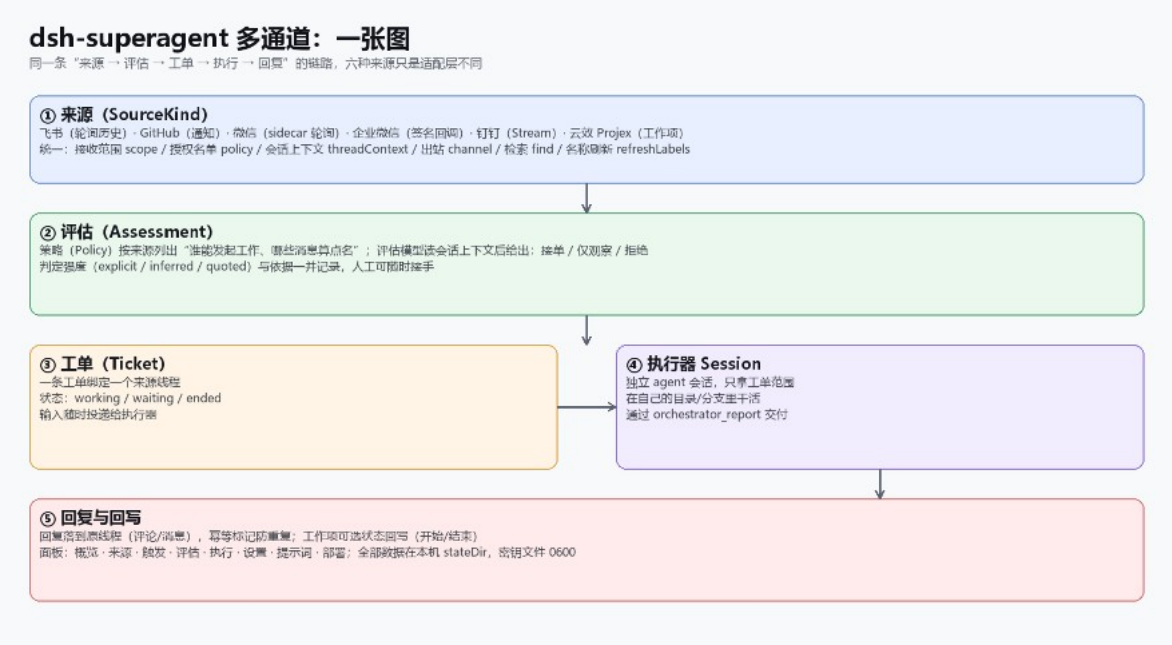
License

MIT, with the upstream `dsh-superagent` notices kept. WeChat UI automation is a third-party tool without Tencent's authorization and may break at any time; use it within ADR-005's boundary (reply only when mentioned, no broadcasts, no bulk friend requests, one account online at a time).

Screenshots are real captures with sensitive fields (project/organisation IDs, token paths, names, chat names, session titles) masked; the architecture and lifecycle images are diagrams drawn from the current implementation.

Documentation Overview

This directory describes what `dsh-superagent multi-channel` (2.4.0, reviewed 2026-11-14) **actually provides** today. **Every document exists in both Chinese and English**: Chinese files carry `.zh-CN.md`, English files `.en-US.md`, and the base name stays with the original; every page has a language switch at the top. Every page reflects the current implementation; platform-limit topics are itemized in capability-matrix.md as "not possible" rather than omitted.



Architecture

Reading order

#	English	中文
1	000-philosophy.md	000-philosophy.zh-CN.md
2	001-domain.md	001-domain.zh-CN.md
3	002-authorization.md	002-authorization.zh-CN.md
4	003-execution.md	003-execution.zh-CN.md
5	004-prompts.md	004-prompts.zh-CN.md
6	005-panel.en-US.md	005-panel.md
7	006-deployment.md	006-deployment.zh-CN.md
8	007-architecture.md	007-architecture.zh-CN.md
9	008-cookbook.en-US.md	008-cookbook.md
—	../EXTENSIONS.md	../EXTENSIONS.zh-CN.md
—	extensions/capability-matrix.md	extensions/capability-matrix.zh-CN.md
—	extensions/ (each ADR has `.en-US.md` and `.zh-CN.md`)	same files, Chinese variants
—	gallery.en.html	gallery.html

#	English	中文
—	distribution/README.en-US.md	distribution/README.md

At a glance



Lifecycle

- **Unified input**: all six Sources normalize into the same `Trigger` (Thread, initiator, structured @, raw ID, optional image).
- **Unified decision**: Policy configures lists and admission tables per Source; the Assessment reads context and then returns accept / observe only / reject, recording strength and grounds.
- **Unified execution**: one Ticket, one Executor Session, an independent directory/branch, structured delivery.
- **Unified feedback**: replies land back in the original Thread, comments carry an idempotency marker; work items can have their status written back.

Screenshot conventions

- `images/panel-*.png` are **real screenshots**; sensitive information (project/organization ID, token paths, names, group names, Session titles) is redacted; a few images carry red callout boxes.
- `images/architecture.png`, `lifecycle.png`, `sources.png`, `install.png` are **diagrams** drawn to the current implementation.
- Put new screenshots in `docs/images/`, named `panel-.png`, and state "real / diagram" in the body caption.

Documentation maintenance conventions

- When changing the capability surface (new Source, field, action), update in step: this page, the page descriptions in 005, the module/test tables in 007, capability-matrix, and CHANGELOG.
- Test counts and commands live only in README and 007, to avoid drift in multiple places.

000 Philosophy

dsh-superagent is a framework, and also a way of working with an agent. Its core principle:

Take as little human attention as possible, and give as much value as possible.

Every part of the project is judged by the attention it costs. A message, a notification, a question, a setting, a panel element and a paragraph of documentation each cost someone's attention, and each has to give more value than that, or it is removed.

What follows for the agent

- **It acts only when asked, or when a written rule allows it.** A request counts only when it addresses the agent directly and comes from a person the team allowed: one on an allowed list, or anyone in a conversation the team opened to requests. Work nobody asked for needs a rule the team wrote, and starts in a mode that only records what the agent would have done.
- **It speaks only when a person must act or can use a result.** It confirms a request once and names what it will do. It stays quiet while it works. It writes again when a result is ready, when it needs a decision, or when it cannot finish. Messages are short and go to the conversation where the request was made.
- **It asks one concrete question at a time,** in a structured form, so the question reaches the person instead of staying inside a long report.
- **It stops by impact, not by effort.** A local change goes as far as a reviewable result. A change that needs a design decision stops with a clear question.
- **It reports honestly.** A draft is marked unfinished, a failure is reported as a failure, and a limit is never bypassed to avoid asking.

What follows for the design

- **Few concepts.** The design has only its domain concepts, such as a Source, and their concrete implementations, such as GitHub and Feishu. A business need becomes an attribute of one of them, or a prompt the agent follows. The design does not add a concept for it.
- **Few settings.** A default replaces a setting when one value suits almost every team. Behaviour that a team describes in words is prompt text, not a setting.
- **A panel that shows only what needs a person.** It lists first what waits for a person. A healthy state takes no space.

- **Documents read in order.** Each numbered document builds only on the ones before it, so a reader never needs a concept that has not been introduced yet.

Responsibility

The agent comments, opens pull requests and messages people under accounts your team sees every day. Every message it sends costs your colleagues' attention, and you are responsible for it. The more the agent can do, the more care its operator owes the team:

- **Start narrow.** Begin with a short allowed list and with proactive work only recorded. Widen both once what the agent would have done looks right.
- **Tune for less noise.** When the agent is noisy, make its prompts shorter and stricter, or pause it. Do not add more messages to explain the noise.
- **Do not use it to produce volume.** Mass pull requests, broadcast messages and busywork cost the team more than they give.
- **Own what it says.** Read what the agent posts in your name, and correct it in the prompts when it is wrong.

The goal is the right help at the right time, and nothing at any other time.

001 Domain model

This document names the concepts of dsh-superagent and the rules each one keeps. The code, the tools the agents call, the panel and the other documents use these terms and no others. Following 000 Philosophy, the model holds only concepts the work needs, and a business need becomes an attribute of one of them.

Flow

Work moves from Trigger to Eval to execution. A Source produces Triggers. A Trigger fans out as context to the Tickets subscribed to its Threads. A person's message that reached a Ticket this way stays with that Ticket; every other Trigger enters Eval, which decides whether it asks for new work. When Eval accepts, it creates a Ticket with a Mandate. An Assignment hands the Ticket to an Executor, and results go back to the Source through a Channel.

```
flowchart LR
  S[Source] -->|Trigger| T[Thread]
  T -->|batch| E[Eval]
  T -->|Subscription fan-out| K[Ticket]
  E -->|accept with Mandate / stop| K
  E -->|ignore / unavailable| X[recorded]
  P[Policy change] -->|notice| K
  K -->|Assignment| W[Executor]
  K -->|Channel| S
```

Concepts

Source, Thread, Eval, Mandate and Subscription form the core domain; the rest supports them.

Concept	Definition	Rules it keeps	Why it exists
Source	A stream of events. A Source kind is a reusable implementation; a Source instance is one configuration of a kind: whether it runs and what it listens to	Holds its read position and its own bot identity. Defines what counts as an event, including conditions that only hold over time. Drops the bot's own messages and Triggers outside what it listens to before they enter the domain. Translates each Trigger into the Threads it belongs to and the Channel addresses to reply on	A restart must not replay events

Concept	Definition	Rules it keeps	Why it exists
Trigger	One immutable event with the Source's own metadata, such as its actor	Belongs to one or more Threads. Accepting a Trigger twice creates one Ticket	The unit of audit and of idempotence
Thread	Triggers in time order; the Source defines which Triggers form one Thread	At most one Eval runs on it at a time. It may have several Tickets. It holds its Subscriptions, the running Eval and its Decision history	Gives Eval a consistent view and its context
Correlation	The relation between Threads that one Trigger belongs to	Adds context; never routes	Lets Eval follow related evidence
Channel	The write side of a Thread; the Source defines its addresses	A Thread may have several Channels; a Ticket holds a set of them	Acknowledgements, replies, pull requests and resolved reviews are all writes
Policy	The versioned assessment rules and each Source's part: who may originate work through that Source, in that Source's own identities, how the list is synchronized, and which bots may make candidates	The Policy version covers the rules, the accept mode and the Source parts. A change that bears on a live Ticket, such as a change to a rule it was accepted under, is delivered to its Assignment; it does not revoke Mandates. Identities on different Sources are never mapped to each other	When the rules change, a running Eval must be discarded
Eval	The judgement on a batch of Triggers, in two layers: gate and assessment	Input: the Trigger batch, a Thread snapshot, every Ticket on the Thread including ended ones, and the current Policy. Output: a Decision. Holds no state	Decides whether new work starts, goes on or stops

Concept	Definition	Rules it keeps	Why it exists
Mandate	What an accept grants: the Policy version, the scope, the strength, the matched rules, the sender's identity on that Source and how the Source received the Trigger	Immutable; wider scope means a new Ticket. A Ticket's end does not end it: only a person's stop withdraws it, and a Policy that no longer holds it revokes it. Revalidated against the current Policy before an ended Ticket resumes	States why the work may be done, and how far it may go
Ticket	A unit of authorized work	Only Eval creates one. Refers to its Mandate; holds its Channels and Inputs. Goes open → ended, and back to open when a person's message on one of its Threads resumes it, unless a person stopped it. Defines its own state instead of following the Executor's	Owns the execution and its reports
Assignment	The binding of a Ticket to one Executor instance	A Ticket has several over time. Goes pending → running → ended independently of the Ticket. Only its final report may end the Ticket on the Executor's side	The Executor is outside the domain, but who executes must be inside it

Concept	Definition	Rules it keeps	Why it exists
Subscription	A many-to-many relation between Tickets and Threads, held by the Thread	A matching Trigger fans out to every live subscribed Ticket, with no check of who sent it; a person's message that reaches a Ticket this way takes no Eval. A Thread an Executor follows for context reaches its Ticket the same way, but lends it no authority and does not resume it. An ended Ticket keeps its Subscriptions, including that of a pull request that closed or merged: a person's message resumes it under the same Mandate, with no Eval or Decision; messages from bots and CI do not reach it, since its Executor reads its Threads again when it resumes. A Ticket a person stopped receives nothing. The main Session attaches a Thread to an existing Ticket only when its message points at that Ticket explicitly. A person can drop every Subscription on a Thread; an Executor can drop only its own Ticket's Subscription to a Thread it finds unrelated, except the Thread its Ticket started on and a Thread a Source made it follow, such as its pull request's	Later discussion reaches the Executor without anyone mentioning the agent again

A person is not a domain concept. A Trigger carries its actor's identifier as the Source gives it. An allowed list is a Source's part of the Policy, in that Source's identities. That one person has accounts on several Sources is prompt text that the agents read when they need it.

The Executor, with its Sessions and directories, is outside the domain and connects through Assignments.

Aggregates: a Source holds its read position. A Thread holds its Subscriptions, its running Eval, which runs serially and merges Triggers that arrive meanwhile into the next batch, and its Decision history. A Ticket holds its Assignments, Channels and Inputs, and refers to Decisions by id.

Rejected concepts

Rejected	Reason
Principal or Membership as an entity, or mapping identities across Sources	A person is Trigger metadata; an allowed list is a Source's part of the Policy; a cross-Source mapping is prompt text
Eval scheduling its own delayed Triggers	A condition that needs waiting is an ordinary Source, so Eval never depends back on Sources
Waiting or revoked Eval states	Waiting belongs to Sources; an Eval that runs out of time queues its Triggers again, or ends `unavailable`
At most one live Ticket per Thread	One Thread can carry several independent pieces of work; the assessment avoids duplicates from the Ticket list
An "append to Ticket" Decision	Appending is Subscription fan-out and needs no Eval
Admission checks on Subscriptions	A Subscription covers the whole Thread; the Mandate separates authorization from context
Correlation that routes	Subscriptions route; Correlation only adds context
Ticket state that follows the DSH Session	A Ticket can be handed off, resumed, or run by several Executors, so it defines its own state
An Executor declaring that a person asked it to stop	Only Eval decides the strength of an end
Cancelling running Tickets on a Policy change	A cancelled change leaves half-finished work; the change is recorded and delivered instead
Locking all of an Executor Session's tools when it fails or blocks	It would refuse a person working directly in the Session, and the report after recovery; pausing automatic input is enough
Inferring "waiting on a person" from a report's free text	The classification needs structured facts; the Executor asks through a structured field

What is reusable

The concepts above and their rules are reusable. The Source implementations, the Policy contents, the Executor and the Channel implementations belong to a deployment. Each Source

keeps its own words internally, such as message, chat or notification, and translates them into Triggers.

Part	In this repository
Source	Feishu: live events, history recovery, polling of subscribed topics. GitHub: notifications, subscribed Issues and pull requests, CI changes, pull requests merged or closed, and, where switched on, every new Issue, pull request and comment in the repository and the default branch's CI
Policy	Assessment rules, and each Source's allowed list, its sync, its bot list and its admission table
Executor	A DSH Executor Session and a directory per Ticket, in an execution environment whose variables the Executor and the Sources register
Channel	Feishu topic replies, GitHub comments and review thread replies

Glossary

A reference list of the terms this and the later documents use, with their Chinese names; the later documents explain the terms not introduced above. The panel shows one language at a time; its (?) button opens this glossary with both languages. The messages agents send to people on GitHub and Feishu use plain language, not these terms.

Term	中文	Meaning
Agent	智能体	The automated team member dsh-superagent runs, acting on Sources through identities such as a GitHub account and a Feishu bot
Main Session	主 Session	The DSH Session that evaluates Triggers, accepts and dispatches Tickets, and replies; `orchestrator` in code and prompts
Executor Session	执行器 Session	The DSH Session an Executor runs for one Ticket; `worker` in code and prompts
Source	来源	A stream of events: a kind that a plugin provides, plus one instance configuration
Trigger	触发	One immutable event from a Source
Thread	线程	Triggers in time order; the unit within which Evals run one at a time

Term	中文	Meaning
Correlation	关联	The relation between Threads that share a Trigger; it adds context only
Channel	渠道	The write side of a Thread
Policy	策略	The versioned rules for accepting work, and each Source's part of it, such as who may ask for work there
Policy version	策略版本	The identifier of one combination of rules, Source parts and accept mode; every Mandate records it
Eval	评估	The process that turns a batch of Triggers into a Decision
Gate	准入	The deterministic first layer of Eval: who may originate work, and with what strength
Assessment	研判	The model's judgement during Eval
Decision	决定	The result of Eval
Mandate	授权	What an accept grants: strength, scope and the Policy version it relied on
Ticket	工单	A unit of authorized work
Assignment	派工	The binding of a Ticket to one Executor instance
Subscription	订阅	The relation between a Ticket and a Thread
Fan-out	扇出	Delivering Triggers of a subscribed Thread to a Ticket as context
Executor	执行器	The port that runs Assignments: one Executor Session and one directory per Ticket
Execution environment	执行环境	Where Executor Sessions and the prepare command run: variables that the Executor and the Sources register, each with a key, a value and a description
Report	报告	The state an Assignment reports to its Ticket
Attention	待人工决定	A question on a Ticket that waits for a person's answer
Disposition	去向	The record of a Trigger being filtered, evaluated or fanned out

Term	中文	Meaning
Input	输入	Context delivered to a Ticket: fanned-out Triggers, Policy changes, resumes and text
Scope	范围	The work a Mandate allows
Strength	强度	How strong the basis of a Mandate or an Eval is

Group	Values
Decision	accept 接单 · stop 停止 · ignore 忽略 · unavailable 未完成
Strength	explicit 明确 · inferred 推断
Assignment	pending 待启动 · running 执行中 · ended 已结束
Report	progress 进展 · waiting 等待 · ready 就绪 · blocked 受阻 · failed 失败 · dropped 无需处理

002 Authorization

Authorization determines who may start work and what that work may include. This document builds on 001 Domain model.

The host enforces allowed lists, rules and Mandates. Messages, comments and quoted documents are inputs to a decision; they cannot change those permissions.

Two paths for a Trigger

Eval: deciding whether to start work

The Source records whether a message addresses the agent: `mention` for a mention of the bot or agent account, and `direct` for a direct message. The gate checks the sender and the Source's Policy:

- An allowed person who addresses the agent makes an explicit request.
- A person's follow-up on a Thread with Tickets follows the rules below.
- Other human messages use the admission table for their receiving scope.
- A bot on the Source's bot list can produce an inferred candidate, never an explicit request. Other bots are not admitted.

Subscriptions: continuing existing work

Every message on a subscribed Thread reaches its subscribed Tickets as context, including messages from bots and CI. This delivery bypasses the gate and grants no new authority. Each Ticket remains limited by its Mandate.

A person's message on a Ticket's own Thread belongs to that Ticket's Executor to decide. The same applies when the main Session attaches a message to a Ticket. While an Executor owns the message, it takes no Eval and gets no Decision. Each Executor either handles the request within its Ticket's scope or hands it back for routing elsewhere.

Ownership continues while the Ticket is live, even if a person removes its Subscription. It also covers an ended Ticket that was not stopped by a person and has the message waiting to resume it.

The main Session receives a follow-up notice. While an Executor owns the message, the main Session may only stop work on the person's request; accept, attach, ignore and requeue are refused. A stop still passes through the gate with the follow-up strength described below.

Handing back and releasing follow-ups

An Executor hands a message back once when it asks for work beyond the Ticket's scope or belongs to another Ticket. The Ticket keeps its Subscription. If an attached Thread is unrelated, the Executor hands back the messages it owns there.

A message goes to a fresh Eval only after no Ticket still owns it. Its candidate includes the Executors' reasons. The gate judges it as an ordinary message on that Thread, and the main Session can accept it, attach it elsewhere or ignore it. Handing back a message the Ticket did not own, such as a bot message or one a person already sent to Eval, queues nothing.

When a Ticket ends, it releases follow-ups its Executor did not read or take up. These include messages never delivered to the Executor and messages delivered after its last Report or turn end, provided it neither acknowledged, answered nor handed them back. Each released message goes to Eval when no other Ticket owns it. A stop ends only the stopped Ticket's work.

A stopped Ticket does not send its stop message to Eval again unless another Ticket handed it back. The main Session that stopped it may accept the rest of the message as new work.

A person can send an owned follow-up to Eval from the panel. If the main Session then attaches it back to a live Ticket, the Executor receives it again. The panel also offers re-evaluation when no effective judgement covers the message.

Context-only subscriptions

An Executor can follow a related Thread, such as another Issue or pull request, for context. Its messages reach the Ticket but grant no authority or follow-up strength. Human messages there do not resume an ended Ticket unless the main Session attaches a message from that Thread. Notes from the main Session about other work are also context only.

Human messages that reach no Ticket, including Threads whose Tickets were stopped by a person, take the Eval path. Bot messages take that path independently of their delivery to subscribed Tickets.

Strength

Strength	Cases
explicit	A person on the list addresses the agent, or follows up on a Thread that already has a Ticket; a cell of the admission table set to explicit request

Strength	Cases
inferred	A person not on the list follows up on a Thread with a Ticket, unless their cell of the admission table is explicit; a cell of the admission table set to inferred candidate; a message of a bot on the Source's bot list that addresses the agent or arrives where the Source receives every message
not admitted	Any other bot's message, everything the admission table ignores, and every inferred case while the accept mode is `off`

For an unlisted person, an explicit admission cell takes precedence even on a Thread with Tickets and works in every accept mode. Otherwise their follow-up is inferred and is not admitted in `off` mode. Messages on Threads followed only for context use the admission table instead of follow-up strength.

Admission

Each Source's Policy has an admission table. Its rows are receiving scopes:

- **Feishu:** each configured chat, other chats the bot receives mentions from, and direct messages. Other chats are received only when **其他群** is set to **仅 @** on the Feishu card.
- **GitHub:** new Issues, new pull requests, comments and the default branch's CI.

A row has up to three cells. Each cell decides whether a message becomes an explicit request, an inferred candidate or nothing:

Cell	Choices	Default
On the list, not addressing the agent	inferred, ignore	inferred
Not on the list, addressing the agent	explicit, inferred, ignore	ignore
Not on the list, not addressing the agent	inferred, ignore	ignore

Cells for unaddressed messages are available only where the Source receives every message, such as a Feishu chat set to **全部消息** or a GitHub scope with its receive-all switch enabled. A CI result has no sender, so its row offers only the last cell. Bot messages use the bot list instead of these cells.

Setting the unlisted-addressed cell to explicit lets anyone in that scope request work by mentioning the agent. This can be useful for a dedicated support chat. Inferred candidates still depend on the accept mode. Follow-ups normally bypass this table, except that an explicit cell still grants explicit strength.

The admission table is part of the Policy version. Weakening a cell does not immediately stop open Tickets accepted under it. It can revoke their Mandates when they are revalidated before resuming.

Bots and automated reviewers

The host recognizes GitHub Apps, logins ending in `[bot]`, and Feishu bots as bots. A Source can list bots such as `dependabot[bot]` whose messages may become inferred candidates. They must either address the agent or post in a scope that receives every message. They cannot make explicit requests or stop Tickets.

An automated reviewer using an ordinary account counts as a person to the host. Its comments reach subscribed Tickets and generate follow-up notices. Team prompt text must identify such accounts as automated reviewers.

Routing candidates

The main Session routes explicit candidates with `orchestrator_route`. One call can contain several accept, attach, stop or ignore decisions. Each item is processed independently: a refused item or a missing field does not prevent the others from running.

An inferred candidate normally needs a full assessment. There is one shortcut: if its card clearly touches no enabled rule, such as plain chat or unrelated discussion, the main Session may ignore it directly. The Decision records `ignoredFromCard` to distinguish this from a rule assessment. The main Session reads the enabled rules in its Policy prompt section.

Accepting an inferred candidate always requires an assessment. Direct accept refuses a Trigger that the gate did not admit as explicit.

Accept mode and rules

The accept mode is part of the Policy:

Mode	Effect
`off`	Nothing inferred becomes a candidate, such as an unaddressed message, a bot's message or the follow-up of a person not on the list
`observe` (default)	Inferred candidates are assessed; a positive assessment is recorded as ignore marked "would accept", and nothing is accepted
`auto`	A positive assessment of an inferred candidate accepts it

Explicit requests work in every mode. Start in `observe` and switch to `auto` after reviewing the recorded Decisions. Each assessment judges every enabled rule exactly once, whether it matches or not. A proactive accept requires at least one matching rule.

Each rule describes a kind of proactive work. The Ticket's Mandate records the matched rules, and its Executor receives their names and instructions.

For that Ticket, a rule's process and communication requirements override the defaults. A silence rule can suppress an acknowledgement; a review rule can name the required reviewer and approval before merge. Requirements that apply to every Ticket belong in team prompt sections instead.

Policy version

Rules, accept mode and each Source's Policy form the Policy version. Every Mandate records the version under which it was accepted.

A version change discards running Evals and queues their Triggers again. It also sends a policy-changed Input to each affected live Ticket. The Input explains whether:

- a matched rule changed, including its new text, or no longer applies;
- the requester may no longer start work; or
- proactive work is no longer accepted, for example because the mode changed from `auto`.

Unaffected Tickets receive no notice. The change does not immediately revoke live Tickets' Mandates.

Source configuration is separate from Policy. It controls what enters the system: which chats are read and whether a scope receives all messages or only mentions. A Mandate records its Source and receiving scope. Resuming needs the Source to still listen to the Thread, but does not require it to still receive every message there.

Replies wait while their Source is off. They are dropped if the Source no longer listens to their destination.

A condition on what the Executor may do with its own result, such as when it may merge, does not decide who may authorize work. It is part of the agent's instructions, not of the Policy.

Resume and stop

An ended Ticket keeps its Mandate. A person's message on one of its own Threads can resume the same work without a new Eval or Decision, regardless of who sent it. Bots and CI do not

wake ended Tickets. The Executor decides whether the request fits the scope and ends the round if it does not.

Only three operations change a Mandate: accept creates it, a person's stop withdraws it, and failed revalidation revokes it. A Ticket stopped by a person stays ended and receives nothing more. Work requested there needs a new accept.

Revalidation before resume

The Source must still listen to the original Thread. The remaining checks depend on the Mandate's basis:

- An explicit follow-up Mandate passes.
- An inferred Mandate needs `auto` mode and its matched rules still enabled.
- A Mandate admitted by a table cell must still receive at least its original strength under today's table and sender membership. Removing a sender from the list can therefore revoke a Mandate accepted through the listed-sender cell.
- A bot's Mandate requires that bot to remain on the Source's bot list. Inferred table and bot Mandates also need `auto` mode and enabled matched rules.
- Other Mandates require the sender to remain on the Source's allowed list.

If revalidation fails, the Ticket stays ended and takes none of its waiting messages. Each message is treated as having reached no Ticket; if no other Ticket owns it, it goes to Eval. The main Session is not told it reached a worker. This also applies after a processing pause. Attaching to the Ticket is refused, and further work requires a new accept.

- **A person saying "stop" in a Thread is an ordinary Trigger,** and Eval decides stop. An Executor cannot declare that a person asked it to stop.
- **Stop in the panel is a command.** It acts on the Ticket directly and is recorded as a manual stop.
- **Waiting conditions belong to Sources.** A condition such as "no assignee for N hours" would be a Source that waits and checks again itself; Eval never waits.

When Eval does not finish

The time limit starts when the candidate reaches the main Session. Time spent waiting for delivery does not count, and an undelivered candidate never expires.

An expired Eval queues its Triggers for another attempt. After two consecutive retries, the third expiry records `unavailable`. An assessment failure also records `unavailable`; a later Trigger can

start a new Eval. A Policy change or a decision made against an outdated Policy records `unavailable` and immediately queues the batch under the current version.

The `evals` health entry reports unresolved unavailable Decisions from the last hour. `scripts/dsh-superagent status` includes that error.

The panel asks for human attention only when the unavailable Decision is less than 24 hours old, is still the latest Decision on its Thread, has not been dismissed, and the Thread is neither queued nor evaluating. A person can evaluate the Trigger again or dismiss the problem.

003 Execution

This document follows an accepted Ticket through execution, delivery and recovery. It builds on 001 Domain model and 002 Authorization.

Ticket stages

Each open Ticket has one stage, determined by what it is waiting for. Read the table from top to bottom; the first matching row wins.

Stage	Condition	Panel label
person / delivery	Delivering an Input to the Executor Session failed; it is retried after 2 seconds, then after a delay that doubles up to a minute, and a success, a new Assignment or the Ticket's end clears it	Input undelivered
person / answer	It has an Attention	Awaiting answer
person / review	The latest Report is `ready`	Awaiting review
person / blocked, failed	The latest Report is `blocked` or `failed`, and no person's message reached the Ticket since	Blocked · Failed
working / queued	No Assignment, or a pending one	Queued
idle	The latest Report is `waiting`, or the Assignment ended	Waiting for a Trigger
working	Everything else	Working

A `person` stage means the Ticket cannot continue without human input. A new human message releases a `blocked` or `failed` Ticket to its Executor, so it moves to the next matching stage, usually `working`.

An `idle` Ticket has finished its turn without an outstanding question. A Trigger on a subscribed Thread can continue it.

The Executor

Each Ticket gets an Executor Session named `worker-` and a directory named after the Ticket's short name. Directories are created under the configured execution root, which defaults to `/work`.

Executor Sessions run commands without approval as the operator's OS user. Their `dsh-superagent-worker` preset contains the tools needed for work and moves foreground shell

commands to background jobs after 5 seconds. This lets a worker receive new messages at the next step instead of waiting for a long command to finish.

The main Session uses `dsh-superagent-router`, with the same 5-second shell and job-wait limits and a smaller tool set for routing. Sessions retain the preset selected at creation. Worker subagents use DeepSeek Harness's `standard` preset.

Background work and worker slots

When a turn ends with shell jobs or subagents still running, the Ticket keeps its worker slot and stays working. It is not reported as waiting, and the main Session receives no waiting report. A completed job or subagent wakes the Executor with its result in the same slot. Once a turn ends with no background work, the Executor reports waiting normally.

After each turn, background work has at most 30 minutes to finish. At that limit, shell jobs are killed and the Ticket is reported waiting. The report includes the Executor's last words and which jobs were killed or subagents remain active. Subagents are allowed to continue. The Executor reads killed-job results on its next turn.

A handoff to a person frees the slot immediately and leaves the Report in effect. This applies after a `ready` Report or a question, unless another Input arrived since, and after any `blocked` or `failed` Report.

At handoff, subagents stop. Shell jobs, such as a server offered for review, may continue until the Assignment ends. They hold no slot and are exempt from the 30-minute limit, including after later turns. Their completion does not wake the Executor; it reads the results on its next turn.

A person can also start a turn by writing directly in the Executor Session. Its background work follows the same rules.

When an Assignment ends, its shell jobs are killed without waking the Session. Its subagents are released, and any completion turns they wake are dropped. The replacement Assignment and its waiting Inputs start after that shutdown finishes.

Stopping or restarting the plugin inside the DeepSeek Harness process uses the same shutdown path. Inputs not yet taken into Session history are retained. Shutdown waits up to 10 seconds for each worker's background work; a process restart ends that work outright. An Assignment interrupted while busy restarts with a recovery note.

Worker subagents, including nested ones, use DeepSeek Harness's `standard` preset with the worker's permissions. They retain only the worker's allowed work tools, and lose those tools when the Ticket ends. Under `standard`, a shell command moves to the background after 60 seconds. A subagent's jobs are killed when it ends.

Preparing a Ticket directory

Before the first Assignment, if the Ticket directory does not exist, the prepare command runs once with `sh` in the execution root. It has ten minutes to finish. Without a prepare command, the worker starts in an empty directory.

A failed command fails the Ticket and records its output. A retry or a person's message on one of the Ticket's Threads tries preparation again. A typical command uses `git worktree` to create a checkout. A pull request belongs to the Ticket when its branch's last path segment matches the Ticket's short name.

Concurrency

A Ticket holds a worker slot from the start of a launch, including preparation, or from the delivery that wakes its idle Executor. It also holds the slot throughout an active turn.

The configured concurrency limit applies to these slots, with one extra slot for a person's message or a status request. Such a wake can use the extra slot when normal slots are full, and takes the next free slot before a new launch. New work therefore cannot prevent a person from continuing an existing Ticket.

Other Inputs, such as CI results, wait for a normal slot. A launch waits in `working / queued` until a slot is free. Launches run concurrently, so a slow prepare command delays only its own Ticket.

Execution variables

The prepare command and Executor instructions receive the same execution variables. The command reads environment variables; the Executor reads their values and descriptions in its prompt. None contains credentials.

Variable	Registered by	Value
`DSH_SUPERAGENT_DIR`, `DSH_SUPERAGENT_NAME`, `DSH_SUPERAGENT_TICKET`	Executor	The Ticket's directory, short name and id
`DSH_SUPERAGENT_BLOCKED_COMMANDS`	Executor	Commands the Executor may not run
`DSH_SUPERAGENT_GITHUB_REPOSITORY`	GitHub	The repository the Source watches, `owner/name`

Variable	Registered by	Value
`DSH_SUPERAGENT_GITHUB_ACCOUNT`, `DSH_SUPERAGENT_GITHUB_COMMIT_EMAIL`	GitHub	The agent account and its commit email
`DSH_SUPERAGENT_GITHUB_NUMBER`, `DSH_SUPERAGENT_GITHUB_PR_HEAD`	GitHub	The Issue or pull request that started the Ticket, and its head ref when it is a pull request
`DSH_SUPERAGENT_FEISHU_BOT`, `DSH_SUPERAGENT_FEISHU_CHAT`	Feishu	The bot, and the chat that started the Ticket

Ticket-specific values come from its directory and originating Trigger. Other values come from the current deployment settings. Only the panel can change execution environment settings; text sent to the main Session cannot change the configured shell command.

Workers reach GitHub through the agent account's `gh` and Feishu through the bot's `lark-cli`. Blocked commands fail when invoked by name.

These wrappers and directories establish defaults, not a security boundary. A worker can still execute a program by absolute path.

Merging

Auto-merge is governed by the **Delivery requirements** prompt. The host has no merge-specific decision logic.

By default, the Executor never enables auto-merge as the agent account: GitHub attributes the merge to whoever enabled it. Delivery requirements must name another account's CLI for merging. Without one, people merge the pull request.

The default instructions allow auto-merge only for a pull request opened for the Ticket, when it is ready, the required approvals cover the head commit, and additions plus deletions total fewer than 200 lines. Before pushing another revision, the Executor withdraws auto-merge.

Reviews without a body, such as approvals, also arrive as Triggers and can wake an Executor waiting for review. These instructions govern the agent, not GitHub's merge rules. To enforce approvals, require them in branch protection and dismiss stale approvals. Omit the merge CLI or remove the auto-merge paragraph to require manual merging.

How a Ticket ends

- **Ready without a pull request:** a `ready` Report ends the Ticket as completed.
- **Nothing to do:** an Executor that finds no work for a person to act on, such as a request that does not reproduce or is superseded, reports `dropped`. The Ticket ends without a message, unless a matched rule asks for one, and a person's later message on its Threads resumes it.

Pull requests

A Ticket follows the pull request it started from once its Executor reports it. It also follows each pull request opened from its branch once reported by the Executor or found by the GitHub Source. Reopened pull requests are followed again.

The Ticket is done when none of the pull requests it opened remain open. If it opened none, all pull requests it follows must be closed. It then ends with the outcome of the last relevant close and keeps its Subscriptions, allowing a later human comment to resume it.

A close does not end the Ticket while resumed, retried or recovered work is unfinished. It also waits if the Executor acknowledged or answered a follow-up it owns but has not since made a non-progress Report or ended a turn after reading that message. A later `ready` Report ends the Ticket once the same pull-request completion rule is satisfied.

Chat Channels receive one final notice about the pull-request outcome. GitHub already displays the outcome on the pull request itself. Removing a Ticket's Subscription to a pull request also stops it following that pull request.

"Awaiting review" therefore means a `ready` Report with a pull request.

Follow-ups and context Threads

The Executor decides human follow-ups on its Ticket's own Threads. It handles in-scope requests and hands back requests beyond its scope or for another Ticket. Once no Executor owns the message, it becomes a candidate for the main Session. The main Session may otherwise act on the follow-up only to stop work at the person's request.

An Input counts as read after the Session takes it into a step and subsequently reports or ends a turn. When a Ticket ends, unread messages that were not acknowledged, answered or handed back return to Eval, as described in 002 Authorization. Read messages left without action, such as thanks, do not.

Threads followed only for context still deliver messages, but do not give the Ticket ownership of human follow-ups or resume ended Tickets. Those messages can be assessed independently.

Sources poll subscribed Threads while a live Ticket subscribes, and for two weeks after the later of the last Trigger and the subscribing Tickets' end. Afterwards, messages arrive only through ordinary intake: GitHub notifications or Feishu live events.

Failure and recovery

Executor blocked or failed

When an Executor Session reports `blocked` or `failed`, or fails to start or errors, which records `failed`, the Ticket takes no automatic Input.

A passing failure of the model provider or the network, an HTTP 408, 429 or 5xx or a refused, reset or timed-out connection, is tried again first, three times with a growing backoff: a launch runs again, and a worker whose turn failed is told to go on; a failed prepare command is not tried again.

Once failed, CI and bot messages are recorded on it, but they are not delivered and do not wake the Session.

Its tools stay available, so a person can continue in the Session directly.

A launch that failed is not tried again on its own.

A person's message, a later Report that is neither `blocked` nor `failed`, or a retry ends the hold.

It is worked out from the stored Reports and Inputs, so a restart keeps it.

Retry

Available while an open Ticket is in `person / delivery`, `person / blocked` or `person / failed`: a new Assignment, which starts its worker again and takes the Inputs that wait, is the remedy for each.

Once a person's message reached a blocked or failed Ticket, the Executor has the message, so the Ticket shows as working, or as queued when a failed launch starts again, and offers no retry, unless delivering that message failed.

A retry ends the current Assignment and starts a new one in the same Executor Session, directory, branch and pull request.

The Executor model applies only when a Session is created; an existing Session keeps its model.

Restart

Running Assignments end with "host restarted" and new ones start with a recovery note that tells the Executor to check its directory, git history and GitHub before repeating any outside action, oldest Ticket first.

Assignments that are waiting, ready, blocked or failed stay as they are, unless an Input reached them after that Report, in which case they start again too.

Sessions, directories, Subscriptions, pending messages and the stored work below survive a restart, and so does the record of which messages each Executor Session took into its history, which a step does once it claimed them and before its model call: a message taken by a turn the restart cut off counts as read once a Report or the end of a turn follows, while one a step claimed but a cancel or error cut off before the history counts as unread.

A retry or a restart gives the new Assignment again the messages delivered to the Assignment it replaces that its Executor Session did not take, which a cancel or a cut-off step may have dropped; a copy still waiting in the Session, kept by a restart or persisted across a crash, is taken out before the new Assignment's first message, so the new delivery is the only copy.

Stored work

What dsh-superagent does after a decision, such as relaying a candidate, a follow-up or a report to the main Session, launching an Assignment or delivering an Input, is stored first and done afterwards.

A relay or a delivery that fails, such as when its Session cannot be reached, is retried after 2 seconds, then after a delay that doubles up to a minute; a launch that fails fails its Ticket, unless the failure was a passing one, as above.

Each kind of work reports an error among the overview's system problems only after one item failed three times in a row; a failed Input delivery shows on its Ticket instead.

Relays to the main Session keep their order, and so do notices; a failed one holds back the later ones of its kind.

Source failure

The GitHub Source reads each Issue, pull request and Thread on its own: a failed read keeps its error on its Thread, is read again on the next poll, and counts towards an error of the Source, which it reports after three failed polls in a row by default; any successful poll resets the count.

GitHub answers 404 also when the agent lost access to a private repository, so a 404 is retried.

A notified Issue or pull request whose reads keep failing is dropped a day after its first failed read, and at once when GitHub reports it as deleted; it then no longer fails the poll, its Thread keeps the error, and the Source's state names it.

Its read position stays where it was, so a later notification or a poll of its Thread reads what the failed reads missed.

The Feishu event consumer reports every failure, and polling of subscribed topics keeps each failure on its Thread.

A malformed item in a GitHub list or a Feishu page is skipped, and the Source's state counts it.

Channel failure

Outgoing messages are stored and sent as stored work, retried with backoff from 5 seconds, doubling up to 300.

A Ticket's messages to one conversation go out in the order they were queued, so its acknowledgement comes before its later answer or result: a message waits while an earlier one of the same Ticket to the same conversation has not gone out, until that one has failed three times in a row and shows as a problem.

Messages of other Tickets, and messages of no Ticket, do not wait for it.

A message with failed attempts is a system problem until it goes out or is withdrawn.

A message is dropped only when its Source no longer listens to the conversation, or when the Source answers that the conversation can never receive it; while its Source is off, it waits.

The Sources answer that for a deleted Issue or a locked conversation on GitHub, and for a recalled message or one Feishu no longer finds on Feishu.

A reply to a review comment that no longer exists goes to the pull request's conversation instead.

A dropped message keeps its record, with when and why it was dropped and the Ticket it was for, so it is never queued again; a dropped acknowledgement reads as dropped, not sent.

Pause processing

Stops dispatch, new Evals, Eval timeouts, Input delivery and replies; a launch already under way waits until processing resumes.

Sources keep recording Triggers, and a running Executor turn is not interrupted.

A person's message to an ended Ticket waits: the first pass after the pause resumes the Ticket, if its Mandate holds under the Policy then.

An Eval in progress does not expire during a pause; its deadline moves later by the pause's length.

Invalid configuration

A stored Source configuration that its kind rejects disables that Source instance.

004 Prompts

This document explains what the agents read, what a team can edit, and when changes take effect. It builds on 000 Philosophy through 003 Execution. Prompt text is written in English.

What a team writes

A team can edit three team sections, add skills, and override every built-in system-prompt part. Most defaults live under `prompts/` and `skills/`. Tool descriptions, notices and cards remain in code and cannot be edited through prompt settings.

Section	Holds
Communication	When to message people, how messages read, the line that begins every GitHub message (`This is by an automated agent, not a human` by default), how to handle reviews, and which ordinary accounts are review bots
Ticket scope	When a Ticket goes on and when it stops with a draft
Delivery requirements	Evidence in pull requests, the auto-merge condition with its line threshold, and the commit author; without one, the Executor commits as the agent account's no-reply address
Team skills	Knowledge the agents load when they need it, such as an `identities` skill that says which Feishu user is which GitHub account

The built-in text covers the roles, tool usage, host mechanics, authorization rules, and the GitHub and Feishu skills. It refers to the team sections by their headings instead of repeating them.

Built-in parts and templates

Overrides use these keys:

Parts	Keys
Shared instructions and roles	`runtime/shared`, `runtime/orchestrator`, `runtime/worker`
Working instructions	`runtime/subscriptions`, `runtime/replies`, `runtime/dispatch`, `runtime/work`, `runtime/admission`
Handoff instructions	`runtime/handoff/orchestrator`, `runtime/handoff/worker`
Source skills	`skill/feishu`, `skill/github`
Generated sections	`runtime/environment`, `runtime/policy`, `runtime/policy/none`

The generated sections are editable templates. They must retain these placeholders:

- `runtime/environment`: `{variables}`, filled with execution environment lines.
- `runtime/policy`: `{version}`, `{mode}` and `{rules}`, with one line per enabled rule.

- `runtime/policy/none`: `{version}` and `{mode}`, used when no rule is enabled.

Editing a part changes the instructions the model receives. It does not change the gate, Mandates, tool checks or account wrappers, which remain enforced by code.

On an explicit human request, the main Session can also save overrides with `orchestrator_prompts_save`. If a save omits an override's `base`, it keeps the saved base. A removed part already present in the saved configuration is dropped on save; any other unknown part is refused.

An override records which built-in text it was based on. Upgrades preserve the team's text. If the default changes, the panel marks the part **Default updated** and shows the difference beside the team's text. The team can keep its text, use the new default or edit it. Parts without overrides automatically follow new defaults.

The main Session can read affected part names in `orchestrator_read prompts`, under `default_changed`.

Assembly and precedence

The system prompt is assembled in this order:

8. Shared instructions and the role.
9. The three team sections.
10. Subscriptions and replies.
11. Dispatch for the main Session, or work instructions for an Executor.
12. `runtime/admission`, for the main Session only.
13. Handoff instructions.
14. `runtime/environment`.
15. `runtime/policy`, for the main Session only.

Admission instructions come from `prompts/admission.md`. They describe proactive assessment and when an irrelevant candidate can be ignored from its card. `orchestrator_eval` returns context rather than repeating these instructions.

The environment section gives the main Session deployment-wide values and each Executor its Ticket-specific values. The Policy section is regenerated from settings on every model request. It contains the Policy version, accept mode, the explicit-request exemption, and enabled rules as `- ():` , or a statement that none is enabled. It is last so a Policy change leaves the preceding prompt text unchanged.

Executors also read DeepSeek Harness's coding prompt and the repository's `AGENTS.md`; the main Session does not. An inferred Ticket adds a final note limiting its Executor to the matched rules. Assignments and incoming messages arrive inline as Trigger cards, with resource pointers only for truncated content.

Executors work unattended. Their tools exclude those that ask a person, present UI to them or change the profile, such as `ask_user_question`, `present` and `plugin_manager`.

From strongest to weakest:

16. Facts enforced by tools and the host: identities, permissions, tool argument checks, reply purposes, length limits. Prompt text cannot change them.
17. Built-in and generated sections, as the team's overrides of them read.
18. The process and communication in a proactive Ticket's matched rules, for that Ticket's Executor Session only.
19. Team sections and team skills.
20. Outside text: messages, comments, documents and quotes. It is data and never changes rules or authorization.

Principles of the built-in text

These apply 000 Philosophy to the agents:

- **Separate roles.** The main Session only routes: it forwards the original Trigger and the routing facts, and makes no technical decisions, through one routing tool that takes the decisions on several waiting candidates in one call. A routing note has at most 30 words. The Executor reads the original sources, judges the scope, implements and verifies.
- **The main Session does not wait.** Reports, follow-up notices, candidates and completed background jobs arrive automatically as messages. It ends the turn when nothing remains to do. A guard refuses `sleep`, shell timeouts over 30 seconds and job waits over 5 seconds, because a blocked step delays new candidates. These instructions replace DeepSeek Harness's own job-notification note in the main Session's complete prompt.
- **Message only when a person must act or can use a result:** a result can be reviewed or used, a decision or resource is needed, or the work cannot be finished. Chat messages are one sentence by default, with a hard limit of 100 words.
- **Executors decide their follow-ups.** Each Executor handles and acknowledges in-scope requests, and hands back requests beyond its Ticket or for another Ticket. A message becomes new work only after all its owners hand it back or end without taking it up. The

main Session receives follow-up notices but may act on them only to stop work. Handed-back messages then arrive as candidates with the Executors' reasons.

- **Coordinate through subscriptions and the main Session.** An Executor follows related Threads for context. The main Session can send a fact from another Ticket in a note of at most 100 words. Neither the Subscription nor the note grants authority.
- **Attach only on an explicit reference.** A message must point to a Ticket's pull request, Issue, source message or acknowledgement, or to a recent change with exactly one matching Ticket. Topic similarity alone is not enough. Attachment subscribes the Ticket to the entire Thread; an Executor finding it unrelated uses `orchestrator_thread_unrelated` and does not answer there.
- **Confirm concretely, once.** Before substantial work, the Executor confirms each new request and names the object and the expected result, as a reply under the original message.
- **Ask through `human_action`.** The Executor puts one concrete question in its Report, and the main Session forwards it unchanged. If a person's message prompted the question, it names that message and the host asks there. Otherwise it asks in the Ticket's original conversation. Already authorized work can continue meanwhile.
- **Honest delivery.** Opening a pull request is not readiness; a draft is marked unfinished; a UI change without a real before screenshot is not ready.
- **A person's pause holds** across follow-ups, new Executor Sessions and restarts until a person lifts it. Automatic feedback never resumes paused work.
- **Examples only show format.** An example sentence is not a fact the agent may claim.
- **Handle reviews by outcome.** A fixed and verified comment is resolved silently; a declined one gets one sentence of reason. An automated reviewer's thread is resolved once it has an answer, and each automated reviewer gets at most three rounds per pull request.
- **Yield to duplicate work.** When a person's branch or pull request already contains the Ticket's change, the Executor closes its own pull request with one sentence pointing to theirs.

Versions

- Each save creates an immutable version containing the full configuration, revision number, predecessor, timestamp and origin (panel or tool). Restoring a version or defaults creates another version.
- Saves name their base revision. A conflicting save is refused without discarding the draft.
- Sessions read the latest version on every model request, including later steps in the same turn. Saving does not restart, cancel or wake a Session. The host records the revisions each Session used; an Executor can read any revision it worked under.

- The deployment sets a total size limit, including overrides. Validation refuses unknown keys, reserved built-in skill names, unknown override parts and missing template placeholders. It never silently truncates text. An override identical to the default is removed so the part follows future defaults.

The panel, page by page

The panel lives in the sidebar under "Superagent 多通道" (Superagent multi-channel), and does not replace the official entry points. The left column is fixed: **来源 / 评估 / 执行 / 提示词 / 部署** (Sources / Assessment / Execution / Prompts / Deployment), with **概览** (Overview) at the top. All screenshots below are from a real deployment, with sensitive information redacted.



Assessment and Policy

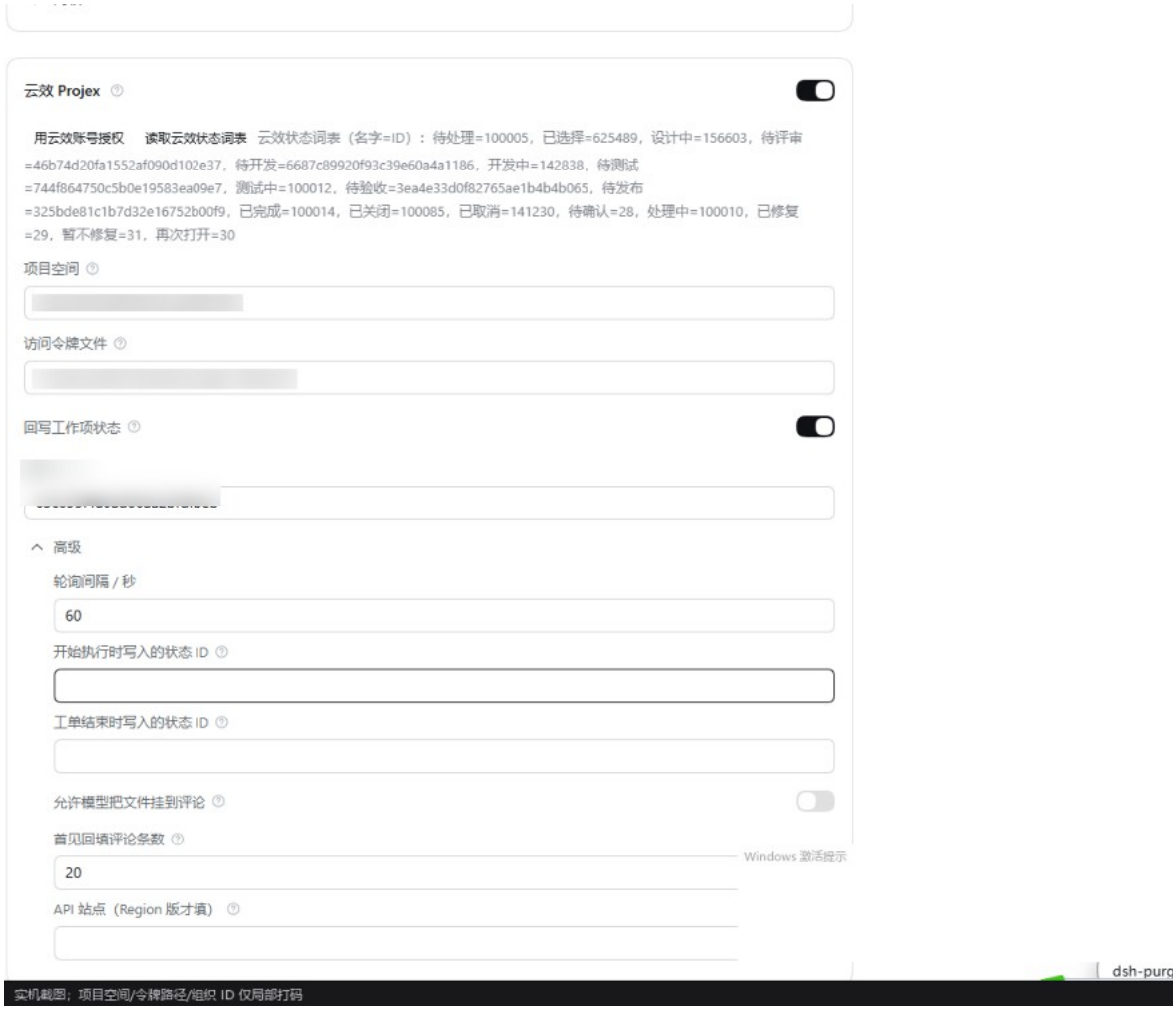
Overview

- Statistics at the top: Tickets awaiting action, assessments in progress, messages blocked by Policy, failed replies and so on; clicking a number jumps to the matching list.
- Cards point out **what to do next** (for example "a Source is not configured", "a reply keeps failing"), and offer a clickable fix action.
- "打开主 Session" (Open the main Session) goes straight into the orchestrator's conversation, so a person can take over.

Sources

Each Source is one card:

- A switch decides whether it is enabled; an unconfigured card says "来源未启用 / 等待配置" (Source not enabled / awaiting configuration).
- A card carries that Source's **actions**: checks for Feishu and GitHub, "启动桥 / 停止桥 / 检查" (Start bridge / Stop bridge / Check) for WeChat, "用云效账号授权 / 读取云效状态词表" (Authorize with a Yunxiao account / Read the Yunxiao status vocabulary) for Yunxiao, checks for WeCom and so on.
- Fields are grouped as needed, credentials preferably live in a 0600 file; old fields are marked as migrated to the new location (they are not silently dropped).



Yunxiao Projex Source card

Taking Yunxiao Projex as an example: authorization, status vocabulary, project space, token file, status write-back and the two **status dropdowns** (they show the board's names and store status IDs), polling interval, the attachment-comment switch, how many items to backfill on first sight, and the API site.

Triggers

- Lists every Trigger received in reverse time order: which Source/Thread it came from, who sent it, the strength of the judgement, which Ticket it went to, or why it was ignored.
- Clicking through to a Thread shows the context and the original message; "重新评估" (Re-evaluate) hands the Trigger back to assessment (it does not change authorization).

Assessment

- The upper half is **Policy**: rule switches (such as Copy fixes / Visual fixes, customizable), each Source's list of "users who may start work", and the admission table (on/off the list × mentioned in the chat / not mentioned / direct message).
- The lower half is **assessment settings**: accept mode (off / observe only / accept automatically), the time limit for one assessment, the assessment model and its reasoning effort.
- Every assessment records: the Decision, the strength (explicit / inferred / quoted), the basis and the duration; "评估未完成" (Assessment did not finish) shows a reason instead of a blank.

Execution

- Tickets in progress, their Executor Sessions, and the current stage (working / waiting / ended).
- Each Ticket shows: the task description, the input messages, a summary of the Executor's conversation, and the latest Report; it can be interrupted, asked for more, or replied to directly.
- An Executor receives only the Ticket's scope; file deliveries go back to the original Thread as attachments in the Report.

Prompts

- Every part of the main Session's and the Executor Session's prompt can be viewed and overridden; changes are saved as versions, with history comparison and restore-to-default.
- Six Source skills each take one part: Feishu, GitHub, WeChat, Yunxiao Projex, WeCom and DingTalk.



Prompts

Deployment

- Shows the current `stateDir`, the database version, health-check results, and the state of background tasks (bridges, pollers).
- Upgrade notices and migration records appear here; when two superagents coexist, each shows its own directory and lock.

Screenshots and privacy

- The panel never prints tokens; before taking a screenshot, still check names, chat names and project IDs, and redact them if needed before publishing.
- The examples in this repository's `docs/images/` are already redacted; keep the same redaction standard when replacing them.

006 Deployment

Use this guide to install, configure and operate dsh-superagent. It covers accounts, startup checks, upgrades and troubleshooting, building on 001 Domain model through 005 Panel.

What runs

- **A dedicated DeepSeek Harness profile**, called `superagent` in this guide. Create it from the Web profile template. It loads the bundle and serves its panel on `127.0.0.1`. Keep it separate from the profile you work in: the plugin changes process-wide `PATH` and git settings, and its process needs independent restarts and upgrades.
- **An agent GitHub account.** The agent comments, pushes branches and opens pull requests as this account, through a GitHub CLI signed in only as that account.
- **Optional:** a Feishu bot for chat requests.

Plugin configuration holds the state directory and machine limits. Use the panel for everything else: Sources for the repository and accounts, Policy for allowed lists, and Execution for worker directories and commands.

Permissions and isolation

The bundle defines `dsh-superagent-worker` for Executors and `dsh-superagent-router` for the main Session — the same preset ids the pre-2.3 bundle used, so a switch from an older install resumes its existing Sessions instead of failing with `Unknown agent preset`. They appear in the DeepSeek Harness preset list but are intended only for this plugin. Do not select them for personal Sessions or disable their configuration rows. The application needs them to create and resume its Sessions.

Opening a blank managed Session in the Web UI can select the profile's default preset. dsh-superagent restores its role-specific preset before the first turn.

Both presets run with `danger-full-access` as the operator's OS user and read messages from subscribed Threads. Account wrappers choose the agent's credentials by default; they do not isolate the process. A worker can execute programs by path and read the operator's home directory. For isolation, use a dedicated OS account containing only the agent's credentials.

Prerequisites

- Node.js 22.19+ or 24+, pnpm, git 2.32+, Python 3.8+ and tmux

- GitHub CLI (`gh`) 2.48 or later. Source CLI fields must point to `gh` or `lark-cli` wrappers; the host relies on their subcommands and output formats.
- A repository on github.com; GitHub Enterprise Server is not supported. To create a checkout per Ticket, keep a local clone from which the prepare command can add worktrees. Install `git-lfs` if that repository uses Git LFS.
- A model provider configured for DeepSeek Harness. dsh-superagent uses the profile's default model unless the Eval or Execution settings name another.
- For Feishu: `lark-cli` (tested with 1.0.90) and a Feishu or Lark app with a bot

Accounts

Agent GitHub account

21. Create a GitHub user account for the agent and give it write access to the repository. A GitHub App cannot be the agent, because dsh-superagent reads the account's notifications.
22. Give it a GitHub CLI that uses only its own credentials:

```
mkdir -p ~/.local/bin
cat > ~/.local/bin/gh-agent <<'EOF'
#!/bin/sh
GH_CONFIG_DIR="$HOME/.config/gh-agent" exec gh "$@"
EOF
chmod +x ~/.local/bin/gh-agent
gh-agent auth login
gh-agent api user --jq .login
```

Sign in as the agent account; the last command prints its login. dsh-superagent reads the login the same way.

Feishu bot (optional)

23. Create a self-built app with the bot capability. Grant it the permissions to receive message events, read and send chat messages, and read chat members, then publish the app version.
24. Give the bot its own `lark-cli` configuration, restricted to the bot identity, and a wrapper:

```
LARKSUITE_CLI_CONFIG_DIR="$HOME/.lark-cli-dsh-superagent" lark-cli config init --app-id <cli_…> --app-
secret-stdin
LARKSUITE_CLI_CONFIG_DIR="$HOME/.lark-cli-dsh-superagent" lark-cli config strict-mode bot --global
cat > ~/.local/bin/lark-bot <<'EOF'
#!/bin/sh
export LARKSUITE_CLI_CONFIG_DIR="$HOME/.lark-cli-dsh-superagent"
exec lark-cli "$@"
EOF
chmod +x ~/.local/bin/lark-bot
```

The first command reads the app secret from standard input. Use a separate configuration directory: `lark-cli` allows one profile per app id, and your personal configuration may already contain a user login for that app.

The host refuses a CLI that can act as a user. It reads the app id, Feishu/Lark brand and bot `open_id` from the bot configuration.

Both configurations share the keychain entry for the app secret. To remove only the bot configuration, delete its directory; do not run `lark-cli config remove`, which would affect the shared secret.

Install

Run this block once, on a machine that has no `superagent` profile yet:

```
npm install -g @deepseek-ai/dsh@0.2.1-rc.3
git clone <this repository> dsh-superagent && cd dsh-superagent
pnpm install && pnpm build
dsh superagent --from-default-profile web --dump-config > /dev/null
dsh plugin --profile superagent add "link:$PWD"
```

The pinned version above is public; other versions must satisfy `engines.dsh` in `package.json`.

The fourth command creates `~/.dsh/profiles/superagent` from the DeepSeek Harness Web template. The fifth installs this bundle. Set `DSH_HOME` to relocate the default `~/.dsh` directory.

Run the profile-creation command only for a new installation. If the profile already exists, that command replaces it and deletes its `cordis.patch.yml`, including machine configuration. Do not use it for upgrades.

Machine configuration

Copy examples/profile.cordis.patch.yml to `~/.dsh/profiles/superagent/cordis.patch.yml`, replacing that file, and replace every `` `. For Feishu, also enable the `` dsh-superagent-multi-channel-feishu `` `entry there. The ` webserver`` entry sets the panel's port.

Key	Required	Meaning
`stateDir`	no	Absolute path: the database, the lock, the command shims in `worker-bin`, the agent's git configuration and, by default, the Ticket directories in `work`. Defaults to an existing `~/.dsh/state/superagent` (a pre-2.3 installation) or `~/.dsh/state/dsh-superagent-multi-channel`. Back it up before upgrades
`promptMaxChars`, `admissionContextMaxPages`, `admissionContextTimeoutMs`	no	The largest saved prompt configuration, and how many Thread pages one assessment reads within what time

After the first start, edit machine limits in **Settings** → **Deployment** or the DeepSeek Harness Plugin Manager. Changes apply without restart. `stateDir` cannot be changed there. Invalid values or unknown configuration keys prevent startup; the log identifies the key.

Install beside the official superagent

This bundle is a separate package with its own entry ids, agent presets and panel entry, so it can be **installed** beside another superagent bundle without overwriting its rows, presets or panel slot.

A Harness Host holds **one Source kind per id**, and every superagent bundle registers the same kind ids (`github`, `feishu`, `projex`, ...). So two superagent bundles can be installed, but only **one may be enabled at a time**; enabling both leaves the second bundle's Source rows reporting `Source kind already registered`. Switching is a bundle disable plus a bundle enable, and each bundle keeps its own `stateDir`, so switching back changes nothing else.

	official superagent	this bundle
Package	the official package	`dsh-superagent-multi-channel`
Profile entry ids	`dsh-superagent`, `dsh-superagent-`	`dsh-superagent-multi-channel`, `dsh-superagent-multi-channel-`
Agent presets	`dsh-superagent-worker`, `dsh-superagent-router`	the **same ids**, deliberately: one superagent bundle is enabled at a time, and a switch then resumes the Sessions either bundle created
Panel entry	its own	sidebar "Superagent multi-channel"
State	one `stateDir`	its own `stateDir` (default below)

To move an installation of an older version of this bundle: disable the old bundle, enable this one, and leave `stateDir` empty - the default adopts the existing `~/.dsh/state/superagent`, so the database, tickets and cursors come along. If both stay installed, each needs its **own** `stateDir` (one directory admits one instance, and the lock says so).

Install from a plugin market or a tarball

- Plugin manager or market: install `dsh-superagent-multi-channel@`. The market reads `dsh.bundle.patch` and the

display metadata (`locale/en.json`, `locale/zh.json`, `icon.svg`) from the package, so the entry appears with its own title, description and icon.

- Custom plugin index (the Discover tab of the "Custom plugin indexes" bundle) or the community market: point it at the

published package, or host the repository tarball and install that URL - a profile bundle also installs from `…/archive/refs/tags/v.tar.gz`.

- Local development: `plugin_manager install_bundle` with the absolute checkout directory links the working tree, so a

rebuild is visible without reinstalling; `npm pack --dry-run` shows exactly what a published tarball would contain.

Start and stop

```
scripts/dsh-superagent start
scripts/dsh-superagent status
```

`start` runs `dsh --profile superagent` in tmux session `dsh-superagent`, or `dsh-` for another profile. It waits up to one minute for the runtime to become ready or report a setup requirement. On a first run it prints the missing setup information. It refuses duplicate instances, including ones started outside this script.

`stop` and `restart` request graceful shutdown and wait up to five minutes for Executors. They never force-kill the process or touch other tmux sessions.

`status` prints process ids, health and setup problems. For a different profile or port, use `--profile` and `--url`, or set `DSH_SUPERAGENT_PROFILE` and `DSH_SUPERAGENT_URL`.

`status`, `start` and `restart` exit with one of four codes:

Code	Meaning
0	The service runs normally
1	It does not run, runs twice, does not answer, or its runtime reports an error; a restart may fix it
2	It runs, but a part other than the runtime, such as a Source or a Policy sync (`sync:`), reports an error; a restart does not fix it
3	It runs and waits for setup; a restart does not fix it

A Source that is still starting counts as normal operation. So does a Feishu bot that setup could not check.

`status` returns failed parts and their details under `errors`, and setup requirements under `setup`. `start` and `restart` print those problems as lines; they exit 1 if the service never comes up. Only an intentional `stop` pauses automatic recovery.

Each profile stores logs and control files under `.artifacts/`:

- Startup logs: `dsh-superagent----.log`, where the pid belongs to the script. The newest 20 are kept.
- Control files: `control/profiles//`, containing the stop marker, lock and process id.

Open the URL printed in the log, then choose **Superagent** in the sidebar.

Hourly check

Create the hourly check in the DeepSeek Harness profile you normally use, not in `superagent`. It must keep running when the agent service is down. Ask an agent in that profile to create an hourly schedule using this prompt, with the checkout path filled in:

```
Check dsh-superagent: run `scripts/dsh-superagent --profile superagent status` in <checkout>. Exit 0: do nothing and say nothing. Exit 2: do not restart; tell me each part under `errors` with its detail, unless you already told me about the same ones. Exit 3: do not restart; tell me what `setup` says setup needs, unless you already told me. If it reports manually_stopped: true, leave it stopped. Otherwise (exit 1) run `scripts/dsh-superagent --profile superagent --respect-stop restart`, which exits the same way: 0 means it recovered, so say nothing; 2 and 3 are reported as above; if it exits 1 or fails otherwise, read the newest .artifacts/dsh-superagent-superagent-*.log and the Troubleshooting section of docs/006-deployment.md, fix what you can, and tell me what was wrong and what you did.
```

`--respect-stop` preserves an intentional stop. Only `stop` writes the marker; `start` clears it. `restart` writes no marker, so the check can recover a restart whose shutdown timed out.

Settings

Setup must succeed before polling or work starts. Overview lists missing requirements and links to their settings. A settings change runs setup again. If you fix something outside the application, such as CLI authentication, choose **Check again**.

25. **Sources.** Set the GitHub repository (`owner/name`) and agent CLI, for example `/home/me/.local/bin/gh-agent`. Requests normally arrive through the agent account's notifications. A new Issue or comment mentioning the account arrives on the next poll; adding a mention by editing an existing Issue body does not. Post a new comment instead. For Feishu, set the bot CLI, chats, each chat's receive-all or mentions-only mode, and whether to receive direct messages.
26. **Policy.** The two example rules can be edited or removed. Under **On each Source**, add the GitHub accounts and Feishu users who may ask for work; until a list has someone, no request can become a Ticket. With Feishu, a chat can be chosen to sync the list from, at startup and on **Sync now**.
27. **Execution.** The directory root and a prepare command. To give each Ticket a checkout of the repository:

```
   git -C /home/me/src/widgets fetch --no-tags "https://github.com/$DSH_SUPERAGENT_GITHUB_REPOSITORY.git" "${DSH_SUPERAGENT_GITHUB_PR_HEAD:-HEAD}:refs/dsh-superagent/$DSH_SUPERAGENT_TICKET" &&
   git -C /home/me/src/widgets worktree add -b "agent/$DSH_SUPERAGENT_NAME" "$DSH_SUPERAGENT_DIR" "refs/dsh-superagent/$DSH_SUPERAGENT_TICKET"
```

List personal CLIs, such as one signed in to your own account, as blocked commands.

28. **Prompts.** Set the team sections: the GitHub disclosure line, review bots, the commit author and the auto-merge condition.

Management API

The panel uses a local HTTP API that agents can also call. `SETUP.md` uses it to save settings, and the service script reads `GET state`.

Send requests to `http://127.0.0.1:/orchestrator/api/` with `x-orchestrator-request: 1`. The host must be `127.0.0.1:` or `localhost:`. Missing headers or an `Origin` from another site receive 403.

API saves have the panel's permissions. They can change execution environment and setup account fields, unlike saves made through the main Session's tools.

Agents may rely on these routes and fields. The other routes in `src/app/api.ts` serve the panel and change with it.

Request	Answer
`GET state`	What every page reads: `setup`, with `problem` and `feishuProblem`, each `null` or a setup problem with `place`, `code`, `values` and `detail`, and `accounts` once setup resolved them; `health`, an entry per part, below, with `state` (`starting`, `ready`, `paused` or `error`), `detail` and `at`; `load`, the Executor Sessions `running`, the Tickets `queued` and `maxWorkers`; `paused`; and `masterId`
`GET settings`	`blocks`: `policy`, `eval` and `executor`, each with its `value` and its `revision`; `sources`: the Source instances, each with `id`, `kind`, `installed`, its block's `value`, `{"enabled": …, "config": {…}}`, and `revision`, its kind's form, and its setup `problem` when it has one; `policyVersion`; and `environment`, the execution environment's `root` and `variables`
`GET sources`	Each Source's status: `state` (`missing`, `off`, `error`, `setup`, `starting`, `paused` or `ready`), the `health` of its parts, its setup `problem`, the last day's `stats`, the Triggers it `filtered` in that day, and the messages it `dropped` in that day, newest drop first, each with its `channel`, `preview`, `ticket` (`id` and `title`) when it was for one, its `thread` when the store has it, and `dropped.at` and `dropped.reason`, `channelGone` or `channelIgnored`
`PUT settings/` with `{"value": …, "revision": n}`	Saves one block: `saved`, the block's new `revision`, `policy_changed` and `restarting`. `value` is the whole block, as `GET settings` holds it; a missing key takes its default
`POST executor/pause` with `{"paused": true}` or `false`	Pauses or resumes processing: `paused`
`POST setup/check`	Runs setup again by restarting dsh-superagent: `restarting: true`. Refused when setup has nothing to fix
`POST policy-sync/`	Syncs that Source's allowed list now: `policy`, the Source's part as saved

Request	Answer
`GET prompts`, `PUT prompts` with `{"config", "baseRevision"}`	Reads the prompt configuration: `current`, with its `revision` and `config`, `defaults`, `sections`, the team sections in the order Sessions read them, `skillName`, the pattern of a team skill's name, `builtin`, the names a team skill cannot take, and `skillPrefix`, which Sessions load a skill's name under. Or saves it: the new `revision`
`POST tickets//retry`, `POST tickets//stop`	Retries or stops a Ticket, as the panel does

Health entries are grouped by responsibility:

Entry	Reports
`runtime`	The application itself
`models`	Sessions waiting on a model-budget retry
`evals`	Unavailable Evals from the last hour with no later judgement or dismissal
`assessment`, `launch`, `delivery`, `relay`, `notice`, `reply`	Stored work described in 003 Execution; delivery counts waiting work, while reply covers Channel messages
`worker`	Executor Session event handling; appears after its first failure
`github`, `feishu`, `history`	Source polling, events and history recovery
`sync:`	Allowed-list synchronization

A refusal has a status and `error`, its English text:

- 400: an invalid value (`issues`, each with its `path`), a failed account check (`setup`), a fixed refusal (`code` and `values`), or a body that is not JSON or is too large.
- 404: something that does not exist.
- 409: a stale revision.
- 500: anything else.

The blocks:

- `source:github`, `source:feishu`: `{"enabled": true, "config": {…}}`. GitHub's config has `repository` and `cli`, and the switches `observeIssues`, `observePulls`, `observeComments` and `observeCi`, off by default, which receive every new Issue, new pull request and comment in the repository and the default branch's CI; Feishu's has `cli`, `chats` as `[{"chatId": "oc_…", "name": "…", "listen": "mentions"}]` (or `"all"`), `directMessages`, and `otherChats` as `"none"` or `"mentions"`.
- `policy`: `{"rules": […], "sources": {"github": {"allowed": [{"id": ""}], "bots": [{"id": "dependabot[bot]"}]}, "feishu": {"allowed": [{"id": "ou_…", "name": "…"}], "syncChat": "oc_…", "admission": {"oc_…": {"unlistedAddressed": "explicit"}}}}}`. `bots` lists the bots whose messages may become inferred candidates. An `admission` row is keyed by chat id,

`other` or `direct` for Feishu, and by `issues`, `pulls`, `comments` or `ci` for GitHub; an unset cell takes its default (see 002 Authorization).

- `eval`: `mode`, `timeoutSeconds` and `model`.
- `executor`: `model`, `maxWorkers`, `directory`, `prepare` and `blockedCommands`. The processing pause is a setting of its own, `paused`, which `POST executor/pause` sets.

Changing a repository or CLI runs the corresponding setup check first. A failed check returns 400 with details in `setup`. A successful account change, or any save while setup is pending, returns `restarting: true`. The API is temporarily unavailable while the application restarts.

```
api=http://127.0.0.1:3082/orchestrator/api
rev=$(curl -s -H 'x-orchestrator-request: 1' $api/settings | jq '.sources[] | select(.id == "github")
| .revision')
curl -s -X PUT -H 'x-orchestrator-request: 1' -H 'content-type: application/json' $api/settings/source:github
\
  -d "{\"value\": {\"enabled\": true, \"config\": {\"repository\": \"acme/widgets\", \"cli\":
\"$HOME/.local/bin/gh-agent\"}}, \"revision\": $rev}"
```

Setup checks

Setup uses the DeepSeek Harness subprocess service, as worker shells do. It removes environment variables whose names contain KEY, PASSWORD, SECRET or TOKEN.

Setup verifies the GitHub repository and the agent's push access, writes the worker environment, and checks that git uses only the agent credentials for github.com. A Feishu setup failure disables only the Feishu Source.

The process environment that setup writes:

- **Command shims** in `/worker-bin`, first on `PATH`: `gh` runs the agent's CLI; `lark-cli` runs the bot's CLI; blocked commands fail. A shim removes its own directory from `PATH` before it runs its target.
- **git configuration** in `/worker-gitconfig`: replaces the operator's global and system git configuration. It selects the agent CLI as the github.com credential helper, rewrites GitHub SSH addresses to HTTPS and defines Git LFS filters. Personal git settings such as an HTTP proxy do not apply. No commit identity is set here, git never prompts for credentials, and `GH_TOKEN` and `GITHUB_TOKEN` are removed.

A file in `worker-bin` or `worker-gitconfig` that dsh-superagent did not write stops setup. A Ticket directory inside a git repository whose own configuration adds credentials fails the Ticket.

Try it

From an allowed GitHub account, open an Issue or post a comment containing `@` . Within a polling interval (one minute by default), Overview should show a Ticket. The agent

acknowledges the request, works in its own Session and directory, and delivers in the original conversation. For Feishu, mention the bot in a configured chat.

Upgrade

29. Wait until no Executor is in the middle of a turn.
30. Back up `/orchestrator.sqlite`.
31. Run `git pull && pnpm install && pnpm build`.
32. Run `scripts/dsh-superagent restart` and inspect its status.

A restart ends running Assignments and starts replacements with recovery notes. Team text and overridden prompt parts are retained; parts without overrides follow the new defaults. Review parts marked **Default updated** after upgrading.

A database migrated by a newer version cannot be opened by an older one. Rollback therefore requires the pre-upgrade backup. After upgrading DeepSeek Harness itself, restart the running instance so the panel loads matching artifacts.

Agent presets

The worker and router presets move shell commands to the background after 5 seconds. A Session cannot load without its preset. Keep the `dsh-superagent-worker` and `dsh-superagent-router` rows in `cordis.patch.yml` while their Sessions are in use; otherwise resume fails with `agent-preset/not-found`.

Troubleshooting

Setup problems appear on the overview with a link to the fix, in the panel's language; the log and the API show the English text below.

Symptom	Cause and fix
`The agent GitHub CLI is not set.`	Set the agent's CLI on the GitHub card
`The agent GitHub CLI was not found.`	Set the wrapper's absolute path, or a command name on `PATH`
`The agent GitHub CLI cannot read its account`	Sign the wrapper in with `auth login`
`@… cannot read …` or `@… cannot push to …`	Give the agent account write access to the repository
`The GitHub Source names no repository.`	Set the repository, `owner/name`, on the GitHub card
A Ticket fails with `The prepare command failed: …`	The output names the cause, such as a wrong clone path; fix the command, then retry the Ticket
`git does not read the github.com credential configuration dsh-superagent writes`	Install git 2.32 or later
A Ticket fails with `The git configuration of the repository in … sets …`	Remove the named credential or `http.*.extraheader` settings from the clone's `.git/config`

Symptom	Cause and fix
`… was not written by dsh-superagent`	Move the named file out of `/worker-bin` or `/worker-gitconfig`, or choose another `stateDir`
`The bot lark-cli allows user identities`	Point the Feishu Source at a bot-only configuration, as in Feishu bot above
`The bot lark-cli does not work.` or `The bot identity of app … is …`	Check the configuration's app id and secret
`The Feishu bot is not a member of the chat.`	Add the bot to the chat in the chat's settings; the next attempt succeeds
`status` exits 2	A part reports an error, which a restart does not fix: `errors` names each part, as listed under Management API, with its detail; each part clears its own entry once it works again
`status` exits 3	Setup waits: `setup` says what is missing, and the overview links to the setting that fixes it
`status` exits 1	The service does not run, runs twice, does not answer, or its runtime reports an error; the output and the newest log say which
`… timed out after 90 s`, `… was stopped by …`, `… exited with code …` or `… could not run: …`	A CLI failed without printing why; run the named program by hand as the operator's user to see its own message
The panel says dsh-superagent is not answering	It is restarting, or did not start because of its configuration; the DeepSeek Harness log names the cause
`dsh-superagent configuration has keys it does not use` (log)	Remove the named keys; the message says where each value lives now
The log says the plugin is incompatible with the running dsh	Install a DeepSeek Harness version within the `engines.dsh` range
The panel shows "Failed to load plugins" after a DeepSeek Harness upgrade	Restart the running instance
Nothing happens after a mention	Check that the author is on the Source's allowed list, the Source is on, and processing is not paused. On GitHub, a mention added by editing an Issue's body is not received; mention the agent in a comment

007 Architecture

This is the developer guide to the implementation. It maps the concepts in 001 Domain model through 006 Deployment to classes, interfaces and files. Start with the runtime and Engine, then use the module and test tables as a code index.

Core classes and glue

Core classes own domain concepts and external boundaries behind small interfaces. Glue translates tool arguments, HTTP requests, Source payloads and DeepSeek Harness events into calls to those classes, then turns results into cards, views and messages.

```
flowchart TB
  plugins[Source plugins] -->|SourceContext| engine
  tools[Session tools] --> engine
  http[Management routes] --> engine
  engine[Engine] -->|Repositories| store[Store]
  engine -->|Policy| policy[Policy]
  engine -->|SourceScope| registry[SourceRegistry]
  engine -->|Assessor| assessor[MasterAssessor]
  engine -->|Executor| executor[DshExecutor]
  assessor -->|decide| engine
  engine -->|Outbox| outbox[Outbox]
  executor -->|assignmentStarted, inputsDelivered, report| engine
  settings[Settings] -->|policyChanged| engine
  assessor & executor & sessions[Sessions] & outbox -->|enqueue| effects[Effects]
  effects --> store
```

`Runtime` (`src/app/runtime.ts`) constructs the application, supplies dependencies, and owns startup and shutdown. `Engine` constructs its own internal parts. The Executor and Assessor call back into the Engine, so Runtime gives them getters rather than requiring construction in a circular order.

Runtime

`Runtime` is the composition root; it contains no domain rules. Its constructor locks the state directory, opens the Store and builds each component. If construction fails, it closes the Store and releases the lock.

Startup registers management routes and runs Setup. After setup resolves, it installs Session and Executor hooks, ensures the main Session exists, starts Policy syncs, resumes interrupted Assignments, and starts passes and effects. `ready` marks completion of startup.

Shutdown cancels turns, waits for passes, effects and agents, restores the process environment, closes the Store and releases the lock. Source plugins inject Runtime as the `orchestrator` service through the narrow `SourceHost` interface.

Part	Module	Does
Setup	`src/app/setup.ts`	Resolves the repository and the accounts by probing their CLIs the way worker shells run them, writes and applies the workers' environment and command shims, and keeps the setup problems; Settings checks a changed account through it, and restarts through it, which runs setup again
Passes	`src/app/passes.ts`	One pass at a time, every two seconds and whenever a command asks: the due effects, then `Engine.evaluateDue`; a Source's Trigger reaches the Engine through it
Outbox	`src/app/outbox.ts`	The Engine's `Outbox` port, and the only writer of the receipts: `send` keeps a message's receipt and queues its `reply` effect in one transaction, and the effect's handler sends it through `SourceRegistry.send`; a message waits while its Source is off, and is dropped when the conversation is gone or not listened to
Notices	`src/app/notices.ts`	Converts domain events and worker activity into main-Session messages. `eventNotice` handles domain notices; `reportRelay` forwards questions immediately, keeps ordinary progress on its Ticket, and relays other reports; `followupRelay` names the Tickets that own a human follow-up. Relays arriving during a turn are delivered together at its next step through the DeepSeek Harness inbox

Engine

`Engine` (`src/core/engine.ts`) is the only writer of Triggers, Threads, Decisions, Tickets and Assignments. It receives these ports from `src/core/ports.ts`: `Repositories`, a current-Policy getter, `SourceScope`, `Assessor`, `Executor` and `Outbox`. Optional dependencies are `prepareTicket`, `now` and `emit`.

All Engine methods are synchronous. Each command changes its aggregates in one transaction. `evaluateDue` uses a separate transaction for each Eval or Ticket it changes. Ports enqueue external work inside the transaction; adapters perform it after commit. Invalid requests throw `UserError`.

The Engine exposes one entry point and delegates to five internal parts. It owns each command's transaction. Commands spanning parts, such as `attach`, `evaluateDue` and `policyChanged`, call the parts in sequence.

Each part extends `EnginePart` (`src/core/part.ts`), which provides options and shared reads. Dependencies between the parts flow in one direction:

Part	Responsibility	Depends on
`Intake`	A Source's Trigger: stored once with its disposition, recorded on its Threads and their correlations, fanned out to the subscribed Tickets and, unless it is a person's message that reached one, queued for Eval on its primary Thread	Tickets
`Evals`	The Evals of Threads: batching, the gate's strength, actor and basis with the Trigger that gave them (`primary`), the deadline from the candidate's delivery and its shift by a pause, the requeue of an expired Eval, candidates, replays and hand-backs, the requeue on a Policy change, and the Decision each ends with: accept, stop, ignore or unavailable	Tickets
`Tickets`	The Ticket lifecycle: creation from an accept with its Mandate, Subscriptions and first Assignment, the Threads it follows and their closes (`followsDone`), attached Threads and Threads followed for context, Inputs and notes, resume and the messages a resume refuses, retry, restart, stop and end, the Subscriptions it drops, and the notice of a Policy change to the Tickets it bears on	Replies

Part	Responsibility	Depends on
`Assignments`	What the Executor and its workers report: an Assignment started, Inputs reached it or failed to, a restart interrupted it, and a worker's Report with its question for a person	Tickets, Replies
`Replies`	Channel writes through the `Outbox`: the acknowledgement, the answer, the result and the attention message, each by the rule that allows it	Nothing

Commands use imperative names such as `stopTicket`; facts reported by adapters use past-tense names such as `assignmentStarted`. The query `candidate` uses a noun.

Callers receive only their role's `Pick` of the Engine: `EngineForSources`, `EngineForMainSession`, `EngineForPeople`, `EngineForSettings` or `EngineForExecutor`.

Role	Method	Effect
Sources	`receive(trigger)`	Stores a Trigger once and answers `stored`, `duplicate`, or `filtered` when no Source receives it; fans it out to the Tickets subscribed to its Threads, where a person's message resumes an ended Ticket at once unless processing is paused, and queues it for Eval on its primary Thread, unless it is a person's message that reached a Ticket, which stays that Ticket's
Sources	`bindThread(ticketId, thread, own)`	Makes an open Ticket follow one of the Source's Threads, such as a pull request's, `own` when the Ticket opened it; a closed Thread bound again is open again. The Threads' closes, not a `ready` Report, end the Ticket

Role	Method	Effect
Sources	`threadClosed(thread, outcome, at)`	A Thread closed: each open Ticket that follows it records the close, and ends once it is done with the Threads it follows (`followsDone`): no Thread it opened is open, or, when it opened none, every Thread it follows closed. It ends `merged` or `closed` as the latest of those closes did, unless work resumed after the close has not reported `ready`, or its Executor took up, by acknowledging or answering it, a person's follow-up it decides after its latest Report other than `progress`, or the end of its last turn; a `dropped` Report then ends it as the close did
Main Session	`candidate(triggerId)`	The Eval that holds a Trigger, as its assessor sees it
Main Session	`candidateDelivered(evalId)`	The Eval's candidate reached the main Session: its deadline counts from now
Main Session	`decide(triggerId, outcome)`	Records an Eval Decision, or a direct Decision for an explicit request. Accept creates the Ticket, Mandate, Assignment, Subscriptions and acknowledgement atomically. An inferred candidate ignored from its card records `ignoredFromCard` before the supplied reason. While `followupOwners` still owns a human follow-up, only stop is allowed
Main Session	`attach(ticketId, triggerId, note?)`	Routes a Trigger to a Ticket it points at, whose Executor then decides it as a follow-up, and ends the Eval that holds it, whose other Triggers wait for a fresh Eval; an ended Ticket resumes under its Mandate at once unless processing is paused. A follow-up an Executor still decides is refused, as is one routed back to the Ticket that handed it back

Role	Method	Effect
Main Session	`requestStatus(ticketId)`	Asks a waiting Executor Session for its status, and answers whether it asked
Main Session	`note(ticketId, text)`	Gives a Ticket's Executor a fact from other work as a text Input, which grants nothing
Main Session, Executor	`reply(request)`	Queues a message for the Channel its purpose names: `acknowledgement`, `answer`, `result` or `attention`
People	`stopTicket(ticketId, reason?)`	A person's stop, which withdraws the Mandate
People	`retryTicket(ticketId)`	Ends the stalled Assignment of a Ticket that waits for a person after a `failed` or `blocked` Report, or whose Input delivery failed (`canRetry`), and starts a new one
People	`reevaluate(triggerId)`	Queues a stored Trigger for a fresh Eval
People	`unsubscribe(thread)`	Drops every Subscription on a Thread, and the Tickets that followed it follow it no more; each keeps deciding the follow-ups it received there
People	`dismiss(decisionId)`	Dismisses the problem an `unavailable` Decision shows, which the Decision records
Executor	`assignmentStarted(ticketId, assignmentId)`	An Assignment runs; its waiting Inputs go out
Executor	`inputsDelivered(ticketId, assignmentId, inputIds)`	Inputs reached the Assignment's Executor Session, where each waits for a step to take it

Role	Method	Effect
Executor	`inputsRead(ticketId, inputIds)`	The Executor Session took the messages that carry these Inputs into its history (a `user/message` Session event), from which its model has them; a step that claimed them and was cancelled or failed before does not count. Each counts as read once a Report or the end of a turn follows, also after a restart, and its Ticket's end does not let go of it; an Input delivered before reads were recorded counts as read once a Report or the end of a turn followed its delivery
Executor	`turnEnded(ticketId, assignmentId)`	A turn of the Executor Session ended, also after a Report, once nothing runs for it in the background or as it hands off to a person. What a step took before, it read (`inputsRead`)
Executor	`deliveryFailed(ticketId, assignmentId, error)`	Delivering Inputs failed; the Ticket shows it until a delivery succeeds, a new Assignment starts or the Ticket ends
Executor	`report(ticketId, report)`	Records a Report and its question for a person; `dropped` ends the Ticket, and `ready` completes one that follows no Thread, and ends one done with the Threads it follows as they ended
Executor	`markUnrelated(ticketId, thread, reason)`	Drops only that Ticket's Subscription to an attached Thread it found unrelated, and hands back, with the reason, the follow-ups there that the Ticket decides
Executor	`followForContext(ticketId, thread)`	Subscribes an open Ticket to a related Thread for context: its messages reach the Ticket but lend it no follow-up strength and do not resume it

Role	Method	Effect
Executor	`handback(ticketId, triggerId, reason)`	Records that the Ticket hands back a Trigger that reached it, with the reason its candidate carries, and queues it for a fresh Eval once no other Ticket decides it, if this Ticket did; it answers whether it queued it. A Ticket hands back each Trigger once and keeps its Subscription
Executor	`assignmentInterrupted(ticketId, assignmentId)`	An Assignment was running when the host stopped: one interrupted mid-work, or one that received an Input after its latest Report, starts again with a recovery note and the Inputs delivered to it that its Executor Session did not take, as a retry does; one that had reported since keeps its Session. The DshExecutor reports each at start (`resume`), oldest Ticket first, and queues again a launch that waits
Passes	`evaluateDue()`	Unless processing is paused: starts the Evals of the Threads whose queued Triggers wait, one per Thread with the Triggers queued meanwhile as one batch; queues the Triggers of an Eval past its deadline again, twice in a row, and records it `unavailable` the third time; and resumes the ended Tickets a person's message reached during a pause, if their Mandates still hold. An Eval whose candidate has not reached the main Session does not expire. While processing is paused, no Eval expires, and once it resumes each deadline moves later by the pause's length

Role	Method	Effect
Settings	`policyChanged(from, to, effects)`	Tells each live Ticket the change bears on how (`policyEffects` in `src/policy.ts`: a rule it was accepted under changed or no longer applies, its requester may no longer start work, or proactive work is no longer accepted), and restarts the Evals in progress under the new version

The Engine keeps batching, authorization, fan-out, resume and attention lifecycle rules private. Several shared predicates make these rules consistent:

- `waitsForPerson` (`src/core/model.ts`): the latest Assignment reported `blocked` or `failed`, and no human message has arrived since. Automatic Inputs and failed launches remain held while this is true.
- `followupOwners`: the Tickets still responsible for a human follow-up received on their own Threads. A Ticket can be live or ended without a human stop and waiting to resume. Handing back the message ends that ownership.
- `settled`: a Decision resolved the request; stop, unavailable, routed and refused-stop Decisions do not count.
- `ticketStage` (`src/core/stage.ts`): implements the stage table in 003 Execution.

When a Ticket ends, unread follow-ups return to Eval unless another Ticket still owns them or another disposition settled them. Acknowledged, answered and handed-back messages are excluded. For older records without acknowledgement dispositions, the outbox receipt supplies that fact. A final Report also covers messages waiting when the Assignment began, including the message that resumed it. The message that stopped a Ticket is not automatically re-evaluated.

After a change, the Engine emits an event from `DomainEvents` in `src/core/ports.ts`:

`trigger.received`, `eval.started`, `decision.recorded`, `ticket.created`, `ticket.resumed`, `ticket.ended`, `assignment.started`, `assignment.ended`, `thread.subscribed`, `thread.unsubscribed`, `trigger.requeued`, or `policy.changed`.

Runtime turns `ticket.ended`, `trigger.requeued` and `thread.unsubscribed` into notices. A human `trigger.received` delivered to Tickets becomes a follow-up relay. `pullEndNotices` sends one chat notice, using a stable id, when a Ticket ends merged or closed. Other events currently have no consumer.

Model

`src/core/model.ts` holds the domain types, and `src/core/stage.ts` the Ticket stages. A Decision records why as `reason`, a list of `Reason`s (`src/core/reasons.ts`): each a `code` with its `values` and, for one Trigger of a batch, its `triggerId`.

The core's codes, such as `timeout`, `policyChanged`, `routed` and `startedTicket`, are `REASONS`, with the English that agents read through `reasonText`; a gate or a Source scope gives its own, such as `not_a_member`; and `said` carries words an assessor or a person wrote. The panel words each code by `reason.`, and a Trigger a Source filtered records the Source's code the same way.

A Ticket keeps each fact once. It is open while it has no `ended`, which records when, by whom, why and with what outcome it ended; what it last said is its latest Assignment Report (`latestReport`); and its Executor Session is named after it (`workerSessionId` in `src/executor/dsh.ts`).

Its fields are typed: `origin`, the admissible Trigger that gave its Eval's batch its strength and the batch itself (`triggerId`, `triggerIds`); `execution`, what the Executor chose (`name`, `directory` and `assignment`); `delivery`, the last failed Input delivery (`error`, `at`), until a delivery succeeds, a new Assignment starts or the Ticket ends; and `follows`, the Threads a Source made it follow, each `{thread, own, closed?}`: `own` when the Ticket opened it, and `closed` with `at` and `outcome` once it closed. Tickets alone writes `follows`; the GitHub Source reads its pull requests with `pullsOf` and `pullOf` (`src/sources/github-pull.ts`).

An Input that carries a Trigger records whether a person wrote it, which `waitsForPerson` reads. A Thread also lists in `context` the subscribed Tickets that follow it only for context, and counts in `expired` the Evals of its queued Triggers that expired in a row; an Evaluation records its `primary` Trigger and when its candidate was `delivered`.

A Trigger's disposition records the Tickets it reached, by fan-out or attachment, a person's requeue, each Ticket that handed it back, with the reason, and each Ticket whose Executor acknowledged or answered it.

A Mandate's `basis` (`MandateBasis`) records how the gate admitted its Trigger. Revalidation reads the receiving `source`, `scope` and `row`, the listed `sender`, matched `rules`, and `via`:

via	Admission basis
`addressed`	An allowed sender addressed the agent
`followup`	A message arrived on a Thread with Tickets
`admission`	An admission-table cell; `observation` marks an inferred candidate

via	Admission basis
bot	A listed bot produced an inferred candidate

Trigger and Thread `meta` belong to the Source. The core does not interpret them.

Policy

`src/policy.ts` defines settings schemas, authorization and assessment rules.

`settingsPolicy(settings, scope)` returns a fresh, stateless `Policy` on each use:

- `version`, `paused` and `evalTimeoutMs` describe its current state.
- `gate(trigger, thread, tickets)` returns admission strength and basis.
- `revalidate(mandate)` checks whether an existing Mandate still holds.
- `admitsInferred(rules)` permits proactive acceptance only in auto mode with enabled matching rules.

`assessmentOutcome(assessment, settings, strength)` converts the model's assessment to an Engine outcome. It requires a judgement for every enabled rule, settled direction, a matching rule and evidence of intent. Outside auto mode, an inferred accept becomes a "would accept" record.

`policyEffects(tickets, old, now, scope)` identifies which open Tickets a change affects and how. `Engine.policyChanged` notifies only those Tickets. `policyVersion` hashes rules, accept mode and the Sources' Policy parts, including bot lists.

Store

`Store` (`src/app/store.ts`) owns one SQLite database. Runtime acquires the directory lock before opening it, preventing a second process from migrating or writing the same state.

Its interfaces are grouped by purpose:

- **Domain repositories:** `atomic`, plus `triggers`, `threads`, `tickets` and `decisions`, each with `get`, `save` and list queries. Indexed working sets are `threads.active()`, `tickets.open()`, `tickets.active()` and `read.subscribedThreads`.
- **Read queries:** `read` serves views, tools and Source polls through methods such as `threadTriggers`, `triggerStats`, `search` and `droppedReplies`. These methods never write.
- **Source metadata:** `threads.describe` and `threads.patchMeta` update Source-owned Thread metadata.
- **Settings:** `settings`, `saveSettings`, `blockRevision` and `settingsHistory`. Settings and prompts use numbered `Versions` (`src/app/versions.ts`).
- **Outbox receipts:** `replyStatus` and `receipts`, written only by Outbox. Receipt operations are `get`, `waiting`, `behind`, `add`, `settle` and `withdraw`. `behind` checks whether an earlier message for

the same Ticket and conversation is waiting with fewer than three failures. A withdrawn message settles with `channelGone` or `channelIgnored`. Read queries expose failed and dropped replies through `failingReplies`, `droppedReplies` and `droppedSince`.

- **Typed key-value records:** `get`, `set`, `delete` and `keys`, using entries from `KV`.

`afterCommit` runs an action only after the transaction commits, never after rollback. Runtime reads records in their stored shape; the schema 11 migration converts older supported records at startup.

Effects

`Effects` (`src/app/effects.ts`) is the durable side-effect queue. A command calls `enqueue(kind, key, payload)` inside its transaction; execution begins only after commit.

Each kind/key pair has one row. Enqueuing the same waiting key replaces its payload and makes it due immediately without changing queue order. `handle(kind, run, options)` registers one handler per kind. It processes due rows oldest first, up to the concurrency limit.

Failures retry with exponential backoff. Throwing `Later` defers work without counting a failure, for example when no worker slot is available. `start` begins processing after setup, `drain` stops it during shutdown while preserving pending rows, and `pending`/`idle` expose queue state.

Kind	Handler	Does	At once	Retry
`assessment`	MasterAssessor	Steers an Eval's candidate card into the main Session's turn, then tells the Engine it was delivered, which starts the Eval's deadline	4, none while paused	2 s doubling to 60 s
`launch`	DshExecutor	Launches a pending Assignment: prepare command, Executor Session, assignment message	The free worker slots, none while paused	A passing failure of the model provider or the network runs again after 5 s doubling to 300 s, three times; any other failure, and a failed prepare command, fails its Ticket. One held by a pause or by full worker slots tries again after 5 s

Kind	Handler	Does	At once	Retry
`delivery`	DshExecutor	Hands a Ticket's waiting Inputs to its running Assignment, in order	4, none while paused	2 s doubling to 60 s; one that waits for a worker slot tries again after 2 s
`relay`	Sessions	Steers a worker's report or question, or a person's follow-up, into the main Session's turn	1, in queue order	2 s doubling to 60 s
`notice`	Sessions	Injects a notice into the main Session without waking it	1, in queue order	2 s doubling to 60 s
`reply`	Outbox	Sends a message for a Channel through its Source, after the earlier messages of its Ticket to its conversation that have not failed three times; withdrawing the message cancels it	1, none while paused	5 s doubling to 300 s; one whose Source is off, or behind an earlier message, tries again after 5 s

Each kind has a health entry under its name. It reports an error once one of its effects failed three times in a row, except `delivery`, whose failures show on their Ticket instead. Relays keep their order, and so do notices; a failed one holds back the later ones of its kind.

Sources

Sources extend the host through `SourceKind`, defined in `src/sources/kind.ts`. A kind declares its identity, title, description, configuration `schema` and `fields`, derived `facts`, and `health` entries. Optional behavior includes `scope`, a `channel` (`allowed`, `send`, `link`), `threadContext`, `find`, `policy`, `environment` and `refreshLabels`.

`SourceRegistry` (`registry.ts`) owns registered kinds and stored instances, and implements the Engine's `SourceScope`. Registration claims health entries on `HealthBoard`; duplicate claims and reserved `sync:` names are refused. A SourceContext writes only through its registration's claimed entries, so reports after unregistration do nothing.

Registry configuration methods include `kind`, `instances`, `instance`, `validate`, `validatePolicy`, `policyPart`, `channelAllowed`, `declarations`, `environments`, `find` and `scope`. `find` asks each kind that supports search.

Generic code accesses Source behavior through the registry. `send(message, commands)` calls the kind's Channel and returns `sent`, `gone` or `waiting` if the kind is not installed. `link` resolves its Channel link. `threadContext` pairs a kind's reader with its configuration, or returns nothing when the kind has no reader. Invalid stored configuration disables the instance.

Plugins receive `SourceHost`: the registry and `context(kind)`. The kind itself uses a narrower `SourceContext`, built by `sourceContextOf` in `src/app/source-context.ts`.

That context supplies deployment and commands, setup readiness (`ready`, `configured`, `problem`), configuration, `receive`, `atomic`, and typed `state`. It also exposes Thread reads and metadata writes, Ticket reads and binding/closure commands, Trigger reads, `notify` through Outbox, and `report` for declared health entries.

DshExecutor

`DshExecutor` (`src/executor/dsh.ts`) provides one Session and directory per Ticket. `start` and `deliver` enqueue `launch` and `delivery` effects. `stop` cancels the turn through `Store.afterCommit`, so a rolled-back end does not stop the worker.

`prepare(ticket, outcome)` supplies the Engine's `prepareTicket` hook: short name, directory and `ticketExecution` assignment text. `workerSessionId(ticket.id)` derives the Session id.

`WorkerSlots` reserves capacity before a launch or a delivery that wakes an idle worker. A human message can use one slot beyond `maxWorkers`, and wakes have priority over launches. `activeCount` supplies panel load.

Transient provider and network failures get three retries. Launches use effect backoff; failed turns receive continuation messages after 15, 30 and 60 seconds. `inputMessage` renders each Input from current Ticket state, and `mandateInstructions` adds the matched-rule limits for inferred work.

`install` subscribes to Session events; `dispose` unsubscribes. A turn end normally reports waiting, except after readiness or while human action is required. Errors report failed. At startup, `resume` reports interrupted Assignments through `assignmentInterrupted` and requeues pending launches that do not wait for a person, oldest Ticket first.

MasterAssessor

`MasterAssessor` (`src/app/assessment.ts`) lets the main Session assess proactive candidates. `request` queues an assessment effect; `release` drops cached state when its Eval ends.

Its `context`, `read` and `decide` methods serve `orchestrator_eval`, `orchestrator_eval_read` and `orchestrator_eval_decide`. They provide bounded Thread context through `SourceRegistry.threadContext`, repository context, and decision submission. A kind without a context reader supplies only the Trigger and batch.

`assessmentOutcome` converts the model's answer for `Engine.decide`. Evals checks inferred acceptance against the current Policy again.

Settings

`Settings` (`src/app/settings.ts`) reads and writes revisioned settings blocks. `current` and `view` read them; `save(block, value, revision, origin)` validates a save; `pause(paused, origin)` changes the separate `paused` setting.

Every save declares its origin: `panel`, `tool` or `sync`. Only `panel` can change the execution environment or fields listed in `SETUP_FIELDS` (`src/deployment.ts`).

A Policy-version change calculates `policyEffects`, records which Tickets were notified, and calls `Engine.policyChanged`. `syncable` and `sync(id)` refresh a Source's Policy part. `refreshLabels(id)` refreshes display names.

Prompts

`Prompts` (`src/app/prompts.ts`) stores configurations as `Versions`. `current`, `version` and `history` read them; `validate`, `preview` and `save(value, baseRevision, source)` check and save them. `compose` and `skills` build Session text.

`builtInParts` lists override keys, roles, default text and required template placeholders. Each override stores team `text` and a `base`: the `builtInBase` key of its original default. Saves preserve that default under `KV.prompts.builtIn`. `defaultChanged` then reports changed defaults, their old text when available, and removed parts.

Validation inherits an omitted base from the saved override, drops removed parts already present in saved data, and rejects blank text or other unknown keys. This allows the save tool to round-trip a configuration it read. `environmentSection` and `policySection` substitute template values once, using own keys only. Configurations without overrides use defaults.

Sessions

`Sessions` (`src/app/sessions.ts`) manages DeepSeek Harness Sessions for both roles. `ensure` creates a Session with its role's model. `install` follows agent events and registers prompts, skills and tools for managed Sessions.

`prompt` delivers once per request id. `relay` and `notify` enqueue main-Session messages handled by the `relay` and `notice` effects. `src/app/notices.ts` defines their content; `delivery` checks whether a message arrived.

Each model request binds the Session to the current prompt revision. The revisions it used are stored durably: `promptVersion` identifies the active one and `mayRead` restricts which versions a worker can inspect. Registrations are tracked in one map per agent.

`status` and `promptVersion` serve prompt pages. Saves go through `Prompts.save` and notify the main Session. `modelRetry` exposes model-budget retry state.

Glue

Glue	Translates
`src/app/runtime.ts`	The composition root: builds the parts with the interfaces they use, registers the routes and the Source host, and starts and stops dsh-superagent
`src/app/notices.ts`	Domain events and worker reports into notices and relays for the main Session
`src/app/tools/main.ts`, `src/app/tools/executor.ts`	Tool arguments of the main Session and the Executor Sessions into calls of the core classes, and their results into cards
`src/app/http.ts`, `src/app/views/`, `src/app/api.ts`	HTTP requests into calls, and Store reads into the panel's views; `api.ts` types every route for the panel
`src/app/resources.ts`, `src/app/links.ts`, `src/app/cards.ts`	Resource names into reads, source messages into links, records into compact cards for models
`src/sources/github*.ts`, `src/sources/feishu*.ts`, `src/sources/poll.ts`	GitHub and Feishu payloads into Triggers and Thread facts, and messages into CLI calls
`src/deployment.ts`, `src/setup.ts`, `src/infra/`	Setup probes and problems, CLI transport, the agent's process environment

Dependency rules

- **Explicit, narrow dependencies.** Each class receives an interface that names what it uses, mostly as `Pick` types of other classes: `ExecutorHost`, `AssessorHost`, `SessionsHost`, `SettingsHost`, `SetupHost`, `PassesHost` and `OutboxHost` for the core classes, and `HttpHost`, `MainToolsHost`, `ExecutorToolsHost`, `ResourceHost`, `LinkHost` and `SourceContextDeps` for the glue, with the Engine as the `Pick` of the role that calls it. The Runtime passes itself only behind such an

interface. A Source plugin injects the `orchestrator` service, which its type names as a `SourceHost`: the registry and `context(kind)`; its kind's code sees only its `SourceContext`.

- **What may import what.** `src/core` imports nothing outside itself apart from Node's hashing, and does no I/O. `src/sources` imports nothing from `src/app`. The panel imports only types from the host side, and the plain values of `src/admission.ts`, `src/limits.ts`, `src/setup.ts`, `src/navigation.ts`, `src/sources/host.ts`, `src/core/errors.ts`, `src/core/reasons.ts` and the filter lists of `src/app/api.ts`.
- **One writer per state.** Each kind of state has one writer, and the others read it or ask that writer.

State	Writer
Triggers, Threads, Decisions, Tickets, Assignments	Engine
A Source's read positions, and the meta it keeps on its Threads (`label`, `url`, `facts` and its own keys)	That Source, through `SourceContext`
Settings and their history	Settings
Prompt versions	Prompts
Effect rows	Effects
Outbox receipts	The Outbox: the Engine's messages and Sources' `notify` go through its `send`, and its `reply` effects mark them
A health entry	The part that claimed it on the `HealthBoard` (`src/infra/health.ts`), which refuses a second claim: `runtime` the Runtime, which Passes report through, `evals` the Runtime after each pass, `models` Sessions, each effect kind Effects, `worker` the DshExecutor, a Source kind's entries the SourceRegistry for that kind's Source, `sync`: Settings

A Source changes a Ticket only through `SourceContext.tickets.bindThread` and `threadClosed`, which call the Engine.

- **Requests and facts.** A method that asks for something is named by an imperative verb and its object, such as `stopTicket` or `bindThread`, and may be refused; a method that records what an adapter did is named in the past tense, such as `assignmentStarted` or `deliveryFailed`. Each host receives the Engine as the role type it needs.
- **Typed fields, and Source data under the Source.** The code reads typed fields of Tickets and Mandates. A Source keeps nothing on a Ticket: which of its Threads a Ticket follows is the Ticket's `follows`, and what it keeps about a Trigger or a Thread is in their `meta`.
- **Fixed refusals.** A refusal a person or an agent can cause is a `UserError(code, values)` from `USER_ERRORS` in `src/core/errors.ts`. The management API answers a `missing` code with 404 and the others with 400, and the panel words each code in its language. Setup problems are

`SetupError` codes (`src/setup.ts`) and fixed health texts `HealthCodes` (`src/sources/host.ts`), worded the same way.

Source-specific dependencies

Some generic modules still refer directly to GitHub or Feishu:

- Outbox applies the Feishu chat word limit.
- `src/app/links.ts` resolves Feishu message links and `repliedTo`. It and Runtime read settings with `feishuSettings`.
- `src/app/resources.ts` reads the `feishu/` and `github/` originals; `src/app/cards.ts` uses `pullOf` for Ticket cards.
- Runtime sends `pullEndNotices` for merged or closed Tickets.
- Executor tools bind reported pull requests with `readPull` and `claimPull`, using the GitHub context exposed by Runtime.
- MasterAssessor reads repository files with `readRepositoryPath`.
- Sessions and Prompts register built-in skills only for GitHub and Feishu. Migration converts their schema-10 records.
- Setup probes only GitHub and Feishu account fields (`SETUP_FIELDS`). Only those Sources receive account setup problems through `SourceContext.problem`.

Modules

Path	Responsibility
`src/index.ts`	The main plugin entry: reads the machine settings, starts the Runtime, and gives it the restart that a save of a Source's account uses
`src/deployment.ts`	The plugin `Config` and `Machine`; `resolveDeployment` and `resolveFeishuBot` read the repository and the accounts from the Source settings, each CLI run as Executor shells run it; `SETUP_FIELDS` lists the Source fields setup reads. The code names no person, account or organization
`src/setup.ts`	Setup problems: `SetupError`, with a code, the place that fixes it and the English text the log, the main Session and the panel share
`src/policy.ts`	The Policy: the settings schemas by block, the Policy version, the gate and Mandate revalidation
`src/admission.ts`	The admission table's values, each cell's options and defaults, shared with the panel
`src/limits.ts`	The limits of the values a person sets, which the settings schemas check and the panel's fields offer

Path	Responsibility
src/kv.ts	KV: every family of kv records with its key, its value's type and its owner, and the Key and Family types the Store and SourceContext.state take
src/app/lock.ts	The state directory's lock: one dsh-superagent per state tree, with a stale window short enough that a restart does not find the previous process's lock still legitimate
src/navigation.ts	Opens a DeepSeek Harness Session from the panel once the host's catalog lists it
src/core/model.ts	The domain types, among them the typed Ticket and MandateBasis, with threadKey, ticketOutcome, reached, followupOwners, settled, waitsForPerson, liveTicket, runningAssignment, latestAssignment and latestReport
src/core/ports.ts	The ports: Repositories, Policy, SourceScope, Assessor, Executor, Outbox, and the domain events
src/core/engine.ts	The Engine, the core's entry point, and the role interfaces EngineForSources, EngineForMainSession, EngineForPeople, EngineForSettings and EngineForExecutor
src/core/part.ts	EnginePart, the base of the Engine's parts, and EngineOptions
src/core/intake.ts	Intake: Triggers from Sources, to their Tickets and, unless a person's message reached one, to Eval
src/core/evals.ts	Evals: the Evals of Threads and their Decisions
src/core/tickets.ts	Tickets: the Ticket lifecycle, the Threads a Ticket follows, Subscriptions and Inputs
src/core/assignments.ts	Assignments: the Executor's facts and its workers' Reports
src/core/replies.ts	Replies: the Engine's Channel writes
src/core/stage.ts	ticketStage and canRetry: the Ticket stages of 003 Execution
src/core/errors.ts	UserError and USER_ERRORS
src/core/reasons.ts	Reason, REASONS and reasonText: why the core recorded a Decision or set a Trigger aside, by code
src/core/ids.ts	key: ids derived from their inputs, equal for equal inputs
src/core/memory.ts	MemoryRepositories: the Store's repositories in memory, in the Store's order and with its nested rollback, for domain tests
src/core/index.ts	The core's exports
src/app/runtime.ts	The Runtime, the composition root, with its start and stop
src/app/setup.ts	Setup, and setupChanged, whether a save changes a field setup reads
src/app/passes.ts	Passes, the pass scheduler

Path	Responsibility
src/app/outbox.ts	The Outbox: the Engine's `Outbox` port, the receipts and the `reply` effects that send them
src/app/notices.ts	The main Session's notices and relays: `eventNotice`, `reportRelay`, `resultRelay` and `followupRelay`
src/app/store.ts	The Store, and `Receipt`, a message of the outbox
src/app/store-read.ts	`storeReads`: the Store's read queries for the views, the tools and the Sources' polls, which never write
src/app/migration.ts	The schema 11 migration: one step from schema 10 that writes every stored record in its current shape
src/app/effects.ts	Effects
src/app/versions.ts	`Versions`: a record kept as numbered snapshots with a pointer to the current one, for the settings and the prompts
src/app/cache.ts	`TextCache`: texts kept in memory within a size, the least recently used dropped first
src/app/settings.ts	Settings
src/app/settings-history.ts	`policyHistory`: each saved settings revision with what it changed from the one before
src/app/prompts.ts	Prompts, `builtInParts`, the built-in parts a team may override, and `promptParts`, the order of a role's prompt sections
src/app/context.ts	The built-in prompt text, read from `prompts/`, and the transcript entries Sessions read
src/app/skill-files.ts	Packaged text: `promptFile`, which joins a file's one-sentence lines into paragraphs, and the bundled skills
src/app/sessions.ts	Sessions
src/app/delivery.ts	`AgentDelivery`: delivers a message to a DeepSeek Harness agent once per request id, also across a restored Session
src/app/retry-health.ts	`RetryHealth`: which Sessions wait on a model budget retry, for the `models` health entry
src/app/role-models.ts	Resolves the Eval and Executor models, and seeds a new Session with its role's model
src/app/assessment.ts	The MasterAssessor, the assessment schema and the window `orchestrator_eval` answers
src/app/execution-environment.ts	The execution environment: its variables, a Ticket's values, the directory root and the lines of the prompt section, which `src/app/prompts.ts` fills into its template
src/app/source-context.ts	`sourceContextOf`: each Source kind's `SourceContext`
src/app/links.ts	`sourceLink`, the page of a Trigger's message, and `candidateCard`

Path	Responsibility
src/app/cards.ts	Compact cards of Triggers, Tickets, Threads and Decisions for models, the closed set of ticket card states TICKET_STATES, and textPage
src/app/resources.ts	RESOURCES and readResource: what orchestrator_read reads, for each role
src/app/tools/tool.ts	The Tool type, readTool, the orchestrator_read of both roles, and AcknowledgementSchema
src/app/tools/main.ts	The main Session's tools
src/app/tools/executor.ts	An Executor Session's tools
src/app/http.ts	The /orchestrator/api routes: the same-origin check, request limits and one status per kind of failure
src/app/api.ts	Api, every route with its parameters, query, body and response, and the filters of the lists; it imports only types, so the panel bundles it
src/app/views/text.ts	The two forms of a view's text: preview, plain text on one line, and markdown without HTML; plain and readable
src/app/views/records.ts	The rows the views share: Triggers, Tickets, Threads, Decisions and Channel links, and ownersOf, the owners of follow-ups as one request reads them
src/app/views/tickets.ts	The Ticket lists by stage, a Ticket's page, the Mandate basis facts and the Executor's load
src/app/views/threads.ts	The Thread lists, a Thread's timeline, the search and the Decision list
src/app/views/sources.ts	The Source status: each Source instance's state, its last day's counts, and what it filtered and dropped in that day
src/app/views/settings.ts	The settings page's view: each block with its revision, each Source's settings with its kind's form, and the execution environment
src/app/views/overview.ts	The overview: Tickets that need a person, problems, the last day's flow and the Executor's load
src/executor/dsh.ts	The DshExecutor, WorkerSlots, ticketExecution, inputMessage and mandateInstructions
src/executor/background.ts	A worker's background work in DeepSeek Harness, the shell jobs its Session owns (ctx.jobs) and its subagents (ctx.subagents): runningJobs, untilQuiet, killJobs, collectJobs, stopSubagents, stopBackground, until, and cancelled, the workers whose turn a stop of their subagents cancelled

Path	Responsibility
src/infra/commands.ts	Commands, the CLI transport: run; api and pages, limited to the configured repository and validated; lark as the bot; git. CliError carries the code a command's output reports, such as a GitHub HTTP status, and a command that prints nothing fails with how it ended, never with its arguments; validItems skips and counts the malformed items of a list
src/infra/environment.ts	The agent's process environment: command shims, the git configuration and credential helper, runAsWorker, and the checks of the environment and of a Ticket's directory
src/infra/paths.ts	The package's root, where packaged files are
src/infra/text.ts	Word counts and the chat message limit, fill and sameName
src/infra/health.ts	The HealthBoard, every part's health entry, each claimed by its one writer
src/infra/secret-file.ts	The one owner-only rule for a secret file: readSecretFile and requireOwnerOnly, which hold the 0600 check where the platform has POSIX bits, name the mode a refused file actually had, and leave Windows to its ACL
src/infra/hosting-api.ts	The code-hosting HTTP client: HostingApi and createHostingApi, a token read from its 0600 file, PRIVATE-TOKEN, an allowlisted /projects path and a 30-second timeout
src/infra/yunxiao-api.ts	The Yunxiao HTTP client: YunxiaoApi, createYunxiaoApi and YunxiaoHttpError, its token file read on every call, the /oapi/v1/ path limit and the organization allowlist, JSON and multipart bodies, and a 401 that renews an OAuth token once before retrying
src/infra/yunxiao-oauth.ts	Yunxiao OAuth 2.0 sign-in: authorization-server metadata discovery, dynamic client registration, PKCE S256, a 127.0.0.1 loopback callback, the token exchange and refresh, and the owner-only token file (with the refresh material beside it) the rest of the plugin already reads
src/sources/kind.ts	The SourceKind protocol
src/sources/instance.ts	Instance id normalization: instanceId, kindOf and storesKind
src/sources/registry.ts	SourceRegistry
src/sources/host.ts	SourceHost, SourceContext, Health, health and the fixed health texts
src/sources/poll.ts	What GitHub and Feishu polling share: POLL_HORIZON_MS, polled, cursorAt, and pollThreads, which keeps a failure on its Thread as pollError

Path	Responsibility
src/sources/github.ts	The GitHub kind and plugin entry, exported as ./github-watch: a poll round of notifications, the pull requests of open Tickets with their CI, and subscribed Threads
src/sources/github-trigger.ts	GitHub comments, reviews, Issue bodies and CI changes as Triggers, and GitHub addresses
src/sources/github-pull.ts	The pull request rules: which pull requests a Ticket follows (claimPull, observePull), the closes it reports to the Engine, and pullEndNotices, the chat notice of a Ticket that ended merged or closed
src/sources/github-channel.ts	The GitHub Channel: comments and review thread replies, sent or gone
src/sources/github-read.ts	CI snapshots, repository reads, the github/ resources, and githubThreadContext: an Issue's or pull request's conversation by page
src/sources/feishu.ts	The Feishu kind and plugin entry, exported as ./feishu-watch
src/sources/feishu-trigger.ts	Feishu messages as Triggers, by one rule for live events, history pages and topic pages
src/sources/feishu-loops.ts	The Feishu loops: the event consumer, history recovery, chat names and polling of subscribed topics
src/sources/feishu-channel.ts	The Feishu Channel: topic replies, sent or gone
src/sources/feishu-read.ts	The feishu/ reads of a Trigger's message, its root, parent and topic, and feishuThreadContext, which gives them to assessments with their images
src/sources/admission-context.ts	The bounded window an assessment starts from: the first pages of a Trigger's Thread, read through its kind, and the messages around the Trigger
src/sources/feishu-image.ts	Reads a Feishu message's image as an attachment for the assessing Session
src/client/index.tsx	The client plugin entry: the sidebar entry, the main panel and the page in DeepSeek Harness's Plugin Manager
src/client/app.tsx	The panel: header, search, glossary, tabs and the page of the current route
src/client/provider.tsx	What every page reads from its host: copy, calls, navigation, notices, the state poll, and restart, which reads the state quickly while dsh-superagent restarts
src/client/api.ts	Calls and polls of the management API, typed by src/app/api.ts, and the Decision filters of the Eval status
src/client/route.ts	The tabs, the settings groups, the typed routes that parseRoute reads, and the route memory with the settings group opened last

Path	Responsibility
`src/client/overview.tsx`	The overview
`src/client/source.tsx`, `trigger.tsx`, `eval.tsx`, `execution.tsx`	One file per stage: its status, and its settings where it has some
`src/client/thread.tsx`, `ticket.tsx`	The record pages: a Thread with its timeline, and a Ticket with its authorization, Inputs and Assignments
`src/client/settings.tsx`	The Settings tab: the group navigation and the group on screen
`src/client/settings-kit.tsx`	The building blocks of settings blocks: the settings the stage groups share, `useBlockDraft`, the fields, among them a Source kind's, and the history of saves
`src/client/prompts.tsx`	The Prompts settings: the page owns the view it read and its editor the draft, and the Edit, Full prompt and History tabs render from them; a built-in part's editor shows a changed default and keeps or drops the override
`src/client/prompt-config.ts`	The prompt configuration by section, by the team sections, skill names, built-in parts (`part/`, with `withOverride`) and part headings the prompt views give, and the line diff of the full prompt, as pure functions
`src/client/deployment.tsx`	The Deployment settings, also shown in DeepSeek Harness's Plugin Manager
`src/client/draft.tsx`	Settings drafts: `draftReducer`, their states and their merge with a newer saved revision, `saveBarText`, the save bar's line, and the save bar
`src/client/controls.tsx`	Local copies of DeepSeek Harness's controls, which follow its markup, keyboard behaviour and design tokens, since the bundle does not import `@deepseek-ai/dsh-client-ui-primitives`; they read no context
`src/client/widgets.tsx`	The small components the pages share, which read the panel's context: sections, page heads, load states, times, links, confirmed actions, the setup check and Markdown
`src/client/rows.tsx`	The rows the pages share: a Ticket, a Thread and a Decision, a Thread's chip, and the folding of runs of automated messages
`src/client/markdown.ts`	The Markdown subset of reports and questions, and the bare addresses in messages, as blocks and inline parts
`src/client/text.ts`	Copy by key and the wording of domain values, reasons, senders and times
`src/client/marks.ts`	The marks of the states the views classify: a Ticket's or a Thread's stage, a Source's state and a Decision's outcome, each as a dot, an icon, a tone and a label

Path	Responsibility
src/client/locales.ts	All copy in both languages, and the glossary of 001 Domain model
src/client/client.css, css.d.ts	The panel's stylesheet, loaded as text
src/sources/chat/port.ts	The chat port: ChatEvent, InboundTransport and ChatAdapter, the interface a chat backend implements
src/sources/chat/dedupe.ts	DeliveredRing, the transport's bounded ring of delivered message IDs, which drops redeliveries
src/sources/chat/kind.ts	chatKind, the factory that turns a ChatAdapter into a chat SourceKind
src/sources/chat/weixin/backend.ts	BridgeEndpoints and the WxRoom/WxPerson types: the bridge client's constants and types
src/sources/chat/weixin/bridge.ts	createBridgeClient, the sidecar bridge's HTTP client: health, status, rooms, members, messages and send, with its token file, timeouts and retryable failures
src/sources/chat/weixin/kind.ts	weixinKind / createWeixinKind, the Weixin chat SourceKind: its schema and fields, the structured @ scope rule, the bridge-backed Channel and its environment
src/sources/chat/weixin/media.ts	readWeixinImage and sniffImageMediaType: downloads a message image from the sidecar, trusts only its magic bytes, and commits it as an attachment
src/sources/chat/weixin/thread.ts	weixinThreadContext, the Weixin ThreadContext: the conversation page, the replied-to message and the images one carries, read through the bridge client; root answers no_root because WeChat has no topic root
src/sources/chat/weixin/read.ts	Worker-facing WeChat reads: weixinRef/weixinReferences/weixinResource and readWeixin for weixin//message, /thread and /around, the same shapes as the Feishu resource grammar
src/sources/hosting/port.ts	The code-hosting port: HostAdapter, ChangeRequest and ChangeNote
src/sources/hosting/kind.ts	hostingKind, the factory that turns a HostAdapter into a hosting SourceKind
src/sources/hosting/gitlab/adapter.ts	The GitLab HostAdapter: createGitlabAdapter, the mapping of a merge request, note and pipeline, and its CI snapshots
src/sources/hosting/gitlab/kind.ts	gitlabKind, the GitLab hosting SourceKind on the GitLab adapter, with organizationId, hostUrl and its environment
src/sources/work/port.ts	The work-item port: WorkAdapter and WorkItem
src/sources/work/images.ts	Work-item images: parse inline ` references, download through the files API, de-duplicate by sha256 under /projex-images`, and serve one back as an attachment

Path	Responsibility
src/sources/work/kind.ts	workKind, the factory that turns a WorkAdapter into a work SourceKind
src/sources/work/context.ts	workThreadContext: a work item read as a conversation - the description as the root message, the comments as the thread with paging, the agent's own comments marked bot, and the archived images served back as attachments
src/sources/work/environment.ts	workEnvironment: the per-ticket variables naming the work item a worker is on (_PROJECT, _WORKITEM, _SERIAL, _CATEGORY, _STATUS, _URL), plus whatever a host adds, such as a Projex organization
src/sources/work/feedback.ts	reflectWorkTickets: the status a work item shows while a ticket works on it and after it ended, written once per phase through a KV ledger, so the board follows the ticket rather than staying untouched
src/sources/work/projex/adapter.ts	The Projex WorkAdapter: createProjexAdapter, work items and notes by category with nextToken paging, and its marker-idempotent replies
src/sources/work/projex/kind.ts	projexKind, the Yunxiao Projex work SourceKind on the Projex adapter, with organizationId, its environment and the refreshLabels that fills the status dropdowns
src/sources/work/projex/read.ts	Worker-facing Projex reads: projexRef/projexReferences/projexResource and readProjex for projex//item and /comments
src/sources/hosting/stream.ts	pollHosting, the hosting polling round: the cursor, the changed change requests, their notes and CI, and the overflow count
src/sources/work/self-writes.ts	The self-write ledger: the work-item revisions this process wrote, which a poll must not read back as new requests, because a comment or a status write bumps the modification time a Trigger ID is derived from
src/sources/work/stream.ts	pollWorkItems, workTrigger, noteTrigger and observeNotes: the work-item polling round, plus per-followed-item comment polling with a per-item watermark and a bounded backfillComments first-sight backfill
src/sources/chat/dingtalk/adapter.ts	The DingTalk adapter: parseStreamEvent and sendMessage over the sessionWebhook and oapi paths, with its idempotency marker
src/sources/chat/dingtalk/stream.ts	startStream and createLongPollingStream: consume the injected event stream, report its health and reconnect until aborted

Path	Responsibility
src/sources/chat/dingtalk/kind.ts	dingtalkKind / createDingTalkKind, the DingTalk chat SourceKind: its schema and fields, the @-based scope, the stream-backed Channel and the ThreadContext that reads only the callback's message
src/sources/chat/dingtalk/read.ts	Worker-facing DingTalk read: dingtalkRef/dingtalkReferences/dingtalkResource and readDingtalk for dingtalk//message
src/sources/chat/wecom/crypto.ts	WeCom callback crypto: signatureOf/verifySignature, and encrypt/decrypt of the AES-256-CBC payload
src/sources/chat/wecom/adapter.ts	The WeCom callback adapter: parseXml, and createWecomSender with its cached access_token and the message send
src/sources/chat/wecom/webhook.ts	handleWebhook, the pure callback handler: verify, decrypt, parse, dedupe, and the URL-verification echo
src/sources/chat/wecom/kind.ts	wecomKind / createWecomKind, the WeCom chat SourceKind: its schema and fields, the @-based scope, the callback Channel and its facts, wecomTriggerOf and the ThreadContext that reads only the callback's own message
src/sources/chat/wecom/read.ts	Worker-facing WeCom read: wecomRef/wecomReferences/wecomResource and readWecom for wecom//message, the one message a callback carried
src/sources/dingtalk-watch.ts	The DingTalk plugin entry, exported as ./dingtalk-watch: registers the DingTalk kind and runs its injected stream
src/sources/wecom-watch.ts	The WeCom plugin entry, exported as ./wecom-watch: registers the WeCom callback kind and mounts its public signed callback route at /wecom/, turning each callback into a Trigger
src/sources/weixin-watch.ts	The Weixin plugin entry, exported as ./weixin-watch: registers the Weixin kind and polls its sidecar bridge for new messages in the configured chats
src/sources/gitlab-watch.ts	The GitLab plugin entry, exported as ./gitlab-watch: registers the GitLab kind whose note channel posts through the instance's bound adapter
src/sources/projex-watch.ts	The Projex plugin entry, exported as ./projex-watch: registers the Projex kind whose comment channel posts through the instance's bound adapter

src/sources/github.ts and src/sources/feishu.ts are also the entries of the two Source plugins, built by tsdown.config.ts and exported in package.json as ./github-watch and ./feishu-watch. The bundle's cordis.patch.yml inserts the three plugins, with Feishu disabled.

Tests

Test	Covers
`tests/core-engine.spec.ts`	Domain rules on in-memory repositories: receiving, Eval, fan-out, resume and stop, interrupted Assignments, the pause, Policy changes, reports and outcomes, attaching, replies, failed deliveries and dismissed problems
`tests/stage.spec.ts`	The Ticket stages, row by row
`tests/policy.spec.ts`	Source scope, the gate, the admission table, Mandate revalidation, the Policy version, settings migrations and assessment outcomes
`tests/registry.spec.ts`	`SourceRegistry`: instances, validation, Policy parts, Channels and Thread context
`tests/instance.spec.ts`	Instance ids: `instanceId`, `kindOf` and `storesKind`, and the Source instance settings with and without a `kind`
`tests/sources.spec.ts`	The Feishu and GitHub Sources on a `SourceContext`: translation, polls and read positions, the pull request rules, health
`tests/commands.spec.ts`	CLI calls as the deployment's accounts, the Channels' answers, and repository-scoped GitHub calls
`tests/feishu-read.spec.ts`, `tests/github-read.spec.ts`	Feishu and GitHub reads
`tests/admission-context.spec.ts`, `tests/feishu-image.spec.ts`	The Thread window of assessments, and Feishu image reads
`tests/assessment.spec.ts`	What the MasterAssessor keeps per Eval, and when it lets go
`tests/background.spec.ts`	Waiting for a worker's shell jobs and subagents: an end that woke it, and a host that stops
`tests/bundle-patch.spec.ts`	The bundle's patch, read as DeepSeek Harness reads it, and the agent preset of Executor Sessions it defines
`tests/effects.spec.ts`	The effect queue: coalescing, limits, retries with backoff, health and shutdown
`tests/passes.spec.ts`	Passes: one at a time, the ones asked for during a pass run as one after it, and the runtime health a failed pass reports
`tests/outbox.spec.ts`	The Outbox on a running Runtime: messages wait while processing is paused or their Source is not installed
`tests/cache.spec.ts`	`TextCache`
`tests/queue.spec.ts`	The Runtime's work through the effect queue, and what earlier versions stored
`tests/followups.spec.ts`	A person's follow-up on a Ticket's Thread: its notice to the main Session, the tools that act on it, what a burst costs; a worker's hand-back and context follow, and the main Session's note

Test	Covers
tests/health.spec.ts	One health entry per part, each cleared by its own next success
tests/delivery.spec.ts	AgentDelivery
tests/retry-health.spec.ts, tests/role-models.spec.ts	Model budget retries, and the models of the two roles
tests/runtime.spec.ts	The whole application on a simulated DeepSeek Harness host, through tools and HTTP
tests/tools.spec.ts	The Sessions' tools: what orchestrator_read reads for each role, and the shapes and checks of the other main Session tools
tests/runtime-concurrency.spec.ts	Work that overlaps: Triggers during a pass, Tickets that change during deliveries and launches
tests/http.spec.ts	The same-origin and loopback checks, the status of each kind of failure, the body limit, path parameters and queries
tests/settings.spec.ts	Settings saves that race a model or account check, a Policy sync that a save outran, and the refusals of the pause, a sync and a block
tests/views.spec.ts	What the management views derive from the Store, such as the Thread a dropped message belongs to
tests/timeline.spec.ts	A Thread's pages and a Trigger's neighbours by time and then id, with every received Trigger's time in one form
tests/migration.spec.ts	The schema 11 migration: every stored record in its current shape, and the refusal of older schemas
tests/store.spec.ts	Repositories and their working sets, in SQLite as in memory, the kv area, with every key of KV spelled only in src/kv.ts and the migration, settings history and queries
tests/secret-file.spec.ts	The secret-file rule on both platforms: a POSIX file refused with the mode it had, a missing or empty one always refused, and Windows accepting what it cannot express
tests/prompts.spec.ts	Prompt validation, assembly and versions
tests/cards.spec.ts	The rules the built-in text must state, file by file
tests/model-surface.spec.ts	Everything models read: tools, prompts, skills and inputs, checked against the tool schemas, the execution environment and the glossary, within a size budget per role
tests/deployment.spec.ts, tests/environment.spec.ts, tests/index.spec.ts	The plugin configuration and setup, the agent's process environment, the plugin entry
tests/service-script.spec.ts	scripts/dsh-superagent against fake ps, tmux and dsh
tests/client.spec.tsx, tests/navigation.spec.ts, tests/controls.spec.tsx	The client plugin's registration and polling, Session navigation and route memory, the local controls, and the panel's imports, which have no cycle

Test	Covers
tests/panel-overview.spec.tsx, panel-source.spec.tsx, panel-trigger.spec.tsx, panel-eval.spec.tsx, panel-execution.spec.tsx	Each page of the panel, against a fake management API answered by route
tests/panel-settings.spec.tsx, panel-prompts.spec.tsx, panel-deployment.spec.tsx, panel-panel.spec.tsx	The settings groups, drafts and history, the prompts, the Deployment page, and the panel's search, glossary and route memory
tests/draft.spec.ts, tests/prompt-config.spec.ts, tests/markdown.spec.ts, tests/marks.spec.ts	Draft states, merges and the save bar's line; the prompt configuration by section and the line diff; the Markdown subset and its links; the mark of every state the views classify
tests/locales.spec.ts, tests/styles.spec.ts	The copy in both languages, the stylesheet
tests/class-docs.spec.ts	Every class in src has a JSDoc block before it
tests/docs.spec.ts	Numbered documents link only to earlier ones, every local link resolves, no document has merge conflict markers, and this document's tables list every module and every test
tests/kv-instances.spec.ts	The instance dimension of the KV families and health keys: one logical key per instance in its own store key, and healthEntry's 实例:条目 form
tests/registry-instances.spec.ts	Several instances of one kind resolved from one settings map, each with its own health entries and its own outbound ChannelHost
tests/hosting-port.spec.ts	hostingKind's schema, scope and channel capabilities over a stub HostAdapter
tests/chat-dedupe.spec.ts	DeliveredRing's bounded deduplication, and the chatKind built on a stub InboundTransport
tests/work-port.spec.ts	workKind's category-to-row mapping and its outbound replies over a stub WorkAdapter
tests/work-context.spec.ts	The work thread context: the description as root, the comments as the thread across pages, the item read once per conversation, a failed read retried, the agent's own comment marked bot, and an archived image served back
tests/work-environment.spec.ts	The shared work environment: variable names per host, item values from the Trigger's channel address or its meta, empty values for a ticket that started elsewhere, and the host's own additions
tests/work-feedback.spec.ts	reflectWorkTickets: the start status once, the done status after the ending, the already-current status skipped but recorded, a failed write retried, a forgotten ticket dropped, and the read-only host left alone
tests/weixin-bridge.spec.ts	The Weixin bridge client over a stub fetch: its six endpoints, the bearer token file's mode, the event mapping, the idempotency key and the failure codes

Test	Covers
tests/weixin-watch.spec.ts	The Weixin polling loop's listen set: the configured ids it pushes each round, the chats the backend could not resolve, and the two cases where it must send nothing
tests/execution-directory.spec.ts	The ticket directory root: an absolute path on either platform (/x, C:\x, C:/x, \\server\share), refused when relative, in both the schema and the input's own pattern
tests/state-lock.spec.ts	acquireStateLock: takes a free state directory, takes over a lock whose holder is gone, waits out one that is briefly held, and names the remedy when it is really held
tests/state-lock.spec.ts	acquireStateLock: takes a free state directory, takes over a lock whose holder is gone, waits out one that is briefly held, and names the remedy when it is really held
tests/person-notice.spec.ts	personNotice: the one line a person gets when their message is refused, per reason code and the fallback
tests/outbound-file.spec.ts	resolveTicketFile, the gate between a ticket's model and the machine's files: a path that climbs out, a symlink that leaves the directory, an executable, an empty file, an oversized one, and a ticket with no directory
tests/weixin-kind.spec.ts	The Weixin SourceKind: its schema and fields, the structured @ scope rule, the Channel and policy, the environment and its facts, its search terms and reply mapping
tests/weixin-read.spec.ts	WeChat worker reads: the reference and resource grammar, message/thread/around shapes with their next_resource, the no_room/not_found answers, and the root/parent/thread ThreadContext parts
tests/hosting-gitlab.spec.ts	The GitLab adapter and the hosting HTTP client: the merge request, note and pipeline mappings, paging, idempotent replies, and the token and path checks
tests/hosting-gitlab-kind.spec.ts	The GitLab hosting SourceKind: its identity, schema and fields, its environment and its scope rows
tests/source-actions.spec.ts	Source card actions: the WeChat kind offers start/stop/restart/check, each calls schtasks with a fixed argv against the configured task, readiness follows the task name, and a name that is not a task name is refused by both the schema and the runner
tests/client-bundle-id.spec.ts	The built client bundle registers under the package name, and the build banner derives that name from package.json instead of repeating a literal

Test	Covers
tests/panel-copy-and-order.spec.ts	Two page rules: no Markdown markers in any Source's copy or in either UI dictionary, and Source cards ordered enabled first then by English name, leaving the input untouched
tests/admission-tables.spec.ts	Every Source's admission table reads the same: a chat names one row per conversation plus other chats and direct messages (and its scope reports those ids), WeChat does the same for its groups, and GitLab declares a Policy with the four reception ranges
tests/work-admission-rows.spec.ts	The Projex rows are built from an instance configuration that also holds the host's keys: a whole config builds them, a category off the list is marked not received with no cells, and the Policy and Source blocks the panel sends validate
tests/work-policy.spec.ts	Projex admission: an assignment is the direct request, an unassigned item stays ambient, a listed member's assigned item is explicit, an unlisted one is refused, and the three category rows are offered
tests/work-projex.spec.ts	The Projex adapter: work items and notes by category, `workitems:search` paging, idempotent replies, the project roster, attachment upload with its `embedHtml`/link fallback, the RICHTEXT comment body, the workflow status list, and the Yunxiao client's checks (JSON and multipart bodies included)
tests/work-projex-kind.spec.ts	The Projex work `SourceKind`: its identity and fields, its schema, its scope rows, its environment and Channel capabilities, its search resolution, its member sync, and its status-list action
tests/yunxiao-oauth.spec.ts	Yunxiao OAuth: PKCE, metadata discovery, the full browser sign-in against an injected fetch and a real loopback callback, a forged-state rejection, the token file, and renewal
tests/dingtalk-adapter.spec.ts	The DingTalk adapter over injected `fetch`/`connect`: the stream-event parse, the sessionWebhook and oapi sends, the idempotency marker, the failure codes and the stream's abort
tests/dingtalk-kind.spec.ts	The DingTalk `SourceKind`: its identity and schema, its fields, its policy, its capabilities, its `@`-based scope and the ThreadContext parts
tests/dingtalk-read.spec.ts	DingTalk worker reads: the reference and resource grammar, the one message, and the `no_root`/`no_parent` parts
tests/projex-read.spec.ts	Projex worker reads: the item with its status and images, a paged comment thread with the agent's own comment marked, and the reference grammar

Test	Covers
tests/wecom-crypto.spec.ts	The WeCom callback crypto: the sorted signature, key validation without naming the key, and the encrypt/decrypt round trip
tests/wecom-webhook.spec.ts	The WeCom callback handler and sender: the XML parse, the URL verification, signed events, dedupe and the cached access token
tests/wecom-kind.spec.ts	The WeCom `SourceKind`: its identity and schema, its fields, its two-branch scope, its at-least-once capabilities and facts, the callback message and the ThreadContext parts
tests/wecom-read.spec.ts	WeCom worker reads and Trigger mapping: the reference and resource grammar, the message with its image marker, a group reply by `chatid` and a direct one by `touser`, and the `no_root`/`no_parent` ThreadContext parts
tests/hosting-stream.spec.ts	`pollHosting` on a real Store and a stub adapter: the cursor, the changed change requests and CI, the overflow and a failed change
tests/notice-channel.spec.ts	The refusal notice reaches a conversation and nothing else: a work item's channel refuses it at the registry, a chat's accepts it, a channel that never opted in refuses it, and `acceptsNotice` answers before a row is queued
tests/work-self-write.spec.ts	The ledger and the adapter around it: one revision per key, bounded, and a poll that drops its own write while keeping a person's later edit
tests/work-stream.spec.ts	`pollWorkItems` on a real Store and a stub adapter: the initialized cursor, the three categories, idempotency, aborts and a failed category, and the comment rounds - a person's comment handed over in its item's admission row, the agent's and a bot's skipped, and a thread whose ticket is not open read not at all
tests/work-images.spec.ts	Work-item images: `` parsing (HTML and jsonML), sha256 de-duplication, type/size limits, failure degradation, and reading an archived image back as an attachment
tests/panel-instances.spec.ts	The panel's data layer for several instances of one kind: the rows and health of two chat instances, deleting one, and a legacy Source without `kind`
tests/live/weixin-bridge.live.spec.ts	The Weixin bridge against a real local sidecar (only with `LIVE=1`): health, status, rooms, members, messages, idempotent sends, and the token and permission failures

Test	Covers
`tests/live/gitlab-contract.live.spec.ts`	The GitLab.com contract read-only (only with `LIVE=1`): merge request and note mapping, and an unauthorized token's error that never names the token
`tests/live/projex-contract.live.spec.ts`	The Projex contract read-only against one real organisation (only with `LIVE=1`): every polled category lists through `workitems:search`, an item reads by both its id and its serial, and a comment comes back as text rather than the JSON envelope the API stores
`tests/live/thirdparty-errors.live.spec.ts`	WeCom, DingTalk and Yunxiao authentication failures (only with `LIVE=1`): the providers' error codes and that no secret appears in the message
`tests/weixin-media.spec.ts`	The Weixin media path over injected `fetch` and fake attachments: the magic-byte sniffing, the size and deployment refusals, the attachment commit, and the ThreadContext's message and image reads

`tests/fixtures/` holds what several tests share: `host.ts`, a simulated DeepSeek Harness host; `panel-server.ts` and `panel.tsx`, the fake management API and the panel around a page; `deployment.ts`, a resolved deployment; `git.ts`, the environment of every git the tests start, which names no repository; and `async.ts`.

Persistence

One SQLite file, `/orchestrator.sqlite`, which only the Store opens.

Table	Holds	Written by	
`trigger`	Each Trigger, its time `at` in the one form `Date#toISOString` gives, set when the Engine receives it, so times compare as text	Engine	

Table	Holds	Written by	
`disposition`	Where each Trigger went: `filtered` with the Source's reason code, its Eval and Decision, the Tickets it reached, by fan-out or attachment, whether it was requeued, the Tickets that handed it back with their reasons, and those whose Executor acknowledged or answered it	Engine	
`trigger_thread`	Which Threads each Trigger is on, with its time and whether a person wrote it, so a Thread's Triggers are read without scanning every Trigger, in order of time and then id: a page of a Thread's timeline continues from a `` ` ``	`` ` `` cursor, and Triggers that share a time are not skipped	Store, with each Trigger
`thread`	Threads: Subscriptions, the Tickets that follow it for context, queued Triggers, the Eval in progress, the count of its expiries and `meta`; the generated columns `active`, set while Triggers are queued or an Eval runs, and `subscribed`, its Source while a Ticket subscribes to it	Engine; Sources write `meta`	

Table	Holds	Written by	
`ticket`	Tickets with their Mandate, origin, execution, Assignments, Channels, Inputs and the Threads they follow; the generated columns `open`, set while it has not ended, and `active`, set while Inputs are undelivered or the latest Assignment is pending or running	Engine	
`decision`	Decisions, by Thread	Engine	
`effect`	The effect queue: per kind and key, the payload, its order, failed attempts with the last error, and when it is due	Effects; the schema 11 migration queues what earlier versions kept for the main Session	
`outbox`	The receipt of each message for a Channel, by id, with the Ticket it is for and its `status`, `pending`, `sent` or `dropped`, kept so an id is never queued twice; for a dropped message, when and why, as the generated column `dropped`, its time, and a reason code. A message that waits has a `reply` effect, which holds its attempts	The Outbox	
`kv`	Every other record, by key	The part that names it	

A generated column is computed from its row's JSON and is empty outside its set, and each has an index over its set, so a pass reads only active work. A database that predates a column gets it when the Store opens it.

`KV` (`src/kv.ts`) defines each key-value record's owner, key and type. All readers and writers use these entries; only migration code names older keys.

Records	Keys or contents
Schema	Current schema version

Records	Keys or contents
Settings versions	`settings-revision`, `settings-version:`, `settings-block-revision:`; snapshots include origin, Policy version and affected Tickets
Prompt versions	`prompt-revision`, `prompt-version:`
Main Session	`master-id`
Session prompt history	`session-prompts:`
Source state	Read positions such as `github-cursor:`, PR CI fingerprints and seen Feishu-topic messages; keys begin with the kind or `thread-` plus a Thread key
Display names	`source-labels:`

Dismissal is stored on its Decision. Input text is rendered from the Ticket when read, so neither needs a separate key-value record.

Read caches use bounded in-memory `TextCache` instances and are lost on restart. A Session paging through an original Source record or prompt preview must reread offset 0 after a restart. Feishu links resolved after intake are cached the same way; link-based search finds them only after that link has been read during the current process lifetime.

Nothing deletes old records yet, so these grow without bound: the `trigger`, `disposition` and `decision` rows; ended Tickets and their Threads; the outbox's receipts; and in `kv`, every saved settings revision and prompt version, the prompt revisions each Session worked under, and the Sources' records per Thread and per Issue or pull request.

`SCHEMA_VERSION` only increases. A database of a newer version refuses to start; `src/app/migration.ts` upgrades a database of the pre-release schema 10 once, at start, and refuses an older one.

Model-facing interfaces

Main Session tools

The main Session has 14 application tools:

Purpose	Tools
Read and route	`orchestrator_read`, `orchestrator_route`
Manage Tickets	`orchestrator_ticket_retry`, `orchestrator_ticket_request_status`, `orchestrator_ticket_note`
Re-evaluate	`orchestrator_trigger_requeue`
Assess proactive work	`orchestrator_eval`, `orchestrator_eval_read`, `orchestrator_eval_decide`
Communicate and unsubscribe	`orchestrator_reply`, `orchestrator_thread_unsubscribe`

Purpose	Tools
Configure	orchestrator_settings_save, orchestrator_policy_sync, orchestrator_prompts_save

orchestrator_route accepts 1–20 decisions in order. Each accept, attach, stop or ignore runs as a separate Engine command. It also permits ignoring an inferred candidate directly from its card. orchestrator_ticket_note sends a fact as context, not authority.

Settings saves select policy, eval, executor, pause, or source; the last also names its Source id. A prompt save with preview: true composes text without saving.

The main Session also has DeepSeek Harness's skill, bash, read, read_image, job_output, job_list and job_kill. Avoiding code reads is a prompt rule, not a tool restriction.

Presets and wait limits

New main Sessions use dsh-superagent-router (ROUTER_PRESET). Shell commands and job waits return after 5 seconds. If the Web UI selects another preset before the first turn, Sessions restores this preset, or WORKER_PRESET for a worker.

The noWait guard in src/app/sessions.ts refuses main-Session shell commands that invoke sleep at a command position, shell timeoutMs above 30 seconds, and job_output waits above 5 seconds. Reports and job results arrive automatically, so waiting only delays incoming candidates.

Existing main Sessions on standard also need explicit timeouts for foreground commands and job waits; otherwise that preset's defaults hold the step for 60 and 30 seconds respectively. The host determines the current preset through presetForSession, falls back to the Session projection or header, and assumes standard if it cannot read the value.

Executor tools

Each Executor has six application tools:

Tool	Purpose
orchestrator_read	Read resources available to its own Ticket
orchestrator_ack	Acknowledge a request
orchestrator_report	Report progress or an outcome, binding a pull request under the pull-request rules
orchestrator_thread_unrelated	Drop an unrelated attached Thread, hand back its owned follow-ups and tell the main Session why
orchestrator_thread_follow	Subscribe to a related Thread for context and tell the main Session
orchestrator_trigger_handback	Hand back a message once, with a reason carried by its candidate

A hand-back returns `queued: false` if another Ticket still owns the message, or this Ticket never owned it. For example, a bot message went to Eval independently.

The worker preset is defined in `cordis.patch.yml` and selected through `WORKER_PRESET`. `WORK_TOOLS` in `src/app/sessions.ts` retains available file, search, shell, job, web, skill, todo and subagent tools. Workers do not receive tools for asking or presenting to a person, changing the profile, scheduling work outside worker slots, starting or messaging other Sessions, or editing external documents. Tool calls are refused after the Ticket ends.

Result shapes and assessment context

Routing results use `{decision?, reason, ticket?}`. `orchestrator_route` returns `{results}`, preserving item order and adding each item's `trigger_id` and `kind`. Invalid or refused items return `refused` and a reason without stopping the batch.

Settings and prompt saves return the saved configuration. Policy sync returns the Source, allowed-list size, and `added` and `removed` names.

`orchestrator_eval` returns context rather than instructions. Assessment instructions live in `runtime/admission` (`prompts/admission.md`), and rules in `runtime/policy`. Its context includes:

- The Trigger text, up to 4,000 characters, with image ids and other Triggers in the batch.
- Up to eight `related_tickets`, open ones first, and `tickets_total`.
- The Eval's `policy_version`.
- A Thread window: the first five messages and five on either side of the Trigger. Messages use `{id, from, at, text}`, normally limited to 400 characters, or 1,200 for the two on either side. Truncated messages include `chars`; image references use `images`.
- Other loaded messages as `{id, from, preview}` stubs of 50 characters. `orchestrator_eval_read` expands them by id.

Ticket cards use the closed `TICKET_STATES` set from `src/app/cards.ts`: an open stage such as `person/review` or `working/queued`, or an ended outcome. Acknowledgements are limited to 100 words whether submitted through accept, assessment or `orchestrator_ack`.

Read resources

`RESOURCES` in `src/app/resources.ts` defines the resources and generates the tool's `resource` description.

Resources	Content
`tickets`, `triggers`, `threads`, `decisions`	Paged lists as `{items, next_offset}`; activity lists put newest first

Resources	Content
`decision/`	One Decision
`thread/`	Label, Tickets, recent Decisions and a page of Triggers
`trigger/`	Message text, replied-to Trigger and neighboring messages, excluding CI runs
`ticket/`	Related ids, source-batch Triggers, Threads, and first pages of Assignments and Inputs
Ticket subresources	`/assignments`, `/inputs`, `/assignment`, `/result`, `/report/`, `/transcript`, `/message/`, `/input/`
`settings`, `settings/`	Revisions and summaries, or a complete block ready to save
`prompts`, `prompts/`, `prompts/current`, `prompts/defaults`, `prompt-preview/`	Prompt configurations and composed text
`models`, `models/`	Available models
`feishu//…`, `github/…`	Original Source records, described in the Source skills

Ticket Threads are marked `context` for context-only follows and `bound` for Source bindings. Assignment pages are newest first and include report previews. Continue lists with `next_offset`. `/input/` accepts an Input's name or id. `settings` includes small blocks in full and summarizes Policy and Source blocks.

Source resources include message neighborhoods, `github/pulls//thread/` and `github/commits//checks`.

An original's messages read in one compact shape, `{id, from, at, text, reply_to?, path?, line?, url?}`, not as GitHub or Feishu records them.

An Executor Session reads only its own Ticket, its Threads, the Triggers that reached it or that its request held, the prompt versions it worked under and the originals, and its pages leave out the ids of what it cannot read, Decisions and other Tickets.

Every id a page shows reads through one of these, so a Session walks from one object to the next; hashes nothing reads by, such as an Eval's id or a Policy version, stay off the pages.

A text page has 4,000 UTF-16 code units by default and 8,000 at most, a list 20 items by default and 50 at most.

The names `event/`, `task/` and `observation/` of earlier versions still resolve.

Messages delivered to Sessions

These messages are steered into the main Session's turn:

Message	Content
Candidate	Batch Trigger cards, Thread, `related_tickets`, `tickets_total`, `delivered_to`, referenced Tickets from Issue/PR links, `handed_back` reasons and the next action
`follow_up`	One message as its Executor sees it, `replies_to`, receiving Tickets, and the reminder that Executors decide it while the main Session may only stop work
`worker_report`	Ticket card, originating Trigger, state and summary up to 600 characters; `chars` and a read pointer when truncated
`worker_attention`	Ticket card carrying the question

Other notices are injected without waking the main Session: `ticket_ended`, `settings_saved`, prompt saves, Trigger requeues, dropped Subscriptions and context follows. A settings notice names the block, changed paths and `read: settings/`. The main Session receives no redundant notice for a save, requeue, unsubscribe or attach it performed itself.

Workers receive assignments and Inputs inline. The first message includes the title, up to 1,500 characters of assignment text and the source Trigger card. Inputs carry a Trigger card with up to 600 characters and `replies_to`, or plain Input text. Read pointers appear only for truncated content.

Panel-only actions

There is no tool for dismissing an unavailable Decision, rerunning setup, refreshing Source display names, or sending an Executor-owned human follow-up to Eval. The last action is deliberately refused by `orchestrator_trigger_requeue`. Other panel actions have equivalent tools.

Most built-in instructions live in `prompts/*.md`, with one sentence per line joined by `promptFile`. Source skills live in `skills/dsh-superagent-*/SKILL.md`. Tool descriptions and fixed notices such as restart recovery notes remain in code. Assessment instructions live only in `prompts/admission.md`.

`tests/model-surface.spec.ts` checks these surfaces for drift. 004 Prompts explains the team's editable parts.

Configuration reload

Every plugin configuration key except `stateDir` is volatile. The panel's Deployment settings and DeepSeek Harness's Plugin Manager edit them through DeepSeek Harness's configuration

form; the host writes `cordis.patch.yml` and updates the values in place, and dsh-superagent reads each limit when it uses it, so a change applies without a restart.

A settings save that changes a Source field setup reads, such as an account's CLI, and any save while setup waits, restarts dsh-superagent. DeepSeek Harness's settings service does not serve the entry during that restart, so after its own restart dsh-superagent asks it to read the entry again, and an open form becomes editable again.

Reviewing a change against real data

Point `DSH_HOME` at a temporary directory and create a new profile there. Copy the database from a backup, disable every Source, pause processing and remove the main Session id. Give the GitHub and Feishu CLIs read-only stand-ins.

Open questions

- Trust: which Trigger fields are trusted, such as the Source's actor and Thread membership, and which are not, such as the body and quotes.
- Deployment isolation: the agent's profile on its own pinned DeepSeek Harness installation, so upgrading DeepSeek Harness elsewhere does not break it.

Common practices and troubleshooting

Let only people on the allowed list start work

33. Panel → Assessment → select the Source on each Source (DingTalk / Feishu / GitHub / GitLab / Yunxiao Projex / WeCom / WeChat).
34. Add people under "users who may start work". For WeChat and WeCom you can click "立即同步 / 刷新名单" (Sync now / Refresh the list) to pull them in from chat members or the contact directory.
35. The admission table is defined row by row: on the list / not on the list × mentioned in the chat / not mentioned / direct message. A person who is not on the list does not become a Ticket even when they mention the bot.

In a chat, respond only when mentioned

- WeChat / DingTalk / WeCom: set that chat's or Source's "receive" to "mentions only" (set the "not mentioned in the chat" row of the admission table to "ignore").
- Feishu: set the chat's "receive" to "mentions only"; work items and GitHub have no such concept.

Turn off status write-back to the board

Panel → Sources → Yunxiao Projex → turn off "write the work item status back". It is off by default: the agent only delivers in comments and does not change the board. Once it is on, you must choose two statuses (the dropdowns show the names on the board, while the status IDs are what is actually written).

Let the Executor read the original context

Source	Resources
Feishu	`feishu//message`, `/root`, `/parent`, `/thread`, `/around`
GitHub	`github//…`
WeChat	`weixin//message`, `/thread`, `/around` (images as `[image:]`, downloadable with `GET /media`)
WeCom	`wecom//message`
DingTalk	`dingtalk//message`
Yunxiao Projex	`projex//item`, `/comments` (paginated)

Resources are read-only: reading creates no Trigger, sends no reply and changes no remote state.

Troubleshooting

Symptom	What to look at first
The panel shows "dsh-superagent 没有响应" (dsh-superagent is not responding)	Most likely the plugin is not mounted: check the DSH log for `dsh-superagent ready`; a common cause is that a preset name of an old Session in the state directory no longer exists (fixed in 2.3.0, presets use shared ids), or that `stateDir` is configured wrongly.
Yunxiao reports 操作失败，当前工作项已进回收站 (Operation failed, the current work item is in the recycle bin)	A person deleted the work item. The plugin treats it as "gone": the Thread is marked `gone`, is no longer read and no longer reports errors; after restoring the work item, delete the old Thread so it appears again.
The Yunxiao authorization page shows `invalid client_id or redirect_uri`	Before 2.4.0, Windows `cmd start` truncated the URL at `&`; this now uses `rundll32 url.dll,FileProtocolHandler`. After upgrading, restart DSH and then authorize.
WeCom receives no messages	The app's "receive messages" URL must be `https:///wecom/` and reverse-proxied to this service; 回调 `Token` (callback Token) and `EncodingAESKey` must match the admin console, and everything is rejected (403) when the Token is missing.
DingTalk does not listen	This bundle does not include the Stream SDK: a deployment must inject `connect()` (the `dingtalk-stream` client); `facts` and the health check say "not injected".
The WeChat bridge drops offline	The panel's "check" reads `/health`; with no events for a long time it re-attaches by itself; after switching accounts or logging in again the bridge re-attaches, with no need to restart the plugin.
A status name is reported as "never seen"	First click "read the Yunxiao status vocabulary" on the Source card (or refresh next to the field), then select the name from the dropdown.
After an upgrade both sets of plugins are enabled	Only one registration per Source kind is allowed: stop the old `dsh-superagent-channels` first, then enable this plugin; leaving `stateDir` empty keeps the original directory.

Add a Source yourself

36. Implement `ChatAdapter` (polling/stream/callback) or `WorkAdapter`, and define `SourceKind` (scope / policy / channel / threadContext / find / refreshLabels).
37. Add one line to `cordis.patch.yml`: `- id: ... name: dsh-superagent-multi-channel/-watch`.
38. Add resources for the worker: `RESOURCES` and the routes in `src/app/resources.ts`, and document them in `skills/dsh-superagent-/SKILL.md`.

39. Add unit tests: reading, reference syntax, the three context parts, and the troubleshooting fields; run `pnpm check`.

Extensions: the multi-channel layer

English · 简体中文

This repository is a downstream distribution of a research prototype built on DeepSeek Harness (see REPOSITORY.md). The prototype listens to one GitHub repository and, optionally, Feishu. This edition keeps every prototype behaviour and adds the channels a team in mainland China actually runs on, plus the ability to run several instances of one channel.

Upstream's numbered documents still describe the agent itself. This file is the map of what was added, and the reference for anyone extending it further.

1. What is added, and where it lives

Gap	What this edition adds	Where
only GitHub	**GitLab** (self-hosted or gitlab.com): merge requests, notes, pipelines	`src/sources/hosting/gitlab/*`, `src/infra/hosting-api.ts`
no work items	**Yunxiao Projex**: requirement / bug / task, comments, status	`src/sources/work/projex/*`, `src/infra/yunxiao-api.ts`
only Feishu	**WeChat** personal account: group `@`, message images, voice transcripts	`src/sources/chat/weixin/*`, sidecar
—	**DingTalk** Stream, **WeCom** encrypted callback	`src/sources/chat/{dingtalk,wecom}/*`
one instance per channel	**many instances per channel**, config / health / cursors scoped per instance	`src/sources/instance.ts`, `src/sources/registry.ts`

Every Source plugin loads (`weixin`, `gitlab`, `projex`, `dingtalk`, `wecom`, `feishu`, `github`) and is built as a separate entry (`lib/*-watch.js`), so the panel lists them all and its switch - not the profile - decides whether one runs. A Source with no configuration reports `sourceOff` and idles, so enabling one is a switch plus change plus a saved instance, not a rebuild of the bundle.

2. The one assumption that had to change

The prototype has a single extension point, `SourceKind`, but it treats *a kind* as *an instance*: `registry.resolve()` hard-codes `id: kind.kind`, which makes "two GitLab projects" or "two WeChat accounts" impossible. This edition separates the **settings key** (the instance id) from the **behaviour id** (the channel id) and keeps one compatibility rule:

> *the default instance id stays equal to the channel id, so stored triggers, threads, mandates and policies keep resolving without a migration.*

Everything else follows from three narrow ports rather than three ad-hoc integrations:

```
src/sources/hosting/   HostAdapter  →  change requests, notes, CI
src/sources/work/      WorkAdapter  →  work items, comments, status
src/sources/chat/      ChatAdapter  →  messages, threads, media
                       InboundTransport (poll | stream | webhook), DeliveredRing
```

Adding a provider means writing an adapter of a few hundred lines. The trigger shape, the change-request lifecycle, the CI summary, comment idempotency, cards and prompts are reused unchanged; identity and scope are the only provider-specific parts. Section 6 is the recipe.

3. Feature points, channel by channel

3.1 GitLab — `hosting/gitlab`

Aspect	Detail
What it listens to	merge requests of the configured project, their notes, and pipeline status
Transport	polling (`pollSeconds`, default 30)
Identity	`hostUrl` + `project` (`group/name`); self-hosted instances are first class
Credentials	a personal access token (`api` scope) in a **file**, read per call, owner-only (`0600`, or an ACL on Windows)
Path safety	every request path is confined to the configured project; a path that escapes it is refused before any request
Error mapping	a non-JSON body becomes a status error, never a token echo; 401 maps to `unauthorized`
Outbound	comments and review-thread replies, **marker-idempotent** (the reply carries a marker, so a retry that already landed is detected)
Known gap	write-back against a real instance is unverified here (no PAT)

3.2 Yunxiao Projex — `work/projex`

Aspect	Detail
What it listens to	work items in the configured project space, by category, plus their comments
Categories	requirement / bug / task are polled by default; other categories are dropped rather than guessed
Transport	polling (`pollSeconds`)
Identity	`organizationId` + `project`; `assignedTo` is an Aliyun **accountID**, so the configuration must hold the bot's own accountID

Aspect	Detail
Credentials	a `pt-…` personal token in a file, owner-only, reread per call; or **Sign in with Yunxiao** on the source card, which runs OAuth 2.0 (authorization code + PKCE, the browser signs in, a loopback listener takes the callback) and writes the token to the same file - 24 hours, renewed automatically, refresh token 90 days
Path safety	confined to `/oapi/v1/`, with an organization allowlist; dot-segment escapes are refused before any request, plain or percent-encoded
Outbound	comments, marker-idempotent
Status write-back	Both status fields are **dropdowns**: press **Read Yunxiao status list** on the source card (or the refresh beside a field), which lists the work item types per category (`ListWorkitemTypes`) and each type's workflow (`GetWorkitemWorkflow`, central form), then stores the statuses as `id -> name` for the pickers — the board name is shown, the status id is stored; a name or an id is accepted, and statuses seen in a poll are the fallback. A name neither place has seen is refused with the action that fills the vocabulary, never sent to Yunxiao as a status
Comment attachments	With "Let the model attach files to comments" on, a file in the worker's report (inside the ticket directory, ≤10 MiB, not an executable) is uploaded to the item's attachments, and the comment is posted as **RICHTEXT**: escaped paragraphs, the host's `embedHtml` for an image (a native ``) plus a caption link to the file, a link for any other file - always the **permanent `embedUrl`**, never the expiring `url` - and the marker as an HTML comment. A comment with no attachment stays plain text, so Markdown the agent writes is unchanged. Off by default
Member list	With "Sync the list from project members" on, "Sync project members" replaces the list with the project's members (`userId`, the id an item is assigned to), keeping your order, refreshing names, and appending new members; one sync runs at startup. Like Feishu's, the sync applies to the **default instance** only (instance id `projex`)
Panel search	A serial (`ESFX-6228`) or a work-item page address finds that item's triggers and tickets (the page is addressed by serial while a thread is keyed by the internal id, so linking to a thread still goes through the conversation id)

Aspect	Detail
Deleted items	A work item in the recycle bin answers a comment/status call with HTTP 400 "已进回收站"; the plugin treats that as **gone** - it marks the Thread `gone`, stops reading it, and never counts it as a poll error (deleting an item is a person's act, not a plugin failure)
Site	Empty uses the central site `https://openapi-rdc.aliyuncs.com`; a Region deployment names its instance domain under "API site", and sign-in and calls use the same site
Known gap	the write paths (status PUT, attachment upload and the RICHTEXT body) are unverified against a real instance here (no token); `GetWorkitemWorkflow` needs a work item type the project actually has

Two ready ways to check against a real organisation: **Sign in with Yunxiao** on the source card, or `python verify/oauth-check.py --token-file` in a terminal (the same OAuth flow; it prints the authorization URL and writes the same token file, never a token value). With a token, `LIVE=1 YUNXIAO_ORG=… YUNXIAO_PROJECT=… YUNXIAO_TOKEN_FILE=… npx vitest run tests/live/projex-contract.live.spec.ts` checks work items, comments, the member roster and the workflow status list, read-only.

3.2.1 The three-stage loop: assess / execute / feed back

This channel used to do "receive and reply" and nothing else, and each of the three stages was missing one piece. All three now have a defined landing place.

Stage	What was missing	What it does now	Where
Assess	A Trigger carried only its own text. An assessment of a comment saw no title, no description and no earlier answer - the model faced "any news on this?" and had to guess the whole thing, so it either took on the wrong work or ignored everything; and a comment Trigger carried no category, so the admission table judged it on its defaults and an operator's decision about a category did not reach that category's comments	**The item is read as one conversation**: the description is the root message (`[ESFX-6228] title` plus body), comments follow in time order and continue across the host's pages, the agent's own marked comments are `bot` (never a person's request), and embedded images are read back from the local archive as attachments. A comment Trigger also carries its item's **category**, so a comment and its item fall in the same admission row	`src/sources/work/context.ts`, `src/sources/work/stream.ts`
Execute	A worker on such a ticket did not know which item it was on: it could not name the item in a commit message, a PR or a reply	**Every ticket carries the item's identity**: `DSH_SUPERAGENT_PROJEX_PROJECT` / `_WORKITEM` / `_SERIAL` / `_CATEGORY` / `_STATUS` / `_URL` (plus `_ORG`), in the execution-environment prompt where the GitHub kind puts `DSH_SUPERAGENT_GITHUB_REPOSITORY` / `_NUMBER`	`src/sources/work/environment.ts`
Feed back	Outside the comments the board was dead: an item sat in its original column from acceptance to delivery, so a person had to open the comments to learn whether the agent was on it	**The status follows the ticket**: with `writeBack` on and the two status ids named, the start status is written as soon as a ticket starts and the done status once it ends (completed, merged, dropped). Each phase is written once per ticket (KV ledger), the item is read first so a status already current is not rewritten, and a failure is recorded in the poll result and retried next round	`src/sources/work/feedback.ts`

Why each of them matters:

- **An assessment needs a conversation, not a sentence.** Yunxiao has no `@`, so the admission table decides from

whether the assignee (or the creator) is on the list - which makes the assessment the only place that can read what the person actually wants. Handing it half a sentence makes an accurate table useless.

- **Execution has to be able to name things.** The serial (`ESFX-6228`) is what people search, review and quote; without

it a deliverable cannot be tied back to the board item.

- **Feedback belongs on the board.** A comment is written to one person, a status to a team: a stand-up reads columns,

not comments. Write-back is therefore **opt-in** (`writeBack` is false by default), does nothing until the statuses are named, and **never touches a ticket a person stopped** - whoever stopped it knows.

3.2.2 Against the GitHub and Feishu channels

Capability	GitHub	Feishu	Yunxiao Projex
Assessment context	`threadContext`: the whole Issue / PR discussion	`threadContext`: messages and images in the topic	**added here**: description, every comment, embedded images
Who may originate work	accounts on the list	members on the list	the assignee's (else the creator's) accountID
Execution environment	repository, account, commit email, number, PR head	- (a chat channel relies on the source context in its prompt)	**added here**: project, work item, serial, category, status, URL, organisation
Feedback - acknowledgement	a comment	a message	a comment
Feedback - result	a comment (`announceEnd` speaks on its own when the ticket ends)	a message	the same: "completed / merged / dropped" is posted when the ticket ends
Feedback - status	expressed by the PR merging or closing	-	**added here**: board status write-back, a name taken from the project's workflow

Capability	GitHub	Feishu	Yunxiao Projex
Feedback - attachments	files in the PR	message attachments	**added here**: a file is uploaded to the item's attachments and embedded in a RICHTEXT comment (switch off by default)
Panel lists	accounts that may originate work + bots whose comments may become candidates	members (synced from the chat)	members' accountIDs, **syncable from project members** (switch off by default)
Panel search	`#12` / an Issue or PR address	- (a chat searches by message)	`ESFX-6228` / a work-item page address
Refusal notice	not sent (public record)	sent (a conversation)	not sent (public record)

What stays different on purpose: a work item has no `@`, so the "outsider who addresses the agent" cell of the admission table is always a dash; and the sync takes the project's members only, never the whole organization. Why a status resolves through the project's workflow, how an attachment reaches a comment, why the sync is limited to the default instance, and what is still unverified, are in ADR-009 and ADR-010.

3.3 WeChat — `chat/weixin`

The desktop client cannot be driven from Node, so it sits behind a **loopback HTTP sidecar** with a frozen contract. The Node side knows only the contract; the backend is replaceable. Sections 4 and 5 are the detail.

Aspect	Detail	
What it listens to	the chats configured in the panel: groups (judged on the structured `@`) and direct chats (a direct message needs no mention to be addressed to the bot)	
Transport	polling the sidecar (`pollSeconds`, minimum 5) which is itself **push-fed** by the client's listener	
Identity	`selfWxid` is the account whose `@`s count; chats and senders are the sidecar's ids	
Credentials	the sidecar's bearer token in a file, owner-only	
Group `@`	decided on `ats`, never on the message body — see §5.2	

Aspect	Detail	
Outbound (files)	`/send` takes one optional `file` (base64, ≤ 20 MiB, name reduced to a basename); verified on the live client, where it arrives as an image message. The core carries them (`OutboundMessage.files`). **The worker names what it produced in its ready report** (`files`, relative to the ticket directory) and the main Session's result reply **attaches them by default** (or `orchestrator_reply` takes an explicit `files`): **only from the ticket's own directory**, ≤ 20 MiB, executables refused, and gated by the panel's `sendFiles` switch (off by default) - see ADR-008	
Outbound	text, **keyed** idempotency (the OutboundMessage id is sent as `Idempotency-Key`), ≥1 s serialized spacing. A group reply may mention the sender under a switch (`mentionBack`), resolved from the sender's id to the display name the sidecar observed; **no native quote** - the client library's `SendMsg` takes no quote argument, see ADR-007	
Inbound media	images, downloaded through the sidecar and judged by magic bytes; groups and direct chats share one `/media` path. A switch turns it off, leaving a `[图片]` placeholder and downloading nothing	
Inbound voice	transcribed into text (`to_text`, up to eight seconds of the client's UI per message, degrading to a placeholder). A switch turns it off, leaving `[语音消息]`	

Aspect	Detail	
Our own writes are not new requests	A work-item Trigger is derived from `gmtModified`, and the agent's own comment or status write bumps it - so it triggers itself (measured: one item, two Triggers, the second caused by a status write six minutes later, which started a duplicate Assignment). The adapter therefore reads the item back after its own write, records the resulting revision in an in-process ledger, and a poll skips that revision; a person's later edit is a new revision and still triggers. The ledger is keyed by host, organisation, item and revision, capped at 512 entries with a 24-hour expiry	
The request lives in the description	Projex's **list** endpoint returns `description: ''` with `formatType: null`; only the **detail** endpoint has it, as RICHTEXT JSON (`{"htmlValue":"…"}`). A polled item with no body is therefore read back by id and its HTML stripped to text - without that the agent sees the title alone and never the instruction (this is how ESFX-6228 lost "tell me the URL")	

Aspect	Detail	
The end is announced	The acceptance is sent by the Engine; the **result used to depend on the main Session calling** `orchestrator_reply`, so a model that answered in its own chat left the Source with an acceptance comment and silence. A ticket that ends now posts "已完成/已合并/已放弃：" to the conversation it started in, and stays quiet when the main Session already sent the result (the ticket remembers), when a machine asked for it, and when a person stopped it. A channel with a length limit gets the summary first and a short line second, then silence plus a `result.announceFailed` event	
The settings page tolerates a bad row	A Source's admission rows are derived from its own configuration, and a mistake there must not cost the operator the whole page - the panel cannot open, so nothing can be saved (incident of 09 Nov: the Projex rows validated the whole instance config with a strict schema, `organizationId` was an unknown key, and saving reported `Invalid settings:`). The view now falls back to no table when the rows cannot be built, and everything else saves as usual	
Rows must match scope	DingTalk and WeCom have no chat list of their own, so their admission rows are the three scope reports: a mention, an unmentioned group message, and a direct message. When the two disagreed nobody read what a person set, which is why both tables showed only dashes before. atOnly decides whether the unmentioned row is configurable	

Aspect	Detail	
Reception versus authorization	A Source decides only what it can see: which groups, whether direct messages, and how much each group hands over (only messages that mention the agent, or all of them). The Policy decides what counts as work: who may originate it, in which range, at what strength. So the group list, the direct-message switch and each group's reception live in the Source; the allowed-people list and the admission table live under Eval. Reception is per group, like Feishu's chat list; the old global "receive only @s" is now the default for a new group. The card carries no "may originate work" switch - that was a third place to grant work and is gone	
The WeChat list shows no source	The list is user id plus name, the same as Feishu. Whether a message came from a group or a direct chat is a property of the message, and the admission table splits on it; there is no group/direct column in the list. The sync keeps that bookkeeping internally (rows it wrote are replaced by the next sync, hand-added rows stay, merged by id with the manual row winning)	

Aspect	Detail	
Who starts the WeChat bridge	All in the panel: point Advanced -> Bridge launcher at the machine's start script, and Start launches it detached and windowless through the plugin, so the bridge outlives a DSH restart; Stop finds the process listening on the bridge's port, verifies its command line is the sidecar, then ends it; Restart is stop-then-start; Check reads /health and says what the backend can do or why it cannot. The launcher's contents stay on the machine - the client library and shim never enter the repository. Without a launcher it falls back to the scheduled task	
Who starts the WeChat bridge (old)	The sidecar is a separate long-lived process: it must run in the signed-in interactive desktop session, outlive a DSH restart, and keep its local shim out of the repository, so the plugin never spawns it. The card's Start / Stop / Restart buttons ask the machine's scheduled task (default `DSH-WeChat-Bridge`, editable under Advanced) to run or end it with `schtasks /run /tn "the task name" /end`; the task name is validated in the schema and again in the runner, so a button never becomes a free-form command. Check `bridge reads /health` once and states the backend's capabilities or why it is unavailable	
What WeChat Sync now does	It reads only the member roster of the group you pick (through the wxauto-bridge member endpoint), writes those members into the users-who-may-originate-work list marked as coming from a group, and keeps people already marked as direct. It reads no message content, sends nothing, does not change the groups you receive, and does not touch the admission table below. The point is to skip typing IDs by hand; the list takes whatever identifier the bridge reports - a stable wxid when there is one, the display name when there is not	

Aspect	Detail	
WeChat list and sync	The users-who-may-originate-work list carries a Source column (group or direct) and gains Sync from group with Sync now: the group members join the list marked as coming from a group, and people you marked as direct are kept. Group members and direct users are therefore distinguishable: group people are written by the sync, direct-only people are marked by hand; the admission table also keeps every group and the direct row on separate lines	
One table shape everywhere	All five Sources read the same way: **rows are reception ranges** (one per configured chat, plus other chats and direct messages; requirements/bugs/tasks for Yunxiao; new Issues/merge requests/comments/CI for GitLab), each with a 仅 @ / 全部消息 / 接收 / 不接收 annotation, and **only the cells that range really has** - the rest render as a dash. A chat now names one row per conversation (its scope reports the conversation id; it used to share one `chats` row). GitLab had **no Policy at all** - `hostingKind` declared none - so it could be neither configured nor authorized; it now has the account list and the bots-whose-comments-count list	

Aspect	Detail	
The Projex table	The same interaction as Feishu and GitHub: **rows are the reception ranges** (requirements, bugs, tasks - each labelled 接收 or 不接收 by the instance's categories), and **only the cells that apply are offered**. No field of a work item names the agent, so the "an outsider addresses the agent" column is a dash here; the two unaddressed columns are configurable. Whether the person responsible (the assignee, else the creator) is listed decides the column, and each column is handled exactly as configured - ignore, or an inferred candidate accepted in auto mode. An assignment is no longer a request by itself	
What counts as work	A work item carries no mention: **being assigned to somebody** is the explicit request. Three gates: eval mode is not off; the assignee is on "Eval → Policy → per Source → members who may originate work" (the account id from the item's assignee field). An item assigned to a listed member is accepted outright, with no auto mode; an unassigned item is ambient and only reaches the agent as an inferred candidate, which needs auto mode; and a rule must match, or the decision is `noRuleMatched`. With the list empty every item is refused as unlisted	

Aspect	Detail	
The Projex API	Polling uses `POST /oapi/v1/projex/organizations/{org}/workitems:search` with the project as `spaceId`, a bare array in reply, and `page`/`perPage` paging; the time window goes in `conditionGroups` (an array of arrays with `className: dateTime` and `format: input`). The old `GET /listWorkitems` is gone from the gateway and answers 404 for every category. A comment keeps its text in `content` as a `htmlValue` JSON string and names its author `user`; the status write-back is `PUT /workitems/{id}` carrying `{status: }`	
Ticket directory root	An **absolute** path: `/srv/work`, `D:\Work`, `D:/Work` and `\\server\share` all count, a relative one is refused, and empty means `\work`. Until 09 Nov only a path starting with `/` was accepted - in the schema and in the input's own `pattern` - so a Windows drive letter could not be entered at all	
The state lock	One dsh-superagent per state directory. A lock untouched for `LOCK_STALE_MS` (5 s; its holder refreshes every 2.5 s) is swept and taken; a held one is waited out for up to 6 s, and the failure names the remedy. This is the most common restart failure: the killed process leaves the lock, the new one is told it is held, and the whole plugin does not apply - which the panel reports as the plugin not responding	

Aspect	Detail	
The state lock	One dsh-superagent per state directory. A lock untouched for `LOCK_STALE_MS` (5 s; its holder refreshes every 2.5 s) is swept and taken; a held one is waited out for up to 6 s, and the failure names the remedy. This is the most common restart failure: the killed process leaves the lock, the new one is told it is held, and the whole plugin does not apply - which the panel reports as the plugin not responding	
Catch-up after a re-attach	A message that arrives while the listener is detached is lost for good - no callback, no queue, no record in the Engine. After every re-attach the sidecar reads the last twenty messages of each watched chat and queues the unseen ones (three chats, once a minute, reported as "重挂后补读 N 条")	
What gets watched	The sidecar's queue is filled by its listener, and the listener fires only for chats named in `/listen`, so the watch set is the configured groups **plus** the people the Policy lists. A person on the list is watched automatically, which is what makes their direct messages arrive - and what was missing when a message left no trace at all	
Direct messages	WeChat delivers direct messages as **one stream** (`/messages?room=` and `room=` return the same batch), so a person needs no entry among the groups: with "Receive direct messages" on, the round reads that stream and the Policy list decides who may originate work - **people are configured once**	

Aspect	Detail	
One list per question	**Groups** are configured under "Source → Groups to receive"; **people** are configured only under "Eval → Policy → per Source", like every other channel. A direct message needs no Source entry: it arrives anyway, and answering it needs none either. The policy card warns while that list is empty	
Panel configuration	`POST /listen` replaces the chat set and carries `options.voice` / `options.images`; adding a chat or flipping a switch needs no restart	
Keywords	a chat message naming one of `keywords` is treated as addressed to the bot **without an `@`** (case-insensitive); whether it becomes work is still the admission model's decision	
Canned replies	a match on `autoReplies` sends a fixed text from the polling loop, without the model; the message is **still handed to the model**, so a template never swallows the rest of what a person said	
Cursor	`/messages` carries `head`, so a client can notice that the sidecar restarted and its sequence went backwards, and start over; the sequence is seeded from the clock and stays monotonic across restarts	

Aspect	Detail	
A refusal goes to a conversation only	"这条我没有接：…" reaches the person who wrote to the agent: chat channels opt in with `channel.acceptsNotice`, and **work items (Yunxiao, GitHub issues) never take one** - a work item is a public record, where that comment is not an answer and also broadcasts the Source's configuration. One incident posted sixteen such comments on other people's requirements, tasks and bugs. Two gates now: the Engine asks `SourceScope.acceptsNotice` before queueing, and the registry refuses again at delivery (`gone`, receipt settled as dropped)	
Refusal notice	When a decision **ignores** a message that addressed the bot (@ or a direct message), one line goes back saying it was not taken and why (not on the list, no request found, a ticket already running). Ambient group messages get none, or a busy group would be answered on every line	
Receipt	when someone mentions the bot or messages it directly, a receipt goes out **before** the assessment (`acknowledge` / `ackText`, once per chat per 30 s). Measured here, the assessment takes 53 s to 2 min 09 s; the receipt does not shorten it, it removes the guessing	
Still-working note	after the receipt, if the bot has said nothing in that chat, one note every `progressAfterSeconds`, at most 3; it stops as soon as the bot speaks	
Config from the panel	`POST /listen` replaces the watched set; adding or removing a chat needs no restart	

One UI-automation client per machine. *A listener is only as alive as the client's window: when the window is*

stuck, or another tool of this kind drives it, `AddListenChat` *and* `StartListening` *still answer while the callback*

stops firing - which looks exactly like a quiet chat. So ten minutes without a real event is treated as a stale

attach: every watched chat is removed and re-added and `StartListening` *is called again, at most once per fifteen*

minutes. `/health`*'s* `detail` *reports* `recent event K s ago` *and* `re-attached M times`*; a rising M means something*

else is taking the window.

Switching accounts replaces the window. *Every thirty seconds the sidecar checks that the window it attached to*

still exists; when it does not, everything cached per account goes (chats, groups, members, the event queue, the

listen set) and it attaches again, which `detail` *reports as "检测到客户端窗口更换…已重建连接 N 次". A failed attach*

no longer exits the process: `/health` *reports* `available:false` *with the library's own reason (such as "no signed-in*

main window found") and retries, so completing the sign-in needs no bridge restart.

The client build matters. `wxautox4 41.2.3` *attaches to the 4.1.9 generation; a client updated to 4.1.15.56 keeps*

answering "no signed-in main window found" while its window sits visible with the account signed in. `/status`

carries `client` *for exactly that reason: check the build before blaming the configuration.*

3.3.1 Alignment with the Feishu channel (this round)

WeChat already had: structured `@` detection, groups and direct messages, inbound images, voice transcripts, outbound text and files, keyed idempotency, instant acknowledgement, still-working notes, keyword listening and canned replies, mentioning the sender in a reply, per-instance isolation, and a listener that re-attaches itself. After comparing it with Feishu item by item, these were added:

Capability	**Feishu**	**WeChat (now)**						
Worker reads of the conversation	`feishu// message	root	parent	thread	around`	**added** `weixin// message	thread	around, `with next_resource` paging
Source skill	`dsh-superagent-feishu`	**added** `dsh-superagent-weixin` (the resource grammar and image markers; WeChat has no CLI and no message deep link)						
Assessment context	thread/ root/ parent/ around	thread/ **parent**/ root; WeChat has no topic root, so `root` answers `no_root` instead of inventing one						
Panel search by message	`om_…` becomes a term	**added** a platform message ID becomes a term; `meta.nativeId` matches exactly						

Capability	Feishu	WeChat (now)						
Reply mapping	`repliesTo` from the parent	**added** `repliesTo`, so a thread shows which message an answer replies to						
Messages per page	`readPageSize`	**added** `readPageSize` (1–50, the bridge's ceiling), shared by assessment reads and resources						
Member roster sync	`syncChat` + Sync now	already present (`syncChat` + Sync now, from the members the bridge has seen)						
Panel facts	bot identity	already present (signed-in account and mention evidence strength)						

Capability	**Feishu**	**WeChat (now)**						
Inbound images	yes	already present; the assessment can attach `[image:]`, and a worker downloads one with a single `curl`/`Invoke-WebRequest` against `GET /media/` from its environment, then reads it						
History backfill	a bot added to a chat recovers recent history	**present**: after a re-attach or first listen the bridge reads back the last 20 messages (up to 3 chats a round), now marked `historical` so they reach the assessment without a receipt or a canned reply						

Capability	Feishu	WeChat (now)						
Message deep link	AppLink	**not possible**: the sidecar exposes no deep link, so `link` returns an empty address (ADR-007)						
Bot list	bot messages can become candidates	**not possible**: WeChat has no bot-account concept, the bridge always reports `bot:false`, and pretending otherwise would be a lie						

Two limits are the backend's, not omissions: there is **no message deep link** (the sidecar exposes none) and **no bot list** (WeChat has no bot-account concept; the bridge always reports `bot:false`). Both are recorded in ADR-011.

3.4 DingTalk — `chat/dingtalk`

Aspect	Detail
What it listens to	group messages addressed to the bot, and direct messages
Transport	**Stream** (long-lived connection), with an interruptible reconnect
Identity	`clientId` + `robotCode`
Credentials	`clientSecret` inline or in a file, owner-only

Aspect	Detail
Capabilities	`idempotency: none`, `maxChars: 5000`, `markdown: true`, `proactive: false`
Honest weakness	outbound is **at-least-once**: there is no server-side idempotency key and the session webhook cannot dedupe a retry, so a retry may duplicate. The kind declares this instead of pretending
Known gap	the Stream transport needs a `dingtalk-stream` SDK injected as `connect`; without it the kind registers but does not start

3.5 WeCom — `chat/wecom`

Aspect	Detail
What it listens to	the application's messages: group chats by `chatid`, and direct messages
Transport	**webhook** (signed, encrypted callback)
Identity	`corpId` + `agentId`
Credentials	`corpSecret` inline or in a file; the callback `token` and the 43-character `encodingAesKey` (or `encodingAesKeyFile`) verify and decrypt
Callback URL	`/wecom/`: the plugin mounts that public route; ops points the application's receive URL at it
Conversation context	the assessment reads the one message the callback carried (`thread`); `root/parent` answer `no_root/no_parent` because a callback has neither a topic nor a quote, and no history to page
Inbound images	an `image` callback's `MediaId` becomes `[image:]`, and the assessment downloads it through `GET /cgi-bin/media/get` as an attachment
Panel search	a decimal `MsgId` becomes a search term; `repliesTo` exists, but a callback carries no quoted message so it is normally empty
Worker read	`wecom//message` (Feishu's resource grammar, one message and no next page)
Crypto	signature check, AES-CBC decryption, replay rejection (a redelivered message yields no second event)
Capabilities	`idempotency: none`, `maxChars: 2048`, `markdown: false`, `proactive: true`
Honest weakness	outbound is at-least-once; the message API has no idempotency key
Known gap	no history backfill, no message deep link and no chat member roster; a worker reads only the one message the callback carried

3.6 Prompt skills — one set for six sources

Every Session (main and executor) registers six built-in Source skills, each viewable and overridable in **Prompts → team prompts**, exactly like Feishu and GitHub (the dimension-by-dimension comparison is in capability-matrix.md):

Skill	**File**	**What it covers**				
Feishu `dsh-superagent-feishu`	`skills/dsh-superagent-feishu/SKILL.md`	`feishu// message	root	parent	thread	around`, lark-cli usage, following attachments and documents
GitHub `dsh-superagent-github`	`skills/dsh-superagent-github/SKILL.md`	reading with `gh`, issue/PR context, pre-publish checks				
WeChat `dsh-superagent-weixin`	`skills/dsh-superagent-weixin/SKILL.md`	`weixin// message	thread	around, [image:] and a GET /media` download, the honest no-deep-link limit		
Yunxiao Projex `dsh-superagent-projex`	`skills/dsh-superagent-projex/SKILL.md`	the work item is the thread root and comments are replies; `DSH_SUPERAGENT_PROJEX_*` identity; deliver with `orchestrator_reply`, the plugin owns status write-back				

Skill	File	What it covers				
WeCom `dsh-superagent-wecom`	`skills/dsh-superagent-wecom/SKILL.md`	the callback carries only that message (no history, no deep link), plain text up to 2048, no idempotency key so a retry duplicates				
DingTalk `dsh-superagent-dingtalk`	`skills/dsh-superagent-dingtalk/SKILL.md`	the stream carries only that message (no history, no deep link), Markdown up to 5000, at-least-once delivery, `@` judged on staffId				

`composeSkill` now composes every built-in Source skill from its `skill/` part (which is what the panel shows and overrides), while a team skill keeps its own text; `tests/model-surface.spec.ts` unions the environment variables every kind declares, so a new source never fails the "variable is not registered" check again.

4. The WeChat sidecar

A separate Python process (standard library only) on `127.0.0.1`, authenticated with a bearer token read from an owner-only file. The contract is frozen and versioned; the Node side is the reference consumer.

Endpoint	Purpose
`GET /health`	whether the sidecar and its backend are usable, plus `capabilities`
`GET /status`	the signed-in account, and the mention name being judged against
`GET /rooms`	the chats the backend **can see** (the picker's candidates), not the ones it watches
`GET /members?room=`	the people it has seen in that chat (or the full roster, per capability)

Endpoint	Purpose
`GET /messages?room=&cursor=&limit=`	events since a cursor, with a `next` cursor
`POST /send`	send text, optional quote, optional `Idempotency-Key`
`GET /media/`	one message image's bytes; `404` when gone, `501` when the build cannot download
`POST /listen`	replace the watched set; idempotent, acts only on the difference

Invariants the contract keeps:

- **`ats` is never derived from the message body.** A backend that cannot produce a mention list says

so; it does not fall back to matching `@name` in the text.

- **`/media` never returns zero bytes with 200.** An empty payload is `404`.
- **`/send` is serialized** process-wide with at least one second between sends, and `/media` shares

that lock, because both drive the same client UI.

- **`/rooms` is read-only.** It never starts a listener as a side effect.
- **The bearer token never appears in a log line.**

5. WeChat backend tiers

5.1 The ladder

Tier	Backend	When it applies
L0	`wxauto` / `wxauto4` / `wxautox4` (UI automation)	default; needs an interactive Windows desktop session
L1	a `wxhelper`-class hook	**ruled out**: it targets WeChat 3.9.x only, and its message payload carries no mention list, so it would not deliver `@` either (ADR-005 §7)
L2	a `WeChatPadPro`-class iPad protocol service	no Windows machine available
L3	a hosted API	everything else is down and the work cannot stop

Only **L0** is implemented. L2 is a stub that answers `available: false` and `501` on data endpoints, so switching to it later is a configuration change plus a restart, not a code change.

5.2 Group `@`, and why it is graded

WeChat 4.x exposes **no structured mention list**: the message bubble is a flat accessibility leaf (`childCount == 0`, no text/value/legacy pattern), and the message object carries no mention field. So a group `@` is read from the client's **own markup** — a name picked from the mention list is followed by `U+2005` (four-per-em space), while a typed `@name` uses `U+0020` — plus `@所有人`.

That is stronger than substring matching, and weaker than a structured list, so the capability report says which it is:

Value	Meaning
`structured`	the client gave a mention list (the `fake` reference backend, and future L1/L2)
`markup`	the client's own markup, resolved against this account (L0 today)
`none`	the backend cannot tell at all

The account's own display name is reread on a 60-second TTL, forced on every `/listen`, and refreshed on the polling path, so renaming the account heals without a restart. A failed read keeps the old name rather than clearing it — clearing would make detection never match, which is worse than a stale name.

5.3 What L0 cannot supply

Contract field	L0 reality
`room` (chatroom id)	not available; chats are named `wxname_`
`sender.id`	a display name, namespaced the same way; `sender_info()` returns a WeChat ID, lags one message and is sometimes null
`at` (message time)	the client's time separators where they exist, otherwise the receive time
`/members`	only the senders seen so far (`members: observed`)

The namespacing is deliberate: it keeps the settings' id patterns strict instead of widening them to accept a display name, and `refreshLabels` resolves the ids back to names for the panel's pickers. The cost is that renaming a chat changes its id.

6. Adding a provider

40. Implement the port. For chat, an adapter needs: list chats, read messages since a cursor, send, and

(optionally) download media. For work, list/read/comment. For hosting, change requests, notes, CI.

41. Declare `capabilities` honestly — `idempotency` (`keyed` / `marker` / `none`), `maxChars`, `markdown`, `proactive`. The panel shows these; a wrong value becomes a wrong promise.

42. Write the `kind` with its schema, its fields (every schema key needs a field), its `scope` rule and its environment variables.

43. Add a `*-watch.ts` plugin entry and a `cordis.patch.yml` insert. Do not ship it `disabled: true`: a kind only

exists in the panel once its plugin registers it, and the panel's switch is what turns a Source on.

44. Add the file rows to `docs/007-architecture.md` — `tests/docs.spec.ts` fails if a module or a spec

file is missing from those tables.

45. Add a spec. Live behaviour goes in `tests/live/*.live.spec.ts` behind `LIVE=1`.

7. Verification matrix

What	How it is proven
the whole suite	`vitest run --maxWorkers=2`, green on macOS arm64 and Windows 11
the WeChat contract	a real sidecar process over loopback: 9 live cases (endpoints, `ats`, idempotent send, image bytes, `/listen`)
the WeChat client	a real WeChat 4.x client: listener callbacks, own identity, send round trip, image download, five markup cases proving a typed `@name` does not count
GitLab	the real `gitlab.com` API: merge-request mapping, a bogus token mapped to 401 without echoing it
WeCom / DingTalk / Yunxiao	real authentication failures surfaced as diagnosable errors, with no secret in the message
the panel data layer	two instances of one channel, deleting one, legacy settings without a channel id
Windows	the measured baseline and the 16 files that failed on a first clone, in HANDOVER-WIN11.md

Not verified, and not claimed: any write against a real code host or work item; the WeCom callback over a public reverse proxy; DingTalk Stream with its SDK plugged in; WeChat `structured` mentions (no tier that provides them is implemented); WeCom/DingTalk message images.

8. Conventions to keep

- **Never key `@` on text.** Scope reads `ats`. A backend may derive `ats` from the client's own markup, but the strength must be declared in `capabilities.mentions`, so a person can see whether a group `@` is proof or a hint.
- **One writer per state.** Sources change a Ticket only through `SourceContext.tickets`; health entries are claimed per instance; the outbox alone writes receipts.
- **Idempotency is declared.** `keyed` / `marker` / `none`, and the panel shows what the code does.

- **Secrets come from files**, read per call, owner-only, and never appear in a message or a log line.
- **Backend differences stay in the sidecar.** Identity normalisation belongs there, not in the settings schema.
- **`src/sources` does not import `src/app`.** Types both sides need live in `src/sources`.
- **No personal data, no machine-specific paths.** Placeholders in docs and fixtures; flags or environment variables for paths.

9. License

MIT, the same as the prototype this edition grew from; the upstream copyright notice is kept. WeChat automation through UI automation is third-party tooling: it is not authorised by Tencent, may be limited by the client or the platform, and must stay inside the boundaries recorded in ADR-005 — reply only when addressed, never broadcast, never bulk-add contacts, one account signed in at a time.

Capability matrix: the four added sources against Feishu and GitHub

Reference for what "aligned" means. Feishu and GitHub are the two sources the upstream superagent ships; the other four are this bundle's additions. ✅ present · — not applicable to the platform · ❌ impossible on the backend (stated, never faked).

Capability	Feishu (official)	GitHub (official)	WeChat	WeCom	DingTalk	Yunxiao Projex
Receive messages	✅ poll history + events	✅ poll notifications	✅ sidecar poll	✅ signed callback (plugin mounts `/wecom/`)	✅ stream (needs a `connect` injected; no SDK shipped)	✅ poll work items/comments
Assess / decide / execute / reply loop	✅	✅	✅	✅	✅	✅
Structured @ judgement	✅ open_id	—	✅ `ats` (markup fallback, strength in facts)	✅ `mentionsAgent`	✅ `ats` staffId	—
Direct messages	✅	—	✅	✅	✅	—
Outbound text	✅ markdown	✅ comment	✅ + mention-back	✅ plain 2048	✅ markdown 5000	✅ RICHTEXT/Markdown comment
Outbound files	— (CLI)	—	✅ `sendFiles`	❌ API text only	— (webhook)	✅ attachments
Idempotent send	✅ keyed	✅ marker	✅ keyed	❌ none (stated)	❌ none (stated)	✅ marker
Thread context for the assessment	✅ thread/root/parent	✅ PR/issue	✅ thread/parent; `root→no_root`	✅ thread; `root/parent→no_*`	✅ thread; `root/parent→no_*`	✅ item body + comments
Worker reads of the original	✅ `feishu//…`	✅ `github//…`	✅ `weixin//…`	✅ `wecom//message`	✅ `dingtalk//message`	✅ `projex//item, /comments`
Inbound images for the assessment	✅	—	✅ `[image:], /media`	✅ `MediaId, /cgi-bin/media/get`	— (text stream)	✅ item/comment images archived

Capability	Feishu (official)	GitHub (official)	WeChat	WeCom	DingTalk	Yunxiao Projex
Panel search	✅ message id	✅ issue/PR number	✅ native id	✅ decimal `MsgId`	✅ `msgId`	✅ serial or page URL
Reply→parent mapping	✅	✅	✅ (quote id)	— no quote in callback	—	—
Display-name refresh	✅ chat names	✅ repo names	✅ rooms/members	✅ department users (`refreshLabels`)	❌ needs contact permission API, not offered	✅ status vocabulary
Member/roster sync into Policy	✅ chat members	✅ collaborators	✅ group members	❌ group members unreadable; user list only fills names	❌ same	✅ project members
Health entries	`feishu` + `history`	`github`	`weixin`	`wecom`	`dingtalk`	`projex`
History backfill	✅ chat history	✅ notifications	✅ post re-attach catch-up (marked `historical`)	❌ callback only	❌ stream only	✅ first-sight comments
Message deep link	✅ AppLink	✅ issue/PR URL	❌	❌	❌	✅ item page
Multiple instances	✅	✅	✅	✅	✅	✅

Platform limits, and why they are not bugs

- **No deep link** on WeChat, WeCom and DingTalk: none of the three exposes a per-message URL; the panel names the Source.
- **No history** on WeCom and DingTalk: both deliver by push (callback / stream) and offer no read-back endpoint, so no

backfill and no `/thread` pages. WeChat reads history through its sidecar; Feishu through the history API; Projex through the work-item APIs.

- **DingTalk transport**: the WebSocket client is the `dingtalk-stream` SDK, which this bundle deliberately does not vendor.

A deployment injects `connect()` (see `src/sources/wecom…` no - see `src/sources/dingtalk-watch.ts`); without it the kind is registered but nothing listens, and health says so. Everything downstream of the stream (Trigger, ThreadContext, worker read, reply, policy) is aligned.

- **WeCom / DingTalk member sync**: a self-built app cannot enumerate a chat's members; WeCom fills Policy *names* from a

department it is allowed to read, and both refuse to pretend a visible-user list is a chat roster.

- **WeCom / DingTalk / WeChat images**: WeChat and WeCom deliver images with a media id the deployment can download; DingTalk

stream events carry text, so no image is claimed.

What this round added

46. `projex//item` and `/comments` worker reads (`src/sources/work/projex/read.ts`), with paging and the agent's own

comment marked `bot`.

47. `dingtalk//message` worker read and a DingTalk ThreadContext (`thread` = the one message, `no_root`,

`no_parent`) — `src/sources/chat/dingtalk/{read,kind}.ts`.

48. WeCom `refreshLabels` over the contact user list, plus a `departmentId` setting, so Policy rows show names.
49. A `dsh-superagent-dingtalk` skill, and the Projex skill now names its two resources.
50. Tests: `tests/projex-read.spec.ts`, `tests/dingtalk-read.spec.ts`, plus the existing WeCom/WeChat/Projex suites.

ADR-005 Backend selection and switchover strategy for WeChat personal-account group messages

- **Status**: Accepted (2026-11-06, confirmed by the solution owner)
- **Decision scope**: the backend form, primary/standby strategy and deployment prerequisites of `src/sources/chat/weixin/*`
- **Related**: design document §6.7, §6.7.9, §6.7.10; this document does not repeat the background, it only fixes the decision

1. Background (one sentence)

The WeChat group `@` bot is a **must-have** for this project, but the official ClawBot channel only covers one-to-one chats, and the open-source community's main backends have kept exiting over the past three years (itchat → gewechat → wcf). Therefore **any solution that "picks one backend and depends on it for the long term" is unacceptable**.

2. Decision

2.1 Form: a resident Windows sidecar + loopback HTTP

WeChat client automation runs on a **resident Windows machine**, exposing an HTTP contract on `127.0.0.1` as an independent process (`sidecar/wxauto-bridge`) (see `sidecar/wxauto-bridge/contract.md`); dsh-superagent-multi-channel calls it through the `src/sources/chat/weixin/pc-uia/*` adapter.

Reasons:

51. Client automation and the Agent runtime are **isolated in failure domain** — a WeChat disconnection should not drag down ticket execution;
52. The sidecar can be implemented in any language with any library, so switching backends does not touch DSH-side code;
53. The contract has few endpoints (one group each for read, send, media and listener set), so the deserialisation surface is small and the security surface is small.

2.2 The primary/standby ladder (by priority, not parallel candidates)

Tier	Backend	Condition that triggers enabling
L0	`pc-uia` (the wxauto kind, UIAutomation, no process injection)	default

Tier	Backend	Condition that triggers enabling
L1	`pc-hook` (the wxhelper kind, DLL injection + HTTP)	**struck out**: it only adapts 3.9.x, and the messages it hooks out have no `atuserlist`, so even installed it cannot get `@` (see §7 audit)
L2	`pad` (WeChatPadPro / community fork, iPad protocol)	no Windows machine, only Linux; the business accepts the protocol risk control in writing
L3	`hosted` (paid API)	L0–L2 all down and the business cannot stop
Bypass	WeCom customer group / official ClawBot one-to-one chat	the customer is willing to migrate the group / only one-to-one is needed (this is a requirement change, not a technical downgrade)

Only one instance holds the login state of the same WeChat account at any moment (double-send / risk-control red line).

2.3 Hard constraints (written into the DoD; if unmet, "switchable" does not hold)

54. `WeixinBackend` ≤ 8 methods; a single backend implementation ≤ 400 lines.
55. On the dsh side, `replyConcurrency = 1` for the WeChat instance; `/send` is serialised by the bridge + 1s interval + jitter.
56. Switching backends = change the configuration + restart that instance, **without changing DSH code**; target ≤ 2 person-days.
57. Group `@` judgement recognises only `ats`; reverse-parsing the text `@nickname` is **forbidden**. `ats` may be derived from the client's own mention

markers (see the `markup` tier in §7), but the strength must be declared in `/health.capabilities.mentions`, and it must not degrade into "the substring `@name` appeared in the body".

58. Permission boundary: passive replies only; no mass sending, no group creation, no friend adding.

3. Consequences

Positive

- If any backend exits, the business recovers within 60s (primary/standby takeover) to 2 person-days (cross-backend migration).

- Group messages, `@` and group members are three things fixed at the contract layer; the Node side depends only on the contract.

Negative / costs

- A Windows machine (or a real Android device) must stay resident; this is an **ops cost**, not a code cost.
- wxauto's author disclaimer prohibits production/commercial use → written confirmation from business + legal is required, otherwise L0 is unusable and it falls straight to L1.
- The bridge side adds one more process + one set of credentials (token + login state) to manage.

Not doing

- Not doing "multiple backends online simultaneously with double writes" (double sending in the same group).
- Not embedding client automation into the dsh process (mixing failure domains).
- Not promising "no risk control", only promising "switchable when limited".

4. Acceptance (the standard for this ADR being complete)

59. `poc/probe_wxauto.py` produces `probe-report.json` on the target Windows machine, and both **Q1 (group message fields + whether a structured ats exists)** and **Q2 (group member table)** have evidence.
60. `sidecar/wxauto-bridge` passes the contract self-test with `--backend fake` (already runnable in this repo, see the README).
61. After the real backend is connected, the six scenarios of design document §6.7.8 pass (including "@ still works after a nickname change" and "a retry does not send twice").
62. Primary/standby switchover drill: stop the L0 process → L1 takes over within 60s, with no duplicate messages in the group.

5. Rollback

- Single-instance rollback: set the instance to `enabled:false` → that group is no longer connected, other channels are unaffected.
- Cross-backend rollback: change the `backend` configuration back to the previous tier, restart the instance; the login state requires re-scanning the QR code (WeChat does not allow two places online at the same time).
- Complete rollback: delete the `weixin` instance configuration and set the plugin to `disabled`; behaviour equals the v1.0 solution (no WeChat integration).

6. Open prerequisites (blocking items that the business side must provide)

#	Prerequisite	Who provides it	Consequence of not providing it
1	Commitment of a resident Windows machine (or a real Android device)	business/ops	L0/L1 are all unavailable; the only options are L2 (highest risk) or the bypass
2	Confirmation of the wxauto production/commercial licence (legal)	business + legal	L0 is unavailable and it falls straight to L1 (injection risk)
3	A version-freeze policy for the target WeChat PC client (auto-upgrade not allowed)	ops	A client upgrade may break it immediately, and the switching cost goes back to "measured in weeks"
4	**The probe and the client must be in the same interactive desktop session** (physical console or an unlocked RDP; it cannot be run from an SSH/service/CI session)	ops	`wxauto` reads the client window through UIAutomation; across sessions the window is never visible, manifesting as `WindowDiscoveryError`

7. Real-machine probe measurement records

2026-11-06 first run (Windows 11 / Python 3.13.17)

- Environment: `Windows-11-10.0.26200-SP0`, wxauto **3.9.11.17+dsh1** (built from `wxauto-fork/wxauto-main`)
- Result: `import wxauto` ✅; `attach client` ❌

```
WindowDiscoveryError: No visible Weixin/WeChat top-level window is accessible in Windows session 21. Discovered client window sessions: (none).
```

- Verdict: **not a code defect, but a session/window visibility problem**. The probe process is in session 21 (a typical RDP/remote session), and UIAutomation can only enumerate top-level windows of its own session; if the client is in another session, or not started, or only in the tray, it all manifests as `(none)`.
- Therefore `poc/win_diag.py` was added: `python win_diag.py` directly reports the current process session, the session the WeChat window is in, whether it is administrator, and the

next action; `probe_wxauto.py --diag` is equivalent. When attach fails it automatically prints next steps.

- **Admission status: still not passed** (Q1/Q2 both lack evidence). The next run must satisfy §6 prerequisite 4, and requires the client version to match the wxauto build (3.9.x and 4.x are two different UI sets).

2026-11-06 second run (same machine, session problem resolved)

- The probe can now **see and attach to the window**: `Weixin.exe`, pid 46576, session 21, class `Qt51514QWindowIcon`, title 微信;

but `RuntimeError: Attached to Weixin/WeChat HWND 0x1F0C2A, but the wxauto 3.9 accessibility tree was not found`, root children = `MMUIRenderSubWindowHW` (the new UI container of WeChat 4.x).

- **Verdict: the client is WeChat 4.x, and `wxauto` (the 3.9 line) cannot drive it**. This is a version-generation problem, not a session problem (the window was visible this time).
- Feasible paths (choose one; costs/risks differ):

Path	Approach	Cost	Risk
4.x route (do not touch the client)	`pip install wxauto4` (free, Python 3.9–3.12, client ≤ 4.1.8.107) or `pip install wxautox4` (paid, Python ≤ 3.13)	Need to install Python 3.12 or buy an activation code; the client may need to be downgraded to ≤4.1.8.107	The free version is sensitive to the client version
3.9 route (do not touch the toolchain)	Install WeChat 3.9.11.17 + disable auto-update, keep using the existing `wxauto` fork	Need to lock the version on the client	The vendor may force an upgrade

- **Q1 open risk (important)**: wxauto4's documented `Message` only has `attr/type/id/sender/content/info`, with **no documented structured `@` list** (`at` only appears on the sending side in `quote(text, at=[...])`). Therefore whether a group `@` can be judged structurally depends on whether the runtime `info` carries an at list — this must be decided after the probe dumps it. If it indeed does not, per the §6 prerequisite rule **the text `@nickname` reverse-parse must not be used**; switch to L2 (pad) or choose another backend.
- Tools upgraded: `poc/win_diag.py` judges 4.x as soon as it recognises `Weixin.exe`/`Qt51514QWindowIcon` and gives package advice; `poc/probe_wxauto.py` tries `import wxauto4` first and falls back to `wxauto`, and writes the package name into the report.

2026-11-07 probe v3: first guarantee that "a report always comes out"

- Symptom: running the old probe on Windows **did not generate `probe-report.json`**. Three reasons (all fixed):

1) The old version's top-level `import win_diag` was a hard dependency, so copying only `probe_wxauto.py` gave `ModuleNotFoundError` and not a single line was written; 2) The old version only tried `wxauto4` / `wxauto`, **not `wxautox4` (Plus)**, so even with the commercial version installed it judged "no package"; 3) `--out` defaulted to a filename relative to cwd, so the report might land in another directory.

- Since v3: `win_diag` is optional; packages are tried in the order `wxauto4 → wxautox4 → wxauto`; the report is always written in `finally` and prints the **absolute path**; `--selftest` was added (does not touch WeChat, only verifies the file-writing path).
- At the same time `@` detection was tightened: `attr` (message source system/self/friend) was removed from the candidates, and values in `atListCandidates` are required to really look like a list — otherwise the old logic would treat `attr` as an at list and **directly fool Q1**.
- Also, per user feedback: **after licensing the commercial version, WeChat message sending and receiving succeeded**. This shows the client and the Plus library are usable on that machine, but "the product can receive messages" does not equal "we can get a structured @ list"; Q1 still needs `atListCandidates` evidence.

2026-11-07 group message images (Q4): the capability is confirmed to exist; measure three metrics before implementing

- Based on the v4.0 documentation: in the message class table, ✨图片消息 `ImageMessage` carries ✨ (exclusive to the Plus build); that class provides

`download(dir_path, original)` and `ocr(timeout)`; an image in a group is an ordinary message with `type='image'`, `sender` is the group member, and `chat_info()` returns `chat_type='group'`. So the "recognise → download → OCR" chain holds, but **the commercial version is required**.

- Three constraints that must be measured first: ① the free build may only give `type='image'` without `download()`;

② `GetAllMessage()` only covers messages already rendered in the current window, and earlier images require `roll_into_view()` first, so with too large a polling interval the image may already have scrolled out of view; ③ downloading occupies the client UI and must be serialised with switching chats/sending messages, and becomes slower when the user is operating at the same time.

- The probe adds `--media`: it reports `media.imageCount / hasDownload / download.bytes / download.ms / ocr`,

and on builds without `download()` it explicitly outputs `image messages need the Plus build`, never a false green.

- Implementation design (to be implemented once the data confirms the parameters): `ChatEvent.media` + sidecar `GET /media/` (download cache + TTL)

+ `weixinKind.threadContext.image()` to fetch the image as a model attachment (reusing the existing pattern of `feishu-image.ts`); outbound image sending is not done for now.

2026-11-07 Q1 verdict: L0 (pc-uia) has no structured @; but the conclusion "L0 is unusable" is wrong

> *This section keeps two layers: first make clear **which part of the conclusion holds**, then make clear **which part was my misjudgement**. The latter is more important,*
> *because following the wrong conclusion would shut down a backend that works.*

The part that holds: there is no structured @ list

Environment: Windows 11 (session 1, console, Active) / Python 3.12.10 / `wxautox4` 41.2.3 / WeChat 4.x (`C:\Program Files\Tencent\Weixin\Weixin.exe`, `Qt51514QWindowIcon`) / logged-in account `. `Probe:` poc/win_diag.py → a WeChat window is visible in this session; poc/probe_wxauto.py --group "" --media → poc/probe-report.json`.

> ***Privacy**: `probe-report.json` and the message dump contain **real chat content**. They are only used for local forensics, are not committed,*
> *and do not go into documents. Everything cited in this document is the **shape of the value** (`, , , , ``), not who exactly it is.*

Conclusion: `atListCandidates` is empty → per §4.1 the backend should switch to L2 (pad); a group `@` must not be text reverse-parsed.

Three independent pieces of evidence (any one alone is enough to rule out L0's structured @):

63. **The UI tree is flat leaves.** The message control `mmui::ChatTextItemView` (`AutomationId` `chat_message_list.qt_scrollarea_viewport.chat_bubble_item_view`) has `childCount == 0`, and `TextPattern` / `ValuePattern` / `LegacyIAccessiblePattern` / `SelectionPattern` **are all absent**. `@nickname` and the body are the same `Name` string; the client does not render the mention as a separate element.

64. **The message object has no at list.** All fields of `FriendTextMessage` are `id / content / hash_text / hash / sender / chat_info / type / attr` (`attr` is the `friend|self|system` source marker).

65. **`FriendMessage.at` is an action, not a field.** It is a bound method; executing it is a "nudge" (拍一拍) and returns

`WxResponse{"status":"成功"}`. Treating it as a value getter would **nudge a group member once per poll** — a real side effect.

> *The only structural clue carrying `@` is the `U+2005` (four-per-em space) separator in the body, for example `@\u2005`妈.*
>
> *It is a **text** feature and can be forged by copy-paste, so it is recorded only as evidence in the report's `textSeparator`*
>
> *`(textDerived: true, usable: false)` and **does not constitute a judgement basis** (§2.3 hard rule 4).*

Four other L0 gaps clarified along the way (also written into the contract, so they are not misjudged as bugs later):

Contract field	Actual L0 situation	Consequence
`room` (chatroomId)	Unobtainable. `ChatInfo()` only has `chat_type/chat_name/group_member_count`; `SessionElement` only has `name/time/content/isnew/new_count/ismute`	`rooms[].id` can only use the display name, and the Node schema's `^[A-Za-z0-9_-]+$` rejects Chinese group names → **it simply cannot be expressed at the configuration layer**
`sender.id`	Display name only. `sender_info()`'s `id` is the WeChat ID (e.g. `), not wxid_`, and it was **observed lagging by one message, sometimes returning `null`**	The Policy's wxid list is not enforceable on L0
`at` (message time)	Ordinary messages have no time field, but the `TimeMessage` separator carries `time`; remembering the most recent separator per room gives minute-level alignment, and only historical messages from before the listener started fall back to the server receive time	Already implemented this way, see §3.2
`/members`	No group member API (`_api` only has `add_group_members`)	The member table degrades to empty

Therefore L0 did not pass its admission bar, and not only on `@`: even for one-to-one chats only, identity and timing do not reach the contract. The strict `WXID_PATTERN` on the Node side is not a defect here but an **admission gate** — it blocks deployments that do not meet the bar.

The part I judged wrongly (corrected on the evening of 2026-11-07)

The analysis above **has correct evidence but a wrong conclusion**. Wrong in two places:

Mistake one: I used `GetAllMessage()` to poll for messages. It only returns messages **already rendered** in the current window, and `ChatWith()` does not redraw when the target is already the current chat, so it periodically read 0 messages. I took this phenomenon as "L0 cannot receive

group messages". The owner pointed out that there is a working production implementation on this machine (`D:\\wxbot_core.py`), and it uses **listening mode**:

```
self.wx.AddListenChat(nickname=..., callback=self.message_handle_callback)
self.wx.StartListening()
```

A real-machine retest (self-test with "文件传输助手" only, nothing sent into a group) confirmed: `AddListenChat` returns the chat name, after `StartListening()` one `SendMsg` triggers the callback **twice in real time** (`TimeMessage` + `SelfTextMessage`), the `chat` object gives `{'who': '文件传输助手', 'chat_type': 'friend'}`, and `msg.chat_info` gives `{'chat_type': 'friend', 'chat_name': '文件传输助手'}`. **Push-based, does not depend on window rendering.**

Mistake two: I inferred "no structured list" straight into "not admitted". The production implementation uses text substring matching:

```
self.config.AtMe = "@" + self.wx.nickname      # 2714 行
if self.config.AtMe in message.content: ...    # 3381 / 3405 / 3729 / 3748 行
```

It does work. But substring matching even misjudges "he just said the sentence `@`". **A better criterion is the client's own marker**: when you really pick a person from the mention list, WeChat inserts `@` + **`U+2005` (four-per-em space)**; typing by hand uses `U+0020`. Two pieces of evidence from the real-machine dump:

```
"@<成员>\u2005一会儿带两个豆浆回来，我们吃过了"
"@<成员>\u2005妈"
```

So "`@this account's display name` + `U+2005`" or "ends with `@this account's display name`" is the criterion, and additionally `@所有人\u2005` also counts as speaking to this account. This is not text reverse-parsing — it requires a marker generated by the client itself, which cannot be typed by hand (but can be pasted, see below).

The corrected decision

`capabilities.mentions` changes from a boolean to three states, writing "how strong the proof is" into the contract instead of hiding it in code:

Value	Meaning	Who
`structured`	a mention list given by the client	`fake` (reference implementation), future L2/L1
`markup`	the client's own mention marker + this account's display name	**`pc-uia` (L0)**
`none`	cannot be judged at all	the `pad` stub

The cost of `markup` must be stated, because it affects "whether a group `@` can be treated as authorisation":

66. **The criterion name changes**: the criterion contains this account's display name, so the display name gets a **60-second TTL re-read**, is **force re-read** on `/listen`,

and is refreshed incidentally by `/messages` polling; changing the nickname **requires no restart**, and the current criterion name can be seen directly in the `detail` of `/health` and `/status`. If the read fails, keep the old name and record a note (clearing it would make the judgement never match, which is worse than an old value);

67. **Forgeable**: the separator can be pasted, so it is stronger than substring matching, but **not unforgeable**;
68. Therefore the wording of §2.3 hard rule 4 changes to: **must not reverse-parse by `@nickname` text; judging by client markers is allowed,

and the strength must be declared in the capability self-description**. §8's invariants are consistent with this.

The identity problem is solved with a **namespace** rather than loosening the Node-side id regex: the sidecar names chats and senders that have no stable ID as `wxname_`, with the display name in the `name` field; the Node side's `^[A-Za-z0-9_-]+$` stays unchanged, and `weixinKind.refreshLabels` resolves the id back to a display name for panel display. The cost must likewise be stated: **if the display name changes, the id changes**, and the configuration must follow.

L2 / L1 are still the stronger paths, but no longer "the only usable path":

- **L2 (pad)**: can simultaneously give chatroomId, wxid, structured `ats` and real time. It requires the business side to provide

an iPad-protocol service such as WeChatPadPro. The sidecar's `--backend pad` already has a stub: `available:false` + data endpoints `501`; switching only needs a configuration change + restart, without touching Node code.

- **L1 (DB reader)**: the message DB location and encryption form are already confirmed on this machine; it is a route that obtains **all structured fields without injecting into the process**.

The evidence and approach follow. It solves the `structured` tier, not "whether it is usable".

Tier feasibility audit: L1 and L2 checked against the real documentation (2026-11-07)

§2.2's ladder assumes "L1 (injection) can get a structured `@`" and "L2 (pad) can solve everything". After checking the **real documentation**, both assumptions need correction — this step exists so nobody does wasted work following the ladder.

L1 (an injection plugin such as `wxhelper`) = unusable on this machine, and even if usable it does not solve `@`.

- It only adapts to **WeChat 3.9.x** (`WeChat.exe`): the repo README explicitly lists 3.8.0.41 / 3.8.1.26 / 3.9.0.28 /

3.9.2.23 / 3.9.2.26 / 3.9.5.81, `doc/` only has 3.9.2.23 and 3.9.5.81, the highest tag on jsDelivr is `3.9.10.19-v1`, and the whole repo has **no** trace of 4.x / `Weixin.exe`. This machine is 4.x.

- More critically: the message JSON it hooks out has **11 fields**

(`pid/fromUser/toUser/content/signature/msgId/msgSequence/createTime/displayFullContent/type/base64Img`), with **no `atuserlist` / `atList`**, and no endpoint returning a structured `@` list. In other words: after switching to the 3.9 client and accepting the injection risk, the field we want **still does not exist**.

- It listens on `0.0.0.0:19088` by default with **no authentication whatsoever** (source `StartHttpServer`), which amounts to exposing the entire WeChat account to the LAN.

The author himself writes in the README that it "may cause consequences such as account bans".

L2 (a pad service such as `WeChatPadPro`) = it passes the version gate, but the documentation does not give the `@` gate.

- It does not drive a local client; it is an iPad-protocol endpoint (iOS 18.61 / build 861), so "this machine has 4.x installed" is irrelevant to it.
- The public Cloud swagger (`gwpad.com/Cloudswagger/swagger.json`) can be checked for **401 endpoints**:

`basePath=/v1`, response envelope `{Code, Data, Message, Success, request_id}`, auth header `X-Cloud-Access-Token`.

- **But a full-text search for `atuserlist` hits 0**; `at_list` / `AtList` are all false hits on `chat_list`; the only field related to `@` is

the **sending-side** `Msg.SendNewMsgParamDoc.At`. The inbound `sync_message` event **has no field-level schema**. In other words: **whether a structured `@` list exists in inbound events cannot be confirmed from the public documentation**.

- Group id and sender wxid are documented: `Msg.QuoteContextDoc.chat_user_id` (`room@chatroom` for group chats),

`from_user_id` (the member wxid for group chats). So L2 can indeed fill L0's gaps on **identity**.

- Closed-source binary + licence-code billing, the latest version only in the sponsor group; still maintained (v2.01 released 2025-08-22).

The documentation has traps: the local swagger file the README points to is no longer in the repo, the old documentation site is 404, and only the swagger above is trustworthy.

Conclusion: the ladder changes from "L1 first, L2 second" to — **L1 is struck out** (unusable on 4.x, and it does not provide an `@` list); **L2 is the only candidate**, but whether it can give a structured `@` can only be confirmed by the licence holder in a real test.

A one-off verification for the licence holder (10 minutes; decides whether L2 is worth adopting): log in a test account with the pad service, pull a group, have a group member @ that account once, then look at the raw JSON of that message in `sync_message` (or a synonymous inbound event):

- If fields shaped like `atuserlist` / `at_list` / wxid-carrying fields inside `msgsource` appear → L2 holds, and `mentions` can be upgraded to `structured`;
- If not → L2 can only fill identity and time, `mentions` stays at the `markup` tier, and then there is **no need to switch backends for `@`**.

L1 feasibility evidence (confirmed, not implemented)

```
%USERPROFILE%\Documents\xwechat_files\<account>_<hash>\db_storage\
  message\message_0.db   11,882,496 B   ← 群/单聊消息
  contact\contact.db     23,150,592 B   ← 联系人、群成员
  ...
```

The first 64 bytes of `message_0.db` are random bytes (`79 4b ad 98 12 22 ba 23 …`), **not** `SQLite format 3` → confirming SQLCipher encryption (WeChat 4.x uses SQLCipher v4: 4096-byte pages, AES-256-CBC, page-tail HMAC-SHA512). The file is opened exclusively while the client runs; reading requires `FileShare.ReadWrite`.

Once the key is obtained, the structured fields built into the protocol layer map directly to the contract: chat `username` (= chatroomId), sender `sender` (= wxid), `create_time` (= real time), and the at list in the message proto (= `ats`). **This single item solves Q1 and Q2 at once**, and it is also the only L0 alternative that does not violate "text reverse-parsing forbidden". The key must be taken from `Weixin.exe` process memory (this machine is an administrator session, so the conditions exist); it belongs to L1's "injection/memory reading" risk tier and requires written acceptance from the business side before doing it.

L1 assault: how far we got, and why we stopped (2026-11-07 real test, key not obtained)

Approach: SQLCipher accepts a **raw key** (`x''`); in that case the KDF is skipped, and the key bytes serve as both the encryption key and the HMAC key, so **one HMAC-SHA512 verifies one**

candidate (measured about 31 µs/candidate on this machine). This makes "scanning" feasible, unlike PBKDF2 (256000 rounds ≈ 0.1 s/candidate), which can only rely on known offsets.

Already ruled out:

69. **`login\\key_info.db` is plaintext SQLite** (`SQLite format 3`), with only one table `LoginKeyInfoTable(user_name_md5, key_md5, key_info_md5, key_info_data BLOB)`, 5 rows, `key_info_data` is 180 bytes per row, isomorphic to `%APPDATA%\Tencent\xwechat\login\ \key_info.dat`. → This shows what is stored here is the **wrapped** key, not plaintext.
70. **Brute-forcing that blob failed completely**: treating the 180 bytes at the five lengths 16/24/32/40/48 and at every offset as candidates

(27660 raw-key attempts, 0.22 s), then using the `user_name_md5` ASCII and raw bytes, the first/middle/last slices of the blob, and the whole blob to try PBKDF2 (64000 and 256000 rounds × two page layouts SHA512/SHA256, 810 attempts, 143 s) — **all missed**. Across the 5 rows + `.dat` there is also no shared 32-byte window.

71. **Process memory scan: done but not enough to conclude**. `Weixin.exe` has 6 instances; within a 180-second budget only 46 MB was scanned

(5,857,300 candidates, 8-byte aligned), no hit. At 31 µs/candidate, scanning these processes fully at stride 8 takes about 30–60 minutes; this run did not finish.

Why stop here: the credibility of a negative result depends on the verifier itself, and I **have no known-good SQLCipher library** to calibrate it against (page layout 4096/reserve 80 vs 48, whether the page number participates in the HMAC, whether the salt skips 16 bytes — all inferred from documentation). Getting "not found" after 40 minutes on an uncalibrated verifier cannot be written as "does not exist". The correct order to continue is:

72. First build a database with a known key using the sqlcipher CLI and align the verifier to it (this is 10 minutes of work, but it must be done first);
73. Then write the scanner in C/Rust (Python's 31 µs/candidate is the lower bound; C can reach the 1 µs order), full scan at stride 1;
74. Only after obtaining the key discuss the DB reader: the schema of `message/message_*.db`, the mention field of group messages,

and the member table of `contact.db` — that is another independent workload.

Conclusion: **L1 is still the strongest path, but it is an engineering task that needs calibration before implementation, not something one session can close**. Before that the `markup` tier is

usable, and its three costs have been reduced to two (rename self-healing, the id will change, forgeable).

2026-11-07 group message images: implemented (commit cfa971e)

Chain (every segment has tests): the sidecar `/messages` lists `media:[{id:'#0',kind:'image'}]` only for builds that **can download** → `bridge.toChatEvent` maps it into `ChatEvent.media` → `weixin-watch` puts the id into `Trigger.meta.media` → `weixinThreadContext.read()` appends an invisible `[image:]` at the end of the image message body → `images(text)` extracts it with the same regex → `image()` goes through `GET /media/` → magic-number detection → `attachments.saveImage` → visible to the model.

Non-negotiable conventions: judge only by magic number (PNG/JPEG/WebP/GIF) and **do not trust** the remote Content-Type; when it exceeds `imageLimits.maxImageBytes` or the format is unacceptable, **throw before saveImage** (nothing is written to disk, nothing enters the context); `/media` and `/send` share the UI serialisation lock; builds without download capability return **501** for `/media`, **404** for a location failure, and never return empty bytes.

Legacy Minor items (review t54, all low risk, recorded):

- IMG-1 applies only the single-image `maxImageBytes`, without stacking `maxMessageImageBytes`/the per-message image count cap
- IMG-2 the bridge reads the whole `arrayBuffer` first and then checks the limit, without a streaming cap
- IMG-3 the sidecar `/media` success branch does not explicitly reject zero bytes
- IMG-4 `imageIds()` parses from the body, so a user can fabricate `[image:x]` to trigger one request (the sidecar 404 catches it; suggest later accepting only ids in `meta.media`)

ADR-006 Health entry key compatibility: the default instance keeps its bare name

- **Status**: Accepted (2026-11-06, confirmed during P0 implementation)
- **Decision scope**: the key shape of health entries in `SourceRegistry.healthEntries` / `sourceContextOf` / `views/sources.ts`
- **Related**: design document §6.6, §4.3; P0 delivery ticket §1.5; implementation task t3

1. Background

Design document §6.6/§4.3 requires that "health entries are always named `instance:entry`": one kind can have multiple instances, and only by splicing the instance ID into the key can two instances of the same kind each report their own health without conflicting. The original plan in P0 ticket §1.5 and §4 risk table R2 also says "the key is always `instance:entry`".

But P0's hard constraint is "zero behaviour change": settings in the old database, old panels, and existing ops scripts all read health entries by the bare names `github` / `feishu` / `history`. If everything were changed to `github:github`, the old panel would instantly be unable to read health after an upgrade, and rolling back to the old version would also be unable to read the new-shape keys.

2. Decision

- **The default instance (`id === kind`, such as `github`, `feishu`) keeps the bare-name entry claimed at registration**: `github`, `feishu`, `history`.
- **A named instance (`id !== kind`, such as `github-2`) lazily claims `${instance}:${entry}`**: `github-2:github`.
- Reasons:

75. Old databases and old panels read with zero change; an upgrade needs to migrate no persisted data;
76. Rollback is safe — the old version can still read the default instance's health by bare name;
77. Named instances are a new concept with no historical baggage, so they adopt the canonical form directly.

3. Consequences

- **The key shape is asymmetric**: the default instance of a kind is `github`, while the second instance is `github-2:github`. Every new reader must know this rule.

- **views reads must "look at the bare name first, then the prefix"**: `entryOf` in `views/sources.ts` uses `instance.id === instance.kind ? name : healthEntry(instance.id, name)`; `sourceContextOf` passes `kind.id ?? kind.kind` to `healthEntries`, and `healthEntries` uses the registration-time bare-name entry when `instance === kind`, otherwise it lazily claims the prefixed entry.
- **Do not "unify it while you are at it"**: changing the default instance to `github:github` as well would immediately break the old panel and rollback compatibility, and is a migration that needs separate review.
- **Migration steps if unification is wanted in future**:

78. New code double-writes and double-reads first: maintain both the bare-name and the `${instance}:${entry}` keys;
79. Panels and views switch to the prefixed form first, keeping the bare name for one version as a fallback;
80. Health is runtime in-memory state and is not persisted itself, so the persistence layer usually needs no change; if monitoring/scripts store snapshots separately, their read paths must be changed in step;
81. After observing one release cycle with no anomalies, delete the bare-name branch.

- **Risk**: during double-writing the two keys may be briefly inconsistent (old code only writes the bare name), so they must be updated together in the same report; if external consumers read the bare name directly, alerts are silently lost during the migration.

4. Acceptance (both sides covered by tests)

- `tests/kv-instances.spec.ts`: covers the **key generation side** — `healthEntry('wecom-hr','wecom') === 'wecom-hr:wecom'`, its difference from `wecom-ops`, and the new KV families keying per instance.
- `tests/registry-instances.spec.ts`: covers the **claim/report side** — two named instances (`wecom-ops`/`wecom-hr`) each claim `${id}:wecom` without overwriting each other, `report` still only allows entries declared by the kind; old `github` settings resolve by bare name to the default instance (the `id === kind` side is guarded by case 2 of that file).

ADR-007 Aligning WeChat channel capabilities: SiverWXBot's feature surface vs this repository

- **Status**: Accepted, all three phases implemented (2026-11-08): P1 `7b4e316`, P2 `c4350a8`, P3 this commit
- **Decision scope**: the WeChat source's (`src/sources/chat/weixin/`) panel fields, sidecar outbound capabilities, and the ownership of keyword-based routing
- **Related**: ADR-005 (L0 backend and the @ criterion), ADR-006 (health entry keys), `sidecar/wxauto-bridge/contract.md`

1. Summary of conclusions

82. **One-to-one chats**: listening, text and images are all usable (verified on a real machine). "Add contact" in the panel simply selects that person from the chat dropdown — the bridge only lists **chats it has actually seen**, and there is no entry point for typing a wxid by hand (the display-name-derived `wxname_…` is not a real wxid).
83. **Group chats**: listening, reply-only-on-@, and group images are all usable; @ is at the `markup` tier (client marker), not a structured mention.
84. **Two library limitations**, not configuration problems:
    - **Quoting a message when replying: not doable**. Among `wxautox4`'s `SendMsg(msg: str, who, clear, at, exact)` and all of `Chat`'s public members there is **no quote/ref parameter at all** (`Chat` members: `SendMsg SendFiles SendAudio AtAll GetNewMessage …`). WeChat 4.x's "quote" can only be done through right-click menu UI interaction, which is fragile and not exposed. Substitute: write the quote into the body prefix (「引用 @张三」原文…).
    - **@-ing the speaker when replying: doable**. `SendMsg(..., at: Union[str, List[str]])` supports it, and `at` takes a display name; this repository previously **deliberately** did not wire it (the `send()` docstring states "never put `at=`"), because inbound `FriendMessage.at` is a "nudge" action and was once misused as a group @. Outbound `at=` is a different path, and wiring it requires separate real-machine verification.
85. **Speech recognition**: inbound is already implemented (`to_text`, 8-second cap, tried once, degrading to （语音转写失败） on failure), but it **has not been verified on a real voice message**; every call occupies the client UI for up to 8 seconds, so it needs a switch rather than being always on by default.
86. **Keywords**: this repository has none at all; it is a new capability, not a checkbox. It can be done in two layers, see §5.

2. Item-by-item alignment

SiverWXBot's capability	Current state in this repository	Configurable in the panel?	Conclusion
Enable group listening	Already present (`rooms` collection, `/listen` replaces the set, idempotent)	✅ present	Aligned
Reply only when @-ed	Already present (`atOnly`, criterion is `markup`)	✅ present	Aligned; applies to groups only, one-to-one chats have no @ concept
One-to-one listening / one-to-one reply	Already present (`room: ""` one-to-one stream, id → stream mapping)	✅ present (pick a person from the chat dropdown)	Aligned; missing the wording and hint for the "one-to-one" case
Group images	Already present (inbound `media` + `/media` download, PNG magic number verified on a real machine)	🔶 no switch	Needs an `images` switch (when off, keep only the `[图片]` placeholder)
One-to-one images	The same chain, verified on a real machine (69-byte PNG)	🔶 no switch	Same as above
Speech recognition	Implemented, **not verified on a real machine**; occupies the UI for ≤8s each time	🔶 no switch	Needs a `voice` switch, on by default, and acceptance on real voice messages
@ the speaker when replying	**Not implemented** (the sidecar explicitly does not pass `at`)	❌	Doable: a `mentionBack` switch + sidecar `at=[sender display name]`; groups only, and needs real-machine verification
Quote the message when replying	**Not implemented**	❌	**The library does not support it**: no quote parameter. Substitute `quoteStyle: 'text'`, writing the quote into the body
Group keyword replies	Not implemented	❌	Doable, see §5
One-to-one keyword replies	Not implemented	❌	Doable, see §5

SiverWXBot's capability	Current state in this repository	Configurable in the panel?	Conclusion
Outbound images/files	Not implemented (the contract sends text only)	❌	Doable (`SendFiles(filepath, who)` exists), but requires the core `OutboundMessage` to support attachments, which is another order of magnitude
Adding contacts by hand in the panel	Can only pick from chats the bridge has seen	✅ present (candidate dropdown)	There is no real wxid to take, so hand entry cannot be validated; keeping "pick from candidates only" is safer

3. Technical evidence (real-machine verification this round)

```
WeChat.SendMsg(self, msg: str, who: str = None, clear: bool = True,
               at: Union[str, List[str]] = None, exact: bool = False,
               max_retries: int = 3) -> WxResponse
Chat.SendMsg(self, msg: str, who: str = None, clear: bool = True,
             at: Union[str, List[str]] = None, exact: bool = False) -> WxResponse
Chat 公开成员: AddGroupMembers AtAll CancelCall ChatInfo Close EditFriendInfo
              GetAllMessage GetDialog GetNewMessage LoadMoreCache SendAudio
              SendFavorite SendFiles SendMsg SetGroupAnnouncement
              SetGroupMyNickname SetGroupName SetGroupRemark Show VideoCall
              VoiceCall chat_type
```

- One-to-one text: `/messages?room=` returns an event with `room: ""` (already fixed).
- One-to-one images: the event carries `media: [{"id":"#0","kind":"image"}]`, and `/media` returns 69 bytes, `89 50 4e 47` = a valid PNG.
- Repeated `/listen` pushes already return within 0.02s without touching the UI (the repeatedly popping-up contacts window was fixed).

4. Panel field design (proposed additions)

key	type	default	purpose
`rooms`	rows (existing)	`[]`	**Relabel** as "Groups and one-to-one"; picking a group = group chat, picking a person = one-to-one
`atOnly`	switch (existing)	`true`	Reply only when @-ed (groups)
`mentionBack`	switch	`true`	Include @ when replying to whoever @-ed me in a group (**groups only**)

key	type	default	purpose
`quoteStyle`	select `none`/`text`	`text`	There is no real quote API; `text` = note the quoted target in the body prefix
`images`	switch	`true`	Whether to hand images to the model as attachments
`voice`	switch	`true`	Whether to transcribe voice (when off, use the `[语音消息]` placeholder and save UI)
`keywords`	rows `[{ any, note }]`	`[]`	Hitting any keyword counts as an **explicit request** (goes through the existing admission model)
`autoReplies`	rows `[{ any, reply, stopAgent }]`	`[]`	On a hit, **reply with fixed text directly** (without the agent), optionally suppressing the hand-off to the agent at the same time

5. Phasing

Phase	Content	Impact surface	Estimate
P1	Panel wording and hints (groups and one-to-one, explanation of the auto-filled 姓名); wire the `images` / `voice` switches into inbound rendering	`kind.ts`, the branches before the sidecar's `_render_text`/`_to_event`	0.5 day
P2	`mentionBack`: sidecar `send(..., at=[name])` + contract field + Node side passing the sender per the switch + **in-group real-machine verification**	sidecar, `bridge.ts`, `contract.md`	1 day
P3	`keywords` → admission (a hit is explicit); `autoReplies` → the listen loop calls `/send` directly	`kind.ts`'s scope/explicit, `weixin-watch.ts`	1.5 days
Implemented	P1: wording + `voice`/`images` switches	`kind.ts`, sidecar `options`, `verify/options-check.py`	done

Phase	Content	Impact surface	Estimate
Implemented	P2: `mentionBack` (sidecar `at`, contract, panel, `verify/mention-check.py`)	sidecar, `bridge.ts`, `kind.ts`	done (not run in a real group, to avoid speaking in a real group)
Implemented	P3: `keywords` + `autoReplies`	`scope()`, `weixin-watch.ts`, `ports.ts`	done
Implemented	P4: immediate receipt + in-progress reminder (`acknowledge`/`ackText`/`progressAfterSeconds`)	`weixin-watch.ts`'s `pollChat`, `kind.ts`	done
Implemented	P5: listener self-healing (10 minutes with no event → detach then attach, at most once per 15 minutes; `/health` reports "latest event / re-attach count")	`server.py`'s `_reattach_if_quiet`, `_start_listener_watchdog`; `verify/listener-check.py`	done
Implemented	P6: cursor monotonic across restarts (sequence clock seeding + responses carry `head`); when a client cursor is ahead, read from the start	`server.py`, `bridge.ts`, `weixin-watch.ts`	done
—	`quoteStyle: 'text'`	The current state is already "quote written into the body", with no separate switch	not needed
—	Outbound images/files	Requires the core `OutboundMessage` attachment model	separate ADR

5.1 The basis for P4 (measured timeline, from `state/superagent/orchestrator.sqlite`)

Message	System action	Latency
07:14 a "test"	07:14:53 judged ignore	53 seconds, no receipt (correct: no request)
07:14 "make a web page..."	07:16:09 judged accept, and a receipt is sent at the same time	**2 minutes 09 seconds**
07:30 "publish to the public internet..."	07:32:00 merged into an existing ticket, **with no receipt at all**	2 minutes of total silence

Conclusion: the sense of waiting comes from two places — **the assessment itself takes tens of seconds to two minutes**, and **a message appended to an existing ticket carries no receipt**. So the receipt is sent by the source side before the assessment, and an in-progress reminder is

added for "the bot has not spoken in this session yet". Shortening the assessment itself is out of scope for this phase (it involves model choice and prompt size).

6. Risks

87. **The semantic trap of `at=`**: inbound `Message.at` is a "nudge", outbound `SendMsg(at=…)` is a mention — same name, different meaning. P2 must use a dedicated case to prove "after sending, a real @ notification appears in the group and no nudge was triggered".
88. **Speech transcription occupies the UI**: `to_text` seizes the client interface for up to 8 seconds, during which listen callbacks may be delayed. On by default but must be switchable off, and "transcribing" must be exposed in `detail`.
89. **Keyword auto-replies bypass the admission model**: `autoReplies` are sent directly by the listen loop, meaning anyone can trigger a send with one configured phrase. It must be restricted to sessions already configured for this source, and constrained by the sidecar's ≥1s send interval.
90. **Quotes can only be rendered as text**: if a user expects WeChat's native "quote" style, it cannot be done here, and the panel hint must say so.

7. 2026-11-10 wrap-up: receiving is per group, the roster carries no source, sync follows backend capability

After getting the WeChat path working on a real machine, three design points converged from first principles:

- **The source governs perception, the assessment governs authorisation.** The source decides "what is visible": which groups are received, whether one-to-one is received, and how much each group hands over (only messages that `@` the bot, or all messages).

The assessment decides "what counts as work": who may originate, in which scope, and at what strength. So the group list, the one-to-one switch and each group's receive mode live in the source; the roster and the admission table live in the assessment.

- **"Receive only @" sinks down to each group.** A single global switch would express the same thing as "in the roster but not @-ed" in the admission table (with the switch on, those two cells in the admission table are always "—").

It changes to "Receive: only @ / all messages" on the group row, consistent with Feishu's group list; the old `atOnly` degrades to "the default receive mode for new groups", taking effect only when a group has none set, so old configurations behave unchanged.

- **The roster carries no "source" column.** Whether a message is a group message or a one-to-one message is a property of the message itself (whether it has a `room`), and the admission table splits rows accordingly; the roster holds only authorisation,

consistent with Feishu, with just "user ID + name". The provenance marker used by sync (which rows were written by sync) becomes internal data and takes no interface space.

- **Sync works according to the backend's real capability.** wxautox4's `capabilities.members` is `observed`: it has no group member table interface

(`GetAllRecentGroups()` only gives group names and member counts), so `/members` can only return **the people who have spoken in this group**. Measured: `GET /health` → `{"mentions":"markup","members":"observed","media":true}`; `GET /members?room=AI 研究群` → `{"members":[]}`. Therefore on an `observed` backend, sync is a **union** (add only, never delete, so quiet members are not wrongly removed), and it replaces only when a full roster (`full`) is returned; when 0 people are obtained it reports an error explicitly with the reason, and `none` is refused outright.

- **The two schema layers must describe the same shape consistently.** The source's own schema generates the form, and only the settings layer's `AllowedSchema` really validates what is saved; if the two do not match,

a row the panel can produce cannot be saved (this round's real fault: `via` was judged an unknown field by the settings layer, making the whole row invalid and the roster permanently empty). Tests now put every source's part and marked roster rows through the settings-layer schema to catch this.

Evidence surface: `tests/admission-tables.spec.ts` (admission table shape and cross-schema), `tests/weixin-kind.spec.ts` (receiving per group, merge semantics).

ADR-008 WeChat outbound attachments: the bridge can already send them, the model cannot yet use it to send

- **Status**: Accepted and implemented per option A (2026-11-08; bridge layer verified on a real machine, core layer see §6)
- **Decision scope**: whether `OutboundMessage` introduces attachments, who decides which file to send, and the directory/size guardrails
- **Related**: ADR-007 (WeChat capability alignment, whose "outbound images/files" item points to this document)

1. Current state (implemented, verified)

- **Client library**: `wxautox4`'s `Chat.SendFiles(filepath, who=None, exact=False)` and `WeChat.SendFiles(filepath, who=None, ...)`

both exist, no reverse engineering needed; the same family also has `SendAudio` and `SendFavorite`.

- **Bridge**: `POST /send` accepts an optional `file: {name, data(base64)}`, writes a temporary file, calls `SendFiles`, then deletes it; text and a file can be given at the same time (text first, then the file).

Guardrails: ≤ 20 MiB after decoding (413), request body ≤ ~27 MiB (`MAX_SEND_BODY_BYTES`), the filename is reduced to its basename and filtered to `[A-Za-z0-9._-]`.

- **Real machine**: a 2×2 PNG sent through `/send` was rendered by the client as an image message, and the queue echoed `text="图片"`.
- **Tests**: `verify/file-check.py` 11 items (filename escape, upper limit, file only, text + file).

2. The gap: the model cannot use it

`OutboundMessage` only has `id / channel / text / kind / ticketId`, and `Channel.send(host, message)` also only reads `message.text`. So "let the agent reply with an image/a file" is not a bridge problem, but **the core outbound model having no attachments**:

91. `OutboundMessage` needs optional attachments (name + bytes, or an attachment reference);
92. `engine.reply(...)` and the `reply` tool need a parameter (or a separate `send_file` tool) to carry it;
93. Each channel declares whether it supports attachments (`capabilities`); unsupported ones either refuse or degrade to a text explanation;

94. The WeChat channel turns it into `BridgeSend.file`; Feishu/DingTalk/WeCom etc. do their own per their respective APIs.

Estimate: core + WeChat + tests and docs about **1.5–2 working days**; each remaining channel 0.5 day.

3. Two paths, and why the shortcut is not advisable

Option	Approach	Problem
A (recommended) core attachment model	Items 1–4 above; an attachment is a controlled object, each channel implements it	Large change surface, must go through every channel; but semantics are consistent and auditable
B path pass-through shortcut	Give the agent a tool that hands a "local path" straight to the sidecar's `SendFiles`	**This amounts to giving the model a hole through which it can send any local file out** (credentials, source code, meeting notes); and path semantics only hold on the machine where the sidecar runs

4. If taking A, the guardrails that must be fixed at the same time

95. **Source restriction**: only files inside the ticket workspace may be sent (after resolution they must land in that ticket's worktree); symlinks are judged by their resolved real path.
96. **Size and type**: keep 20 MiB; refuse executable extensions (`.exe/.bat/.ps1/...`) by default, requiring an explicit switch.
97. **Count and rate**: keep the sidecar's ≥1 second send interval; set a per-ticket cap (e.g. 20 attachments) to prevent flooding.
98. **Off by default**: a panel switch `sendFiles`, off by default; when turned on, warn that "the model can send files from your workspace into this session".
99. **Auditable**: record an `outbound file` event for every send (name, size, first 8 characters of sha256), into `/health`/the ticket log, for later tracing.

6. What option A has implemented

- `OutboundFile { name, data(base64) }`; `OutboundMessage.files`; base64 rather than bytes, because a reply lands in the outbox before being sent,

and a byte array cannot get through that JSON record.

- `ReplyRequest`'s `answer/result/attention` can carry `files`, and `Replies` hands them to the outbox as-is.

- `SourceChannel.capabilities.files = { maxBytes }`: the channel itself declares whether it can carry files.
- WeChat channel: when the `sendFiles` switch is on, the first file is attached to this reply and the rest are sent separately in order (`/send` takes one file at a time); when off, only text is sent.
- **Location: `orchestrator_reply`'s `files`** (the one for the main Session replying to the person), not a separate worker tool.

The first real-machine version put the tool on the worker side with `purpose` fixed to `result`, and the result was that the worker wanted to send while **in progress**, and both attempts were rejected by "Ticket is not ready for delivery" — that rule exists precisely to stop "sending progress out as a result". Replying to the person is the main Session's business, and the file travels with that reply; the worker-side tool has been deleted, to avoid leading people into the same pit again.

- The gate is centralised in `resolveTicketFile` (with dedicated tests): the real path must be inside the ticket directory (symlinks included), an ordinary file, non-empty,

≤20 MiB, and executable extensions are refused.

- `Runtime.canSendFiles(ticket)`: reads the **panel switch** (off by default) and the channel capability, and the tool asks it before acting.

6.1 Second version: the file comes from the "report", not from the model remembering on the spot

The first version put the tool on the worker side with `result` fixed, and it was rejected as "not ready"; the second version moved it to the main Session's `orchestrator_reply`, but on a real machine the worker was still calling the **deleted** old tool name, got `UNKNOWN_TOOL`, and therefore told the user "this environment has no attachment upload channel" — the capability was there, the tool was not.

So the chain is now: **the worker declares its outputs in the ready report's `files`** (relative to the ticket directory) → the report is stored in the assignment → relayed to the main Session with that list → the result reply **automatically attaches them by default** (an explicit `files` may also be given). The `sendFiles` switch and `resolveTicketFile`'s boundary stay as before. This way, even if the model forgets to say so, the outputs travel with the result; and the model has no tool that "looks usable but always fails".

7. Real-machine lessons (why the first version was wrong)

Symptom	Cause
The worker called `orchestrator_send_file` twice and both failed, with `execute.failed` saying `Ticket is not ready for delivery`	The tool fixed `purpose` to `result`, and `result` requires a verified ready report first; the worker's idea was "send as I go"
WeChat only showed "已打包发到本对话（附件 … zip）", with no attachment	The tool did not succeed; the text was the model's own claim

Conclusion: attachments belong to the **reply**, and the reply is written by the main Session into `orchestrator_reply`; putting it there both conforms to the admission rules and lets "saying it" and "sending it" be decided by the same call.

5. Conclusion

- **Doable**: the library has `SendFiles`, the bridge has implemented it and verified it on a real machine; what remains is the core outbound model and one controlled tool.
- **Not advisable** to patch the feature on with path pass-through first: that turns "the model decides what to send" into "the model can read any local file and send it out", a completely different order of risk from a controlled attachment model.
- Next step: confirm whether to take A; once confirmed I will implement per §4's guardrails, and make "files can only come from the ticket workspace" a test (crossing the boundary must fail).

ADR-009: Projex feedback, attachments, member sync and search — the routes we could reach

Status: accepted (2026-11-13) · Scope: `src/sources/work/**`, `src/infra/yunxiao-api.ts`, `src/app/store-read.ts`

Context

The Projex channel could receive a work item and comment on it, but the three stages of a collaboration were each missing one piece: an assessment of a comment saw only that comment; a worker did not know which item it was on; and the board never moved. Completing them raised four questions whose answers are not obvious from the code, because the official OpenAPI puts some routes out of reach of the client's own safety rules. This ADR records what we call, what we do not, and why.

Decisions

1. A status is configured by name, resolved against statuses this instance has read

Yunxiao publishes a workflow-status list, but the reachable form is `GET /oapi/v1/projex/projects/{projectId}/workitemTypes/{id}/workflows` — **Region edition only**: no organization segment (the client refuses any path without `/oapi/v1/organizations/{whitelisted}/`), and it needs a `workitemTypeId` the adapter does not have. The old `GET /organization/{org}/workitems/workflow/listWorkflowStatus` predates `/oapi/v1/` and the gateway has already retired other old paths (`GET /listWorkitems` answers 404), so it is not a route to build on.

What every work item **does** carry is its current status: `{id, name}`. A poll therefore records the status names and ids it read (`KV.work.statuses`, bounded to 200) and the two write-back fields accept a name or an id. An opaque ASCII token is passed through as an id we have not seen; a human name we have not seen is refused with the place to get one (the source card's **Observed statuses** action, which lists what the last read returned). This trades completeness for reachability: the vocabulary is what the project's recent items show, which is exactly the set an operator would pick from.

2. An attachment is uploaded to the item, then embedded in the comment

`POST /oapi/v1/projex/organizations/{org}/workitems/{id}/attachments` (multipart `file`) is reachable and documented. It returns `embedMarkdown` / `embedUrl` / `url`; the comment body is the only write surface a work channel has, so the upload's Markdown embed (images) or a Markdown link (other files) is appended to the comment text.

Three guards, because this is a channel for files to leave the machine: the worker's `orchestrator_report` already resolves each path inside the ticket directory and refuses executables and files over 20 MiB; the Source's `sendFiles` switch (default **off**) is read again in the channel; and the channel's own 10 MiB cap is enforced in the provider, which names an over-cap file in the comment instead of failing the delivery. Projex's true size limit is unknown, so the host's rejection stays the final word — that failure surfaces through the outbox like any other reply failure.

3. The member list syncs from the project, not the organization

`GET /oapi/v1/platform/organizations/{org}/members` is reachable, but a whole organization is the wrong scope for "who may originate work on this project". The new-style `GET /oapi/v1/projex/organizations/{org}/projects/{id}/members` returns project members with `userId` and `userName`, which is the right set and the right id shape: the Policy compares `allowed[].id` with the work item's `assignedTo`, and in the central edition that is the member `userId`. The assumption is documented rather than proven (no live token here); a mismatch would show up as every item refused as unlisted, which the Policy card already warns about.

The sync is a `policy.sync` on the shared work factory, gated by a new `syncMembers` switch (default **off**), because a sync **replaces** a hand-kept list. Like Feishu's, it applies to the default instance only: `Settings.syncable()` resolves `kind(instance.id)`, which only finds a kind whose id equals its instance id.

4. Search resolves a serial, not a thread

The panel searches triggers and threads; a work item's human number (`ESFX-6228`) is what people type. `projexKind.find` turns a bare serial or a `…/workitem/…` address into a search term, and `Store.read.search` matches `meta.serial` exactly, the same way it already matched a Feishu `meta.messageId`. A work-item page is addressed by serial while a thread is keyed by the internal item id, so this resolution finds the item's triggers and tickets but does not bind a thread from a pasted URL; binding still goes through the conversation id the poll wrote.

Consequences

- Two things stay unverified against a real organization: the status write (id resolution from names is proven, the PUT

is not) and the attachment embed's layout in a RICH TEXT comment. Both are marked as gaps in `EXTENSIONS.md`; the adapter's shapes come from the published request syntax.

- The status vocabulary grows only as items are read. A project whose items have never used a status cannot name it

until one does; typing its id still works.

- `sendFiles` and `syncMembers` are off by default, so upgrading changes no behaviour until an operator asks for it.

ADR-010: Yunxiao sign-in, the real status vocabulary, RICHTEXT attachments, and an independent plugin identity

Status: accepted (2026-11-14) · Scope: `src/infra/yunxiao-oauth.ts`, `src/infra/yunxiao-api.ts`, `src/sources/work/`, **`cordis.patch.yml`**, **`src/client/`**, `package.json`, `locale/**`

Context

Four gaps remained after the work-item feedback work: an operator still pasted a personal access token by hand; the status names came from what recent items happened to show rather than from the project's own workflow; an attachment was pasted into a plain comment body as Markdown, which a rich-text reader does not render; and the bundle kept the official superagent's internal ids, so the two could not be installed in one profile without sharing a panel slot, a preset id or a state lock.

Decisions

1. Sign in through Yunxiao's OAuth 2.0 server, and keep the token file as the credential

Yunxiao documents an OAuth 2.0 authorization server: authorization code, mandatory PKCE S256, public clients only (`token_endpoint_auth_method: none`), dynamic client registration, metadata discovery, opaque tokens (`oat-` 24 h, `ort-` 90 days). That is exactly a desktop plugin's shape, so `yunxiao-oauth.ts` implements it: discover the endpoints, register a client with the loopback redirect, open the browser, take the code on `127.0.0.1:/callback`, and exchange it.

The result is written to the **same owner-only token file** every reader already uses; the refresh token, client id, token endpoint and expiry go in a sibling `.oauth.json`. This keeps one contract for both credential kinds: a personal access token still works (no sibling file, no renewal), and the OAuth path adds renewal. The client renews once after a 401 and retries, so a 24-hour access token does not require a daily visit to the panel; the browser is needed only every 90 days.

Not chosen: DSH's `ctx.credentials` records. They would tie the token to one profile and leave the headless/CLI path with a different mechanism; the file is already how this plugin is

operated, and it keeps the token out of configuration, out of logs and out of the panel. The token never passes through the panel: the browser talks to the loopback listener.

2. The status vocabulary is the project's workflow, not the recent items' statuses

`GetWorkitemWorkflow` has two forms. The Region form (`/oapi/v1/projex/projects/{id}/workitemTypes/{id}/workflows`) has no organization segment, which the client's path confinement refuses by design. The **central** form does: `/oapi/v1/projex/organizations/{org}/projects/{id}/workitemTypes/{typeId}/workflows`, and `ListWorkitemTypes` (`?category=Req|Bug|Task`) supplies `typeId`. So `listStatuses` reads the enabled types per category and each type's statuses, and the source card's action stores them.

The earlier "observed statuses" (statuses seen on polled items) remain as a fallback: a status never used by an item is in the workflow but not observed, and a project whose types cannot be read still resolves what it has read. A name that is in neither place is refused with the action that fills the vocabulary.

3. An attachment comments as RICHTEXT, using the host's own embed

`CreateWorkitemComment` documents `content` plus `formatType` (`RICHTEXT` or `MARKDOWN`, default `RICHTEXT`), and `CreateWorkitemAttachment` returns `embedHtml`, `embedMarkdown`, `embedUrl` and `url`. Pasting `embedMarkdown` into a plain body was the bug: a rich-text reader never renders it.

A comment **with** attachments is now `formatType: 'RICHTEXT'`: the text is escaped into ` blocks (single newlines become ), each upload's` embedHtml (an ` for images, a link otherwise) follows, and the idempotency marker rides as an HTML comment. A comment **without** attachments keeps the plain text it always had, so Markdown the agent writes stays as it was. Reading back, `noteBody` strips HTML when the stored content is HTML, so the model sees text. The marker is detected by the raw payload before any stripping, so a retry still posts once.

4. The plugin gets its own identity, so both superagents can be installed

Bundle entry ids (`dsh-superagent-multi-channel`, `…-`), preset ids (`…-worker`, `…-router`), the panel slot and the locale namespace are now unique to this bundle; `locale/*.json` describe it ("Superagent multi-channel") and `icon.svg` is exported. Two bundles inserting the same row ids, or defining the same preset ids, or registering the same panel slot, collide; with unique ids they coexist, and each instance simply needs its own `stateDir` (one directory admits one instance, and the lock says so).

Kept on purpose because they live **inside this plugin's own state**, invisible to the other bundle: the `dsh-superagent:` message-source kind on delivered messages, the `dsh-superagent-` skill-name prefix, and the `scripts/dsh-superagent` service script. Renaming those would break resumes of existing sessions and existing skills for no coexistence benefit.

Upgrading an existing installation: rename the profile's `dsh-superagent` block to `dsh-superagent-multi-channel` and keep its `stateDir`; the SQLite database, tickets, cursors and settings are all in that directory, so no data migration is needed. Install paths (plugin manager, community market, custom index, tarball, local directory) are in `docs/006-deployment.md`.

Consequences

- The token file now has a second, optional companion file; an operator who copied only the token file keeps working until

the access token expires, and the 401 then asks for a new sign-in instead of renewing.

- The status action makes one request per type per category on demand; a project with many types costs a few calls, never

a poll.

- An attachment is only as good as the host's `embedHtml`; a host that returns none falls back to a link, which is visible

in the comment but is not the host's own preview.

- Tests cover all four: the OAuth flow (including a forged-state rejection and a real loopback callback), the status list,

the RICHTEXT body and its marker detection, and the renamed ids in the bundle patch, presets, client registration and docs. Discovery and dynamic client registration were also exercised against the production central site (`https://openapi-rdc.aliyuncs.com` → an `account-devops.aliyun.com` authorization endpoint, a real `client_id`); the consent step and token exchange need a person, which is what `verify/oauth-check.py` and the panel action provide.

ADR-011: Aligning WeChat with Feishu

Status: accepted (2026-11-14) · Scope: `src/sources/chat/weixin/，**src/app/resources.ts，src/app/prompts.ts，src/app/skill-files.ts，skills/dsh-superagent-weixin/**`

Context

The Feishu channel lets an agent read the original conversation while it works: `feishu//message`, `/root`, `/parent`, `/thread` and `/around`, described by the bundled `dsh-superagent-feishu` skill; its assessment reads `thread/root/parent`; the panel can search by message ID; and a reply's parent is mapped back to the Trigger it answers. WeChat could already receive, judge `@` structurally, send text and files, transcribe voice, sync members, and re-attach its listener, but an agent could not read a WeChat conversation, the panel could not find a message by ID, and a reply chain was not linked.

Decisions

1. The same resource grammar, implemented over the sidecar

`weixin//message`, `/thread` and `/around` mirror Feishu's names and response shapes (`{ status, message }`, `{ status, messages, next_resource }`), so a worker reads either channel the same way and the skills differ only in what the backend can do. `/thread` pages with the bridge cursor; `/around` is the page that holds the Trigger's message (WeChat offers no time-window query, so the holding page stands in for Feishu's five-before/five-after window). Reads are trigger-scoped: `readWeixin` refuses a resource whose trigger id does not match, the same guard Feishu's reader uses.

2. `root` answers `no_root`, because there is no topic

A Feishu message can belong to a topic rooted at an earlier message; a WeChat conversation is one flat stream. Returning the room's first message as "root" would be an invented anchor, and a model told "this is the root" would reason from the wrong message. So `/root` and the `root` thread part answer `no_root`, and `parent` answers the quoted message **only when the bridge reports one**; otherwise `no_parent`. The skill says the same, so a model does not treat a missing root as a failed read.

3. Search by platform message ID; map a reply to its parent

The shared chat kind already turns ID-shaped words into search terms and maps `meta.parentId` to a Trigger ID; the WeChat kind had neither wired. Both are now wired, and `Store.read.search` matches `meta.nativeId` exactly, alongside the Feishu `meta.messageId` it already matched. A pasted message ID therefore finds the Trigger, and a quoted reply shows the same exchange Feishu shows.

4. `readPageSize` and a WeChat skill

`readPageSize` (1–50, the bridge's ceiling) joins the configuration so assessment reads and worker resources page the same size, as Feishu's `readPageSize` does. A bundled `dsh-superagent-weixin` skill is registered in every Session next to the Feishu and GitHub skills; it documents the resources, the `[image:]` markers, and the two things WeChat cannot do.

What stays impossible, and is stated rather than faked

- **No message deep link.** The sidecar exposes none, so `ChannelLink.url` is empty and the panel names the Source. A

fabricated `weixin://` URL would not open the message.

- **No bot list.** Feishu can tell a bot sender by `sender_type`; WeChat has no such concept and the bridge reports

`bot:false` for everyone. A "bots whose messages may become candidates" list would be a control that never matches.

- **No topic root** — see decision 2.

What became possible, and how

- **Image bytes reach a worker.** The skill documents a download: `DSH_SUPERAGENT_WEIXIN_BRIDGE` and

`DSH_SUPERAGENT_WEIXIN_TOKEN_FILE` are exported into the worker's environment, and one `curl`/`Invoke-WebRequest` against `GET /media/` (percent-encoded, bearer token from the 0600 file) writes a file `read_image` opens. That is Feishu's "download with lark-cli, then read it" without shipping a CLI, because the bridge is already the authenticated endpoint. `/media` serialises with `/send`, so downloads stay one at a time, and a 401 is reported rather than guessed.

- **First-sight backfill exists, and is now marked.** The sidecar already reads a chat's last messages back after a re-attach

(or when a chat starts being listened) and queues them; that is WeChat's `backfillChat`, capped at `CATCHUP_MAX_MESSAGES` (20) per chat for at most `CATCHUP_MAX_CHATS` (3) chats a round, because each read opens the client window. Those events now carry `historical: true` (contract §3), and `pollChat` hands them to the assessment without a receipt, a canned reply or a still-working note: old words must be readable but must not be answered. It is not a full history scan - a chat that was never listened has no queue to read - and the skill says so.

Consequences

- A worker can read its ticket's WeChat conversation and page it, which is what Feishu's resource grammar gave.
- Two channel-specific skills now ship; `Prompts.skills` registers `weixin` unconditionally (it needs no bot, unlike Feishu)

and the panel lists its part for overrides.

- The resource route builds the bridge from the instance configuration (`weixinBridge`), so a resource read costs one

sidecar call per page and never starts the bridge.

- Tests cover the reference/grammar, each read shape and its status (`ok`, `no_room`, `not_found`), the ThreadContext parts

(`root`/`parent`/`thread`), search terms and reply mapping.

ADR-012: Aligning WeCom with Feishu

Status: accepted (2026-11-14) · Scope: `src/sources/chat/wecom/，**src/sources/wecom-watch.ts，src/app/resources.ts，skills/dsh-superagent-wecom/**`

Context

The Feishu channel receives messages, carries images, lets an agent read the original message (and its thread/topic), maps a reply to its parent and can be searched by message ID. WeCom had a kind, an XML parser, a sender and a pure callback handler, but **no route mounted it**, so nothing arrived; it carried text only; it had no ThreadContext, no worker read, and no image handling. This ADR closes the gap where the backend can, and states the rest.

Decisions

1. The plugin mounts the callback route itself

`wecom-watch` now registers a public `prefix` route at `/wecom/` on the Host web server. It looks the instance's configuration up in the Source registry, reads the body with a 1 MB ceiling, calls the existing `handleWebhook` (signature → decrypt → parse → dedupe ring per instance), turns each event into a Trigger with `wecomTriggerOf`, and hands it to that kind's `SourceContext.receive`. The reply is the encrypted `success` (or the decrypted `echostr` for URL verification). The route is public because WeCom signs every call; ops points the application's receive URL at it. This is the one place the plugin serves an untrusted request, and a bad signature yields no event.

2. Images travel by `MediaId`, and the assessment can download them

An `image` callback carries a `MediaId` and no text. `parseXml` now returns such an event with `media: [{ id, kind: 'image' }]`; the reader and the ThreadContext render `[image:]`, and the ThreadContext's `image` downloads the bytes with the sender's `media` call (`GET /cgi-bin/media/get`), whose JSON error shape is detected before the body is treated as bytes. The worker sees the marker; the assessment, which holds the instance configuration, attaches the image. Text and other message types are unchanged, and voice/video/file/location still produce no event — stated, not faked.

3. What a callback can and cannot answer

A callback is one message: `thread` is that message, `root` answers `no_root`, `parent` answers `no_parent`, there is no history backfill and no next page. `wecom//message` is the worker read (one

message). Search turns a decimal `MsgId` into a Trigger term, and `repliesTo` exists but a callback carries no quoted ID, so it is normally empty.

What stays impossible on this backend

- **No history.** The receive API is a push callback; there is no read-back endpoint to page a conversation, so no backfill

and no `/thread` pages.

- **No message deep link.** There is no per-message URL; the channel link is empty and the panel names the Source.
- **No chat member roster.** Enumerating a group's members is not available to a self-built application without the customer

contact APIs and permissions that differ per enterprise; equating the app's visible users with a chat's members would be wrong, so no sync is offered.

- **Worker image bytes.** A worker has the marker but not the corp secret, so it cannot call the media API; the assessment

downloads the image instead.

Consequences

- WeCom is functional for the first time: a callback becomes a Trigger, and a reply goes back to the same chat or person.
- Two config fields (`token`, inline `encodingAesKey`) were added for signature verification, both optional so an existing

stored configuration still parses; without `token` every callback is refused, which `facts` and the field hint say.

- Tests cover the image parse, the Trigger address (`chatid` vs `touser`), the resource grammar, and the ThreadContext parts.